

Raven: The Harness of Harnesses for Composable Agentic Intelligence

EverMind AI

<https://evermind.ai/>

<https://github.com/EverMind-AI/Raven>

ABSTRACT

As large language models advance, AI agents are moving beyond isolated, domain-specific tasks toward long-horizon, cross-domain workflows. This transition exposes two challenges: increasing harness complexity makes manual design difficult to scale, while tighter coupling to specific domains limits the generality of a single harness. The central question thus shifts from how to engineer a stronger harness for one domain to how to autonomously construct specialized harnesses, improve them through experience, and orchestrate them across domains. We introduce Raven, *The Harness of Harnesses*, an open-source multi-agent ecosystem that automatically constructs and evolves modular harnesses for specific models and domains, treating each executable model-harness pair as a composable unit of intelligence. To support an *All-Domain Collaboration Network*, its Host Agent decomposes goals, matches subtasks to specialized agents, coordinates execution dependencies, and integrates results, while a host archive and EverOS preserve experience across tasks and Skill Forge makes that experience available as reusable procedures. Our theory establishes sufficient conditions for such composition to expand reliable task coverage beyond that of the available individual agents under a shared resource budget. On complex and long-horizon tasks, Raven significantly outperforms the state-of-the-art agent systems, pushing the frontier of composable agentic intelligence.

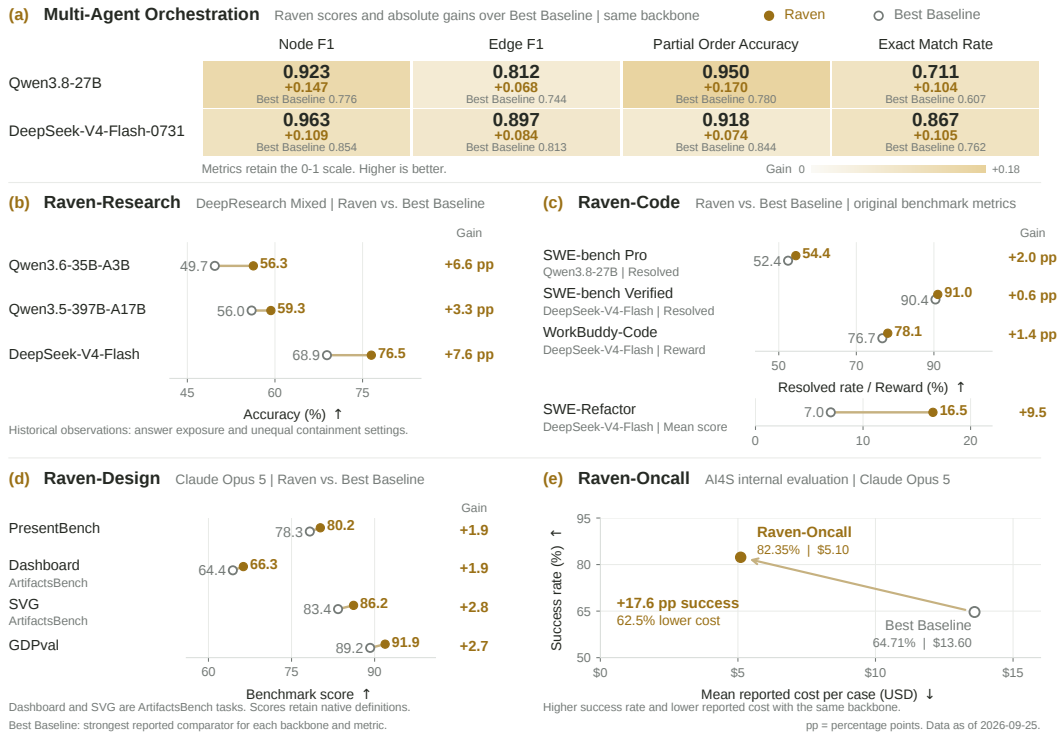


Figure 1: Raven performance overview. Selected results for multi-agent orchestration, research, coding, design, and on-call tasks, compared with the strongest reported alternative in each setting.

Contents

1	Introduction	4
2	Theory	6
2.1	Harnessed Agents and Task-Relative Capabilities	6
2.2	Resource Limits and the Scope of Composition	7
2.3	Host-Mediated Harness Composition	7
2.4	Reliability of Composed Execution	9
2.5	Capabilities Beyond Individual Agents	11
2.6	Reliable Task-Family Coverage and Implications for Raven	12
3	Multi-Agent Collaboration	13
3.1	Task Decomposition and Agent Orchestration	13
3.2	Host-Mediated Graph Execution	15
3.3	Multi-Agent Memory Flow	19
3.4	Shared Group Memory	20
4	Agent Harness Self-Evolution	24
4.1	Modular Harness Design	24
4.2	Diagnosis-Driven Harness Evolution	25
4.3	Harness Gene Bank and Recombination	26
4.4	Gated Harness Screening and Selection	27
5	Memory System	29
5.1	Episodic Trace Formation	30
5.2	Semantic Consolidation	30
5.3	Reconstructive Retrieval	32
5.4	User Context Assembly	33
5.5	Agent Experience and Cross-Session Reuse	34
6	Skill Forge	34
6.1	Curated Skill Catalog	34
6.2	Task-Aware Skill Retrieval and Selection	36
6.3	Experience-Driven Skill Self-Evolution	37
7	Evaluation	40
7.1	Multi-Agent Orchestration	41
7.1.1	The Multi-Agent Orchestration Benchmark	41
7.1.2	Metrics	42
7.1.3	Experimental Setup	44
7.1.4	Results	44

7.2	Harness Self-Evolution	45
7.3	Raven-Research	46
7.4	Raven-Code	48
7.5	Raven-Design	49
7.6	Raven-Oncall	53
7.7	Skill Retrieval and Reuse	54
8	Related Work	56
8.1	Multi-Agent Collaboration and Orchestration	56
8.2	Agent Harnesses and Self-Evolution	57
8.3	Long-Term Memory for Agents	57
8.4	Skill Acquisition, Retrieval, and Evolution	58
9	Conclusion and Future Work	59
A	Additional Theory Details	65
A.1	Notation and Scope	65
A.2	Operation Records and Execution Semantics	65
A.3	A Finite-Budget Information Limit	66
A.4	From Local Experiments to Workflow Reliability	67
A.5	A Typed Task Grammar and Its Compilation	68
B	Method Implementation Details	73
B.1	Notation Across Method Modules	73
B.2	Graph and Prompt Contracts	73
B.3	Implementation Defaults	74
B.4	Harness Evolution Protocol and Evidence	75
B.5	Source Prior for Skill Curation	75
C	Evaluation Details	77
C.1	Multi-Agent Orchestration	77
C.2	Harness Self-Evolution	77
C.3	Raven-Research	77
C.4	Raven-Code	78
C.5	Raven-Design	79
C.6	Raven-Oncall	79
C.7	Skill Retrieval and Reuse	80
D	Author List	82

1 Introduction

Advances in large language models (LLMs), including instruction following [47] and code generation [9], have provided a foundation for agents that interpret user goals and act through software. LLMs can also learn to invoke external tools [55]. Agent systems organize these capabilities into multi-step interactions with external environments. For example, interleaving reasoning with actions and observations allows an agent to gather information and revise its decisions in response to feedback [74]. Specialized execution interfaces further shape what agents can accomplish, as demonstrated in automated software engineering [73]. These developments raise the question of how agents with different expertise and execution mechanisms can work toward a shared goal.

Complex goals often require several specializations within one workflow [17]. Developing and operating a first-person shooter (FPS) game, for example, can involve requirements research, gameplay programming, visual design, integration testing, and sustained operation. Each stage has different tools and completion criteria, and its outputs must support the work that follows. An agent’s practical capability therefore depends on both its model and its *harness*: the tool interfaces, context management, skills, execution policies, and recovery mechanisms surrounding the model. These mechanisms govern how an agent applies its model’s capabilities within a domain [63].

This dependence on the harness creates two related challenges. First, supporting more tools and execution conditions increases the number of interacting design choices that must be configured and validated, making manual harness development difficult to scale across models and domains [56, 24]. Second, specialized harnesses encode assumptions about their tools, working context, and expected outputs. Combining their capabilities requires matching subtasks to suitable executors and establishing compatible handoffs. Analyses of multi-agent failures identify inter-agent misalignment and inadequate task verification as recurring problems [6]. Multi-agent conversation and role-based workflows provide mechanisms for organizing collaboration [69, 22]. However, the benefit of a composition still depends on task structure, local capabilities, and coordination cost [30]. The central problem is thus to construct and improve specialized harnesses while making their capabilities composable across domains.

To address these challenges, we introduce Raven, *The Harness Of Harnesses*, an open-source multi-agent ecosystem that treats each executable model–harness pair as a unit of composition. As illustrated in Figure 2(a), the ecosystem includes the native specialists Raven-Research, Raven-Code, Raven-Design, and Raven-Oncall, alongside independently developed agents such as Claude Code, Codex, Hermes Agent, and OpenClaw. Execution adapters connect these agents to a shared orchestration interface while preserving their tools and internal execution policies. The agent registry describes their capabilities to the Host Agent and can accommodate additional agents as they are integrated.

Within this ecosystem, the Host Agent organizes a collaboration by matching subtasks to registered capabilities and representing their dependencies as a directed acyclic graph (DAG). Each node invokes an agent, and edges identify the dependencies between their outputs and subsequent work. The runtime checks the plan before dispatch, schedules ready nodes, and records artifacts for handoff and reuse. The host handles execution exceptions and integrates the resulting deliverables. In the illustrative FPS workflow in Figure 2(b), a research brief informs parallel gameplay development and visual design. Their outputs then converge for integration and testing before deployment and operation. The graph specifies the inputs each specialist requires and how its output contributes to the requested deliverable.

Beyond coordinating individual workflows, Raven combines harness adaptation with the reuse of experience across tasks. Prior methods adapt agents by retaining insights from past executions [79] or distilling interactions into reusable skills [80]. In Raven, modular harnesses expose execution policies that an external evolver can modify within defined boundaries. Building on our prior work, HarnessBank [39], the evolution process diagnoses failures, proposes candidate harnesses, and evaluates their behavior with a frozen task model. Persistent memory complements these policy changes. A host archive retains user context, while the optional EverOS backend [23, 15] provides semantic access to past experience. For procedural reuse, Skill Forge retrieves task-relevant procedures from local skills, memory-derived skills, and SkillHub, whose corpus and retrieval design build on SkillCorpus [64].

Alongside this system design, our theoretical analysis characterizes when composition can extend the capabilities of the available agents. For a specified agent pool and task family, we formalize capability

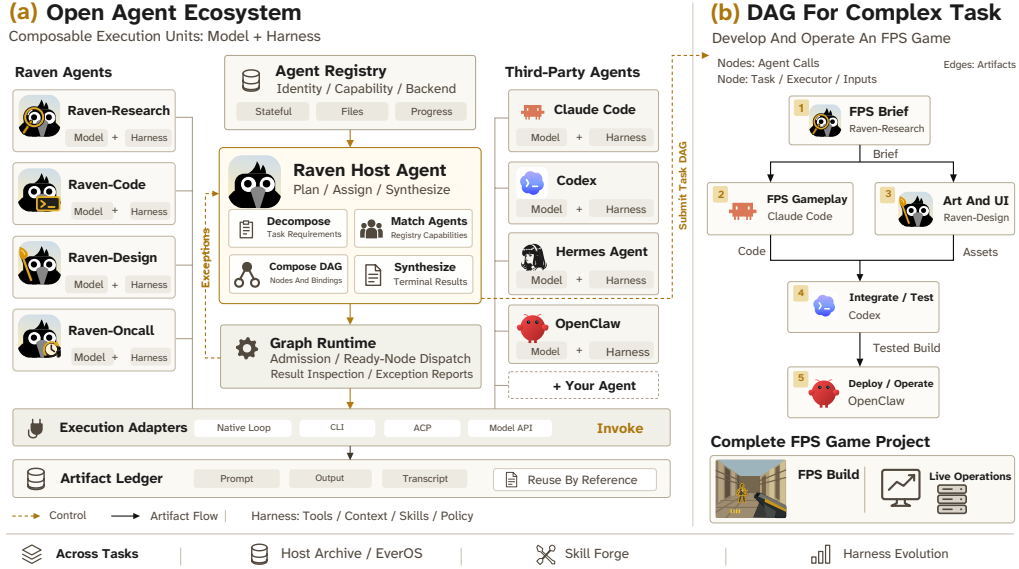


Figure 2: **Raven As An Open Multi-Agent Ecosystem.** (a) Built-in and third-party agents expose executable model–harness pairs through execution adapters. The Host Agent uses registry capabilities to compose and assign tasks, while the runtime validates the plan, schedules ready nodes, and records artifacts for handoff and reuse. (b) An illustrative FPS game project combines gameplay development and visual design before integration, testing, and operation. Nodes denote agent calls, and directed edges denote artifact dependencies coordinated by Raven. Persistent context, reusable skills, and harness evolution support adaptation across tasks.

as reliable task coverage under a common resource budget. We establish sufficient conditions under which complementary local capabilities, compatible handoffs, and bounded planning and execution errors permit the composed system to solve tasks that the available individual agents cannot reliably solve alone under the same budget. The analysis accounts for the cost of planning and coordination alongside worker execution.

To assess Raven empirically, we evaluate planning quality, specialist execution, harness adaptation, and skill reuse separately. We introduce the Multi-Agent Orchestration Benchmark (MAOB), which measures specialist selection and dependency prediction by comparing proposed DAGs with reference graphs before worker execution. Raven ranks first among the compared systems on all four graph metrics under both tested backbones, with Exact Match gains of 10.4 and 10.5 percentage points over the strongest baseline for the two backbones, respectively. Domain-specific evaluations characterize the four native specialists, while the published HarnessBank experiments assess the evolution method with a frozen backbone. The published SkillCorpus experiments show that a curated skill library improves Raven on three benchmarks, with larger gains than OpenClaw on two of them.

In summary, our main contributions are:

- **An Open Ecosystem For Harness Composition.** We present an architecture that coordinates native and third-party agents through a shared interface and explicit execution dependencies, with modular harness evolution, persistent memory, and skill reuse (Sections 3 to 6).
- **A Theory Of Composable Agentic Intelligence.** We formalize task-relative capabilities and sufficient conditions for reliable composition and expanded task coverage under a shared resource budget (Section 2).
- **A Benchmark And Evaluation Across Capability Levels.** We introduce MAOB to evaluate specialist selection and dependency prediction, and organize empirical evidence across orchestration, the four native specialists, harness adaptation, and skill reuse (Section 7).

2 Theory

Raven composes executable model–harness pairs through a host that controls their assignment, information exchange, and execution. To analyze this composition, we formalize *composable agentic intelligence* as reliable task coverage under a common resource budget. The analysis begins with typed execution contracts, establishes deterministic composition soundness, and then derives probabilistic reliability bounds and conditions for capability beyond individual agents. A final result addresses coverage of a task family specified independently of observed system successes. These are sufficient conditions for reliable composition, and their practical applicability depends on whether the implemented agents and host satisfy them. Figure 3 illustrates the composition. Table 6 collects the principal notation, while Appendix B.1 states the notation conventions of the method chapters.

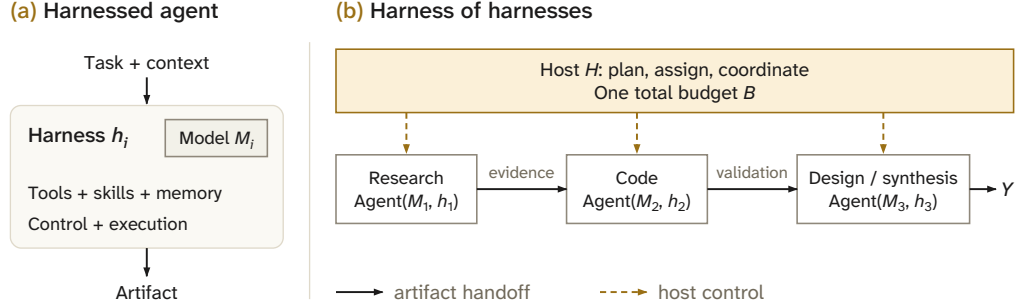


Figure 3: Harness composition. (a) A model and its harness form a callable execution unit. (b) A host assigns and coordinates these units. Solid arrows carry artifacts and dashed arrows denote control. The research–code–design chain is an illustrative plan with explicit final synthesis by a specialist. All operations share one budget. Model labels need not identify different foundation models.

2.1 Harnessed Agents and Task-Relative Capabilities

A harness turns a model into an interactive execution policy. Let M_i be a model and h_i its tool interfaces, context and memory mechanisms, skills, control rules, and execution interfaces. The induced agent is

$$a_i = \text{Agent}(M_i, h_i) : \mathcal{H}_i \longrightarrow \Delta(\mathcal{U}_i), \quad \mathcal{A} = \{a_0, a_1, \dots\}, \quad 2 \leq |\mathcal{A}| < \infty. \quad (1)$$

Here $\mathcal{H}_i$ is the space of the agent’s permitted observation histories, including its request and local memory. $\mathcal{U}_i$ contains tool actions, artifact-return actions, and an abort action. Δ denotes probability distributions. The construction Agent therefore defines a stochastic policy without restricting the model’s internal parameterization. Although the analysis describes the full environment state, the policy has access only to its permitted observation history.

The executor pool includes a_0 , the host’s callable policy with cross-roster delegation disabled. The controller H is a stochastic policy on its permitted history, with additional actions for plan submission and invoking pool executors. The notation $S_H = H[\mathcal{A}]$ denotes the interaction of these policies through the execution protocol below. A synthesis node can invoke a_0 or a specialist. Each a_i is also a standalone comparator, run on the original request with its native interfaces and without calls to other roster members. Models, harness versions, and initial memory-state laws are fixed for a comparison. Several agents may share a model, and one agent may supply several worker instances.

A task is a specification of inputs, interaction, and correctness: $t = (r_t, \mu_t, E_t, V_t) \in \mathcal{T}$. The request r_t is observable. $\omega \sim \mu_t$ takes values in the reference space $\mathcal{W}$ and represents the initial input and environment conditions. E_t gives the distribution of the next observation and environment state conditional on the joint interaction history and a tool action. $V_t : \mathcal{W} \times \mathcal{Y}_{\text{rec}} \rightarrow \{0, 1\}$ judges a record $Y \in \mathcal{Y}_{\text{rec}}$. The record includes delivered artifacts and relevant environment changes. Agent initial-state laws belong to the fixed system configuration. A fixed benchmark instance corresponds to a degenerate input law. For an aborted or nonterminating run, set $Y = \perp$ and $V_t(\omega, \perp) = 0$, with the distinguished failure record $\perp$ included in $\mathcal{Y}_{\text{rec}}$.

We use countable spaces of finitely encoded observations, plans, artifacts, and runtime states, including the represented reference conditions. Whenever an outcome or padded runtime record can equal $\perp$, its space includes that sentinel. State and ledger predicates are extended to be false on padded arguments. We write $\mathbb{I}[\mathcal{E}]$ for the indicator of an event or predicate $\mathcal{E}$. For an event $\mathcal{E}$, $\mathcal{E}^c$ denotes its complement. Together, the task, system policies, scheduler, and budget convention induce a probability law $\mathbb{P}_{t,S,B}$ on the trace space Ω_{tr} , for $S = S_H$ or a standalone a_i . The measurable trace function Y_S extracts the delivered record, or $\perp$ on failure. The nonnegative, possibly infinite total cost C_S charges every operation exactly once using a fixed additive measure, such as total inference and tool cost. Capability is defined by

$$p_S(t; B) = \mathbb{P}_{t,S,B}(\mathbf{V}_t(\omega, Y_S) = 1, C_S \leq B), \quad B \geq 0, \quad (2)$$

$$C_S(B, \delta) = \{t \in \mathcal{T} : p_S(t; B) \geq 1 - \delta\}, \quad 0 \leq \delta < 1. \quad (3)$$

For S_H , cost includes planning, workers, handoffs, runtime verification, and retries. Nontermination is unsuccessful even if its accrued monetary cost is finite. Elapsed time can be a separate constraint, but it is not added to tokens or monetary cost. Objective correctness $\mathbf{V}_t$ remains distinct from an agent or judge declaring completion.

2.2 Resource Limits and the Scope of Composition

Resource limits make capability task-dependent. No-free-lunch results equate search performance measured from sampled objective values for non-revisiting algorithms at a fixed evaluation budget, under uniform averaging over functions between finite input and value sets [68]. Structured task distributions can instead favor appropriately matched prior knowledge, and enough queries permit exhaustive search. The finite-query argument in Appendix A.3 shows that $n_{\text{qry}} < N_{\text{pos}}$ black-box queries cannot guarantee finding every target among N_{pos} positions. It applies equally to an individual agent and a multi-agent system with the same total information budget.

Empirical limitations motivate examining the particular agents available for composition. AgentBench identifies deficiencies in long-term reasoning, decision-making, and instruction following among its evaluated systems [38]. Such results support capability profiling on the intended task distributions, with conclusions limited to the evaluated systems. Routing selects an agent for a complete request, whereas our composition analysis concerns the distinct local contracts needed within one request. Definition 3 formalizes local complementarity after introducing contracts and their realization. A comparison with complete single-agent executions remains a separate step. Individual limitations alone do not imply complementary strengths.

2.3 Host-Mediated Harness Composition

A plan specifies an executable graph together with its semantic and resource obligations:

$$\pi = (G, \sigma, \Phi, \Gamma, \lambda, \mathbf{b}), \quad G = (V, E), \quad \sigma : V \longrightarrow \mathcal{A}. \quad (4)$$

The finite DAG G has a nonempty node set V of worker calls and a set of handoff edges E . The assignment σ selects an executor, including a_0 , for each call. The annotation Γ contains contracts, handoff providers, invocation bindings to states and instances, and the invariants defined below. An analytical rule fixed before execution may supply the contracts and invariants without requiring runtime proof certificates. This rule depends only on the task, planning record, and submitted plan. The schedule is a bijection $\lambda : \{1, \dots, K_{\text{op}}\} \rightarrow V \sqcup E$, with $K_{\text{op}} = |V| + |E|$ and $\lambda_j = \lambda(j)$. It places u before $e = (u, v)$ and e before v . The disjoint union $\sqcup$ distinguishes node identifiers from edge identifiers. The last operation is the plan's designated output node.

Artifacts are stored in an append-only ledger ℓ , separate from the mutable state $s \in \mathcal{S}$. The initial ledger retains the task packet and prior execution record, including planning actions, so the final record can account for the complete run. Only the authorized packet and declared messages become worker inputs. The state includes relevant environment and agent-local state and has a read-only reference projection $\text{ref} : \mathcal{S} \rightarrow \mathcal{W}$, equal to the sampled ω along an actual trace. Let $\mathcal{X}_v, \mathcal{Y}_v$ be a node's input and output spaces, $\mathcal{M}_e$ an edge's message space, and $\text{In}(v)$ its incoming edges. For an authorized task/context packet d in the encoded-packet space $\mathcal{D}_{\text{pkt}}$, the prescribed input is

$$x_v = \Phi_v(d, (m_e)_{e \in \text{In}(v)}), \quad \Phi_v : \mathcal{D}_{\text{pkt}} \times \prod_{e \in \text{In}(v)} \mathcal{M}_e \longrightarrow \mathcal{X}_v. \quad (5)$$

At a source, Φ_v uses only d . Edge transfers first produce messages under their contracts, and Φ_v then assembles those messages into the receiving node's input. Edge transformations and transport are charged to the edge, while receiver assembly is charged to the node. Both stages may use typed projections or serialization. Other computation requires an explicit node, so the accounting covers each operation once.

To specify correctness for these operations, we define node and transfer contracts as typed predicates:

$$P_v : \mathcal{X}_v \times \mathcal{S} \rightarrow \{0, 1\}, \quad Q_v : \mathcal{X}_v \times \mathcal{S} \times \mathcal{Y}_v \times \mathcal{S} \rightarrow \{0, 1\}, \quad (6)$$

$$J_e : \mathcal{Y}_u \times \mathcal{S} \times \mathcal{M}_e \times \mathcal{S} \rightarrow \{0, 1\}, \quad e = (u, v). \quad (7)$$

$P_v(x, s)$ specifies the precondition, $Q_v(x, s, y, s')$ specifies the output and state transition, and $J_e(y, s, m, s')$ specifies faithful transfer and its state effects. Predicates may inspect the analytical reference component of s , although executors cannot. In particular, a judge's acceptance is not the definition of Q_v .

For operation λ_j , define its entry predicate $A_j(s, \ell)$ and transition predicate $D_j(s, \ell, s', \ell')$. A node entry requires its incoming messages and $P_v(x_v, s)$. Its transition requires the recorded actual input to equal the prescribed input, a fresh output record, and $Q_v(x_v, s, y_v, s')$. A handoff entry requires its source artifact. Its transition requires a fresh message record and $J_e(y_u, s, m_e, s')$. Both transitions preserve all earlier ledger records. Missing or ill-typed operation records make the corresponding transition predicate false. The ledger stores actual inputs as well as outputs. Appendix A.2 gives the exact predicates and their behavior under ledger extension. A plan is well-typed when its input, output, and message records have the declared types and all its identifier references resolve.

Beyond type correctness, composition requires compatibility that preserves the assertions needed by later operations, including assertions about mutable state. Fix a set $\mathcal{I} \subseteq \mathcal{S} \times \mathcal{L}$ of admitted initial state–ledger pairs, where $\mathcal{L}$ is the ledger space. The annotations are instantiated for the task and authorized packet. They provide predicates $I_j : \mathcal{S} \times \mathcal{L} \rightarrow \{0, 1\}$ for $j = 0, \dots, K_{\text{op}}$ and a specified projection $\text{rec}_\pi(\ell, s)$ into the final execution record. The projection serializes actual stored artifacts, actions, and state, including the planning record, without computing an additional artifact.

Definition 1 (Compatible plan). A well-typed plan is compatible with task t and initial set $\mathcal{I}$ if I_0 holds throughout $\mathcal{I}$ and the following implications hold for every well-typed state and ledger transition:

$$I_{j-1}(s, \ell) \implies A_j(s, \ell), \quad (8)$$

$$I_{j-1}(s, \ell) \wedge D_j(s, \ell, s', \ell') \implies I_j(s', \ell'), \quad (9)$$

$$I_{K_{\text{op}}}(s, \ell) \implies \forall t (\text{ref}(s), \text{rec}_\pi(\ell, s)) = 1. \quad (10)$$

The first two conditions apply to $j = 1, \dots, K_{\text{op}}$.

The enabling implication checks a join's inputs jointly, while preservation maintains the assertions needed later. This induction uses Hoare-style assertions [20]. Local assumptions and guarantees support componentwise reasoning [32]. We prove the execution result for the serial order λ . It also applies to parallel executions with a certified linearization preserving observations, records, state effects, acceptance, and charged cost (Appendix A.2). A DAG or disjoint output filenames alone do not establish this property.

The allocation vector $\mathbf{b} = (b_H, (b_v)_{v \in V}, (b_e)_{e \in E})$ has nonnegative entries for planning, node calls, and transfers. Resource feasibility means

$$b_H + \sum_{v \in V} b_v + \sum_{e \in E} b_e \leq B. \quad (11)$$

The execution protocol attempts $\lambda_1, \dots, \lambda_{K_{\text{op}}}$ in order. Write (Σ_j, L_j) for the random state and ledger after operation j , and C_j for its cost. (Σ_0, L_0) is the post-planning entry. An operation succeeds if it finishes, is accepted, satisfies D_j , and respects b_{λ_j} . Records after abandonment are padded with $\perp$. Node costs include assembly, verification, continuations, and assigned control work. The last operation also includes serialization and delivery. Its success means that $\text{rec}_\pi(L_{K_{\text{op}}}, \Sigma_{K_{\text{op}}})$ has been delivered. After all K_{op} successes the system terminates with

$$Y_{SH} = \text{rec}_\pi(L_{K_{\text{op}}}, \Sigma_{K_{\text{op}}}), \quad C_{SH} = C_H + \sum_{j=1}^{K_{\text{op}}} C_j,$$

where C_H is the incurred planning cost. Final delivery is charged to the last operation, and a delivery failure counts as failure of that operation.

Lemma 1 (Deterministic composition soundness). *Starting from $\mathcal{I}$, success of the first j operations of a compatible plan establishes I_j and, for $j < K_{\text{op}}$, enables A_{j+1} . If every operation succeeds, planning costs at most b_H , and Eq. (11) holds, the protocol delivers a correct record within B and terminates.*

Proof. I_0 holds initially. Given I_{j-1} , Eq. (8) establishes the next operation's precondition. Its successful transition satisfies D_j , so Eq. (9) establishes I_j without assuming any later success. Equation (8) then gives A_{j+1} when $j < K_{\text{op}}$. At $j = K_{\text{op}}$, Eq. (10) proves correctness. The terminal protocol delivers that record and stops. Summing the successful operations' costs and planning cost proves the budget claim. $\square$

In the research-code-design example, the ledger preserves the scoped claim, evidence, implementation, and test outcomes. An invariant before design records which claims were validated. The design contract must preserve that distinction in the final presentation. Passing a file path alone does not establish the enabling or terminal implications.

2.4 Reliability of Composed Execution

The soundness result assumes that each operation succeeds. To account for execution failures, we define local capability through contract realization in a specified invocation context. A provider κ is a worker call or handoff implementation, including its assembly, control, and acceptance procedures. It has a conditional outcome kernel $K_\kappa(o \mid c_{\text{cfg}})$ on encoded configurations $c_{\text{cfg}} \in \mathcal{Z}_\kappa$ and outcomes $o \in \mathcal{O}_\kappa$. Configurations contain the entry state, ledger, input, and relevant prior or latent information. Outcomes contain the resulting state and ledger, acceptance flag, and cost, or $\perp$ for noncompletion. The kernel may depend on full history through c_{cfg} , without assuming memoryless agents. An entry law ν is a distribution over $\mathcal{Z}_\kappa$. Define the local experiment by

$$\Pr_{\nu, \kappa}(\text{ok}_{D,b}) := \sum_{c_{\text{cfg}}, o} \nu(c_{\text{cfg}}) K_\kappa(o \mid c_{\text{cfg}}) \text{ok}_{D,b}(c_{\text{cfg}}, o). \quad (12)$$

Here $\text{ok}_{D,b}(c_{\text{cfg}}, o)$ equals one exactly for accepted completion in finite time satisfying transition predicate D at cost at most b , and equals zero otherwise. Analytical information in c_{cfg} does not expand an executor's permitted observations.

Definition 2 (Contract realization). For a nonempty family $\mathfrak{L}$ of entry laws supported on entry predicate A , write $\kappa \models_{\mathfrak{L}} (A, D; b, \epsilon)$, with $b \geq 0$ and $\epsilon \in [0, 1]$, if

$$\inf_{\nu \in \mathfrak{L}} \Pr_{\nu, \kappa}(\text{ok}_{D,b}) \geq 1 - \epsilon. \quad (13)$$

Applying this definition to a workflow requires the local experiment to match the actual conditional execution, as established below. An isolated benchmark average does not establish that relationship.

To specify these contexts, fix $t, B, H, \mathcal{A}$ and abbreviate $\mathbb{P} = \mathbb{P}_{t, S_H, B}$. Let Z be the finite planning transcript, including observations and incurred cost, and let Π be the selected annotated plan. If planning is abandoned without an execution plan, set $\Pi = \perp$. Nonterminating planning also gives $Z = \perp$. A direct fallback is instead represented as a one-node plan for a_0 , with earlier planning costs retained. Both variables have countable ranges. For a positive-probability pair $(Z, \Pi) = (\zeta, \pi)$, d_ζ is determined by the transcript and $\mathcal{I}_{\zeta, \pi}$ is the support of the conditional entry (Σ_0, L_0) . A pair is *valid* if its planning cost is at most b_H , the plan satisfies Eq. (11), its protocol has the specified serial or certified parallel semantics, and it is compatible with $\mathcal{I}_{\zeta, \pi}$. The compatibility premise must hold for every entry in this conditional support. The valid-planning event

$$G_t = \{(Z, \Pi) \text{ is a valid planning pair}\} \quad (14)$$

is (Z, Π) -measurable, that is, determined by the planning variables. Validity is an analytical property of a planning pair and need not be decidable by the runtime.

For a fixed valid pair, let Ξ_j, O_j be operation λ_j 's actual configuration and outcome. Its provider, law family, allocation, and error bound are $(\kappa_j, \mathfrak{L}_j, b_{\lambda_j}, \epsilon_j)$. Node and edge subscripts refer to

the same objects through $j = \lambda^{-1}(v)$ or $j = \lambda^{-1}(e)$. Let R_{j-1} be the host-visible execution record: messages, returned artifacts, statuses, and observed costs, excluding hidden reference truth. For parallel execution it is the corresponding prefix of the certified serialization, not necessarily a wall-clock prefix. Define the success events

$$\text{Ok}_j = \{\text{ok}_{D_j, b_{\lambda_j}}(\Xi_j, O_j) = 1\}, \quad F_0 = \Omega_{\text{tr}}, \quad F_j = \bigcap_{k=1}^j \text{Ok}_k.$$

A skipped or nonterminating operation does not satisfy Ok_j . The reliability premise for each positive-probability successful prefix is

$$\mathbb{P}(\text{Ok}_j^c \mid F_{j-1}, Z = \zeta, \Pi = \pi) \leq \epsilon_j. \quad (15)$$

$\epsilon_j \in [0, 1]$ may depend on t, B, ζ, π . Those arguments are suppressed in the local bounds.

We call the selected plan *locally covered* when each provider realizes its contract and, conditional on each $(R_{j-1} = \varrho, F_{j-1}, Z = \zeta, \Pi = \pi)$ with record value ϱ , its entry law belongs to $\mathfrak{L}_j$ and its outcome kernel is $\mathcal{K}_{\kappa_j}$. The kernel-consistency condition and averaging proof are stated in Lemma 2. That lemma derives Eq. (15) from local coverage. The condition retains F_{j-1} because visible records do not necessarily reveal objective correctness. It permits correlated errors through the conditional entry laws and kernels.

All conditional quantities below refer to the same actual system law:

$$\mathbf{S}_t = \{\mathbf{V}_t(\omega, Y_{S_H}) = 1, C_{S_H} \leq B\}, \quad q_t(\zeta, \pi) = \mathbb{P}(\mathbf{S}_t \mid Z = \zeta, \Pi = \pi). \quad (16)$$

Thus q_t accounts for incurred planning cost and the input distribution selected by the host, rather than a fresh standalone execution of π .

Theorem 1 (Reliability of harness composition). *For a valid planning pair (ζ, π) satisfying Eq. (15), define*

$$\epsilon(\zeta, \pi) = \sum_{j=1}^{K_{\text{op}}} \epsilon_j = \sum_{v \in V} \epsilon_v + \sum_{e \in E} \epsilon_e. \quad (17)$$

Then the conditional success probability satisfies

$$q_t(\zeta, \pi) \geq [1 - \epsilon(\zeta, \pi)]_+, \quad [x]_+ = \max\{x, 0\}. \quad (18)$$

Proof. By Lemma 1, $F_{K_{\text{op}}}$ implies $\mathbf{S}_t$ under the conditional law. The disjoint events $F_{j-1} \cap \text{Ok}_j^c$ partition $F_{K_{\text{op}}}^c$. For each positive-probability prefix,

$$\begin{aligned} \mathbb{P}(F_{j-1} \cap \text{Ok}_j^c \mid Z = \zeta, \Pi = \pi) \\ = \mathbb{P}(F_{j-1} \mid Z = \zeta, \Pi = \pi) \mathbb{P}(\text{Ok}_j^c \mid F_{j-1}, Z = \zeta, \Pi = \pi) \leq \epsilon_j. \end{aligned}$$

Zero-probability prefixes contribute zero. Summing gives $\mathbb{P}(F_{K_{\text{op}}}^c \mid Z = \zeta, \Pi = \pi) \leq \epsilon(\zeta, \pi)$. Taking the complement and using nonnegativity proves the bound. No independence assumption is required. $\square$

The preceding bound conditions on a valid planning pair. We next account for the host's probability of selecting such a pair.

Proposition 1 (Host-level reliability). *Suppose $\mathbb{P}(\mathbf{G}_t) \geq 1 - \eta_H(t; B)$, where $\eta_H(t; B) \in [0, 1]$. On every positive-probability valid planning pair, assume the local premises of Theorem 1 and the uniform envelope*

$$\min\{1, \epsilon(\zeta, \pi)\} \leq \bar{\epsilon}(t; B) \leq 1. \quad (19)$$

Clipping limits the accumulated error bound to the range of a probability. Then

$$p_{S_H}(t; B) \geq (1 - \eta_H(t; B))(1 - \bar{\epsilon}(t; B)). \quad (20)$$

Proof. For positive-probability planning pairs use q_t from Eq. (16). A pair with $\Pi = \perp$ has zero success. Completed but invalid plans retain their actual conditional success probability. Since $\mathbf{G}_t$ is (Z, Π) -measurable, conditional expectation gives

$$\begin{aligned} p_{S_H}(t; B) &\geq \mathbb{P}(\mathbf{S}_t \cap \mathbf{G}_t) = \mathbb{E}[\mathbb{I}[\mathbf{G}_t] q_t(Z, \Pi)] \\ &\geq (1 - \bar{\epsilon}(t; B)) \mathbb{P}(\mathbf{G}_t) \geq (1 - \bar{\epsilon}(t; B))(1 - \eta_H(t; B)). \end{aligned}$$

The second inequality uses Theorem 1 uniformly over valid pairs. It makes no independence claim about planning and execution. $\square$

The bound requires a compatible plan, a host that can find such a plan within its budget, and providers that realize their contracts in the selected contexts. Additional nodes may reduce subtask difficulty while increasing handoff cost and error. Internal continuations are included in a node’s realization. Recovery on failed paths preserves this lower bound if the first plan’s all-success path still delivers and terminates within budget. Quantifying additional recovery gains requires accounting for the later plans and reached states.

2.5 Capabilities Beyond Individual Agents

To examine capability gains from reliable composition, we first define local complementarity before selecting executors. A *decomposition* fixes $(G, \Phi, \Gamma, \lambda, \mathbf{b})$, omitting σ , together with the node law families and error allocations. Let $\kappa_{v,i}$ be the bounded invocation of a_i at node v , including its prescribed input bindings and acceptance procedure. Providers compared at this node use a common encoded configuration space $\mathcal{Z}_v$ for the full state, ledger, input, and invocation bindings. Each provider retains its own permitted-observation projection and outcome kernel. Thus $\mathcal{L}_v$ is a family of laws on the same domain for every candidate provider. Define the eligible realizers by

$$\mathcal{R}_v = \{a_i \in \mathcal{A} : \kappa_{v,i} \models_{\mathcal{L}_v} (A_v, D_v; b_v, \epsilon_v)\}. \quad (21)$$

Here (A_v, D_v) are the operation predicates for node v , as indexed earlier by its position j in λ .

Definition 3 (Decomposition-level complementarity). The decomposition has complementary local capabilities if

$$\mathcal{R}_v \neq \emptyset \quad \text{for every } v \in V, \quad \bigcap_{v \in V} \mathcal{R}_v = \emptyset. \quad (22)$$

The first condition ensures that each node has an eligible executor. The second excludes assigning the same eligible executor to every node under the specified allocations. Proposition 3 connects eligible assignments to the reliability theorem, provided transfer contracts, compatibility, and actual invocation-law consistency also hold. However, a different single-agent strategy may still solve the complete task. Hence an independent upper bound on individual end-to-end performance is needed to establish nonempty *new-task coverage*,

$$\mathcal{C}^{\text{new}}(B, \delta) := \mathcal{C}_{S_H}(B, \delta) \setminus \bigcup_{a \in \mathcal{A}} \mathcal{C}_a(B, \delta).$$

Corollary 1 (Capability beyond individual agents). *For fixed B, δ , suppose a task t satisfies the hypotheses of Proposition 1 and*

$$(1 - \eta_H(t; B))(1 - \bar{\epsilon}(t; B)) \geq 1 - \delta > \max_{a \in \mathcal{A}} p_a(t; B). \quad (23)$$

Then

$$t \in \mathcal{C}^{\text{new}}(B, \delta). \quad (24)$$

Proof. Proposition 1 and the first inequality place t in the system’s capability set. The strict second inequality excludes it from every individual capability set by Eq. (3). $\square$

An Explicit Complementary Construction. Let U, W be independent fair bits. In the task $t = (r_t, \mu_t, E_t, V_t)$, μ_t is their uniform law, r_t asks for their parity without revealing them, E_t answers permitted bit queries deterministically, and V_t checks that the returned bit equals $U \oplus W$. Take $\mathcal{A} = \{a_0, a_1, a_2\}$. The native read interface of a_1 exposes only U , that of a_2 only W , and a_0 has neither interface. The original request contains neither bit. For individual executions, let all initial auxiliary information and decision randomness be included in ξ_i , with (ξ_0, ξ_1, ξ_2) independent of (U, W) . The available transcripts are

$$T_0 = f_0(\xi_0), \quad T_1 = f_1(U, \xi_1), \quad T_2 = f_2(W, \xi_2). \quad (25)$$

Each returned answer $\hat{Y}_i$ is a function of T_i . Any additional decision randomness is already in ξ_i . Conditional on (U, ξ_1) , W remains fair, hence

$$\mathbb{P}_{t,a_1,B}(\hat{Y}_1 = U \oplus W \mid U, \xi_1) \leq \frac{1}{2}. \quad (26)$$

Equality holds for a bit-valued answer. An invalid answer or abort cannot improve it. Averaging, and applying the symmetric argument to a_2 and the no-input argument to a_0 , bounds every $p_{a_i}(t; B)$ by $1/2$.

Specify the local policies constructively: on a read-and-report call, a_1 returns U and a_2 returns W exactly. On a call supplied with two bits, a_0 returns their parity exactly. These input-dependent policies are fixed before evaluation. A three-node plan with two exact-copy handoffs therefore satisfies the typed contracts with zero error. Its fixed host schedule has zero planning error. Index the three nodes 1, 2, 0 by their executors. Choose finite allocations that bound their actual calls, handoffs, final delivery, and planning cost. If $b_H + b_1 + b_2 + b_0 + b_{(1,0)} + b_{(2,0)} \leq B$, then $p_{S_H}(t; B) = 1$, and Corollary 1 applies for $\delta < 1/2$. Here b_0 is the synthesis-node allocation. Under zero-error read contracts, the same missing-bit argument makes the read- U and read- W eligible sets disjoint, while the respective copying policies realize them. Together with a_0 at synthesis, this also witnesses Definition 3. The same initial input law and budget are used for every comparator. The gain comes from making complementary interface results available through costed messages. A single agent equipped with both read interfaces could also solve the task. This construction does not assert superiority over that different harness.

Although this result establishes *new-task coverage*, it does not establish strict set inclusion or an improvement in average performance. Retaining the whole union of individual capability sets additionally requires feasible single-agent execution paths and a selector that preserves their reliability. Empirical tests must therefore include strong individual-agent baselines with comparable models, tools, initial information, and total cost. The research–code–design workflow is a candidate for such a comparison, not evidence of a gain by itself.

2.6 Reliable Task-Family Coverage and Implications for Raven

Extending the task-level analysis to a task family requires specifying that family independently of the system’s successes. Appendix A.5 defines a typed grammar of primitive, sequential, and fork–join tasks, gives their relational semantics, and constructs compatible finite DAGs by structural induction. Forking, joining, and transferring artifacts are explicit costed operations. The construction proves plan existence. Its compiler-based specialization (Corollary 3) translates correct input binding and compilation into valid planning. The following general result also permits other valid plans without requiring the host to reproduce the compiler.

Corollary 2 (Reliable task-family coverage). *Fix $\mathcal{T}_\star \subseteq \mathcal{T}$, $B \geq 0$, and $0 \leq \delta < 1$. Suppose the hypotheses of Proposition 1 hold for every $t \in \mathcal{T}_\star$, and, uniformly on this family,*

$$\eta_H(t; B) \leq \eta_\star, \quad \bar{\epsilon}(t; B) \leq \epsilon_\star, \quad \eta_\star + (1 - \eta_\star)\epsilon_\star \leq \delta, \quad \eta_\star, \epsilon_\star \in [0, 1]. \quad (27)$$

Then

$$\mathcal{T}_\star \subseteq \mathcal{C}_{S_H}(B, \delta). \quad (28)$$

Proof. For each task, Proposition 1 yields $p_{S_H}(t; B) \geq (1 - \eta_\star)(1 - \epsilon_\star) = 1 - \eta_\star - (1 - \eta_\star)\epsilon_\star \geq 1 - \delta$. The capability definition gives the inclusion. $\square$

For a grammar with bounded expanded node and edge counts, the appendix derives uniform resource and error envelopes. These supply explicit sufficient conditions for the corollary without assuming that arbitrary composition preserves a fixed budget or reliability level. Allowing a finite budget B_t chosen separately for each task can establish $p_{S_H}(t; B_t) \geq 1 - \delta$ for every $t \in \mathcal{T}_\star$ without providing one finite budget valid for the entire family. The analysis also makes no computability distinction between an orchestrated system and a single program that can simulate it with the same interfaces and resources.

The theory thus gives Raven’s components distinct analytical roles. Agent Orchestration (Section 3) constructs and executes plans. The built-in and connected agents (Section 3.1) provide candidate contract realizers. EverOS Memory (Section 5) and Skill Forge (Section 6) help construct context and configure skills. Harness Self-Evolution (Section 4) changes the policies being compared. Their effects on capability, cost, and conditional reliability require evaluation.

Prior routing results support considering collaboration modes, roles, and model assignments jointly [75]. Controlled scaling experiments also find that coordination benefits depend on task structure and

may be outweighed by overhead [30]. Accordingly, the graph agreement measures in Section 7.1 should be complemented by semantic contract checks, end-to-end outcomes, cost accounting, and failure attribution. Individual-agent evaluations (Section 7) characterize candidate providers, while matched comparisons test new-task coverage. The formal results give sufficient conditions under stated assumptions. Measured average success rates alone do not establish the conditional reliability premises.

3 Multi-Agent Collaboration

Multi-agent frameworks such as AutoGen and MetaGPT organize collaborators within a shared framework [69, 22]. Role prompts and configurations distinguish agents that use the framework’s message interface and control loop. This design supports division of labor, while leaving the system builder responsible for configuring and maintaining the roles within a common harness architecture. By contrast, Raven composes heterogeneous harnesses. Its roster combines the built-in specialists with independently developed agents, such as Claude Code and Codex, that connect through execution adapters. Collaboration therefore joins execution capabilities that differ in tools, context handling, and control policy. These differences can provide the complementary local capabilities formalized in Definition 3.

This heterogeneity introduces coordination requirements. Agents differ in their input interfaces, local-file access, conversation continuity, and failure reporting. When a host relays results through free-form messages, restating those results consumes context and may omit details. Without explicit artifact references, later outputs may also be difficult to trace to their sources. Raven therefore makes the collaboration structure explicit. The Host Agent submits a typed dependency graph, the runtime checks it before any worker starts, and every handoff passes through a recorded artifact.

Collaboration proceeds in five stages. First, the host loads orchestration guidance on demand and uses it to compose a graph. Second, the runtime admits the graph after structural, capability, and environment checks. Third, the executor dispatches nodes as their dependencies settle, judges each result, and returns exceptions and clarification requests to the host. Fourth, a run report returns file locations and terminal outputs to the host for synthesis. Fifth, later graphs and single delegations reuse completed nodes by reference. Section 3.1 describes the first two stages (Figure 4). Section 3.2 describes the next two (Figure 5), and Section 3.3 describes the artifact and memory flow that supports reuse (Figure 6). Section 3.4 describes shared group memory, an optional layer in which the host assesses each agent after a round and returns the assessment at later dispatches. These mechanisms implement the assignment, input assembly, and scheduling components of the plan in Eq. (4). The semantic contracts in Section 2 describe the conditions under which the resulting execution is correct.

As a running example, consider a user who asks for a survey of recent progress in AI for AI research, a reproduction of the leading algorithms in one experimental environment, and a web page that reports the results. Two research nodes can independently investigate the algorithms and their benchmarks. A coding node needs both investigations, and a design node needs the reproduction results. The corresponding plan is

$$\{v_{\text{alg}}, v_{\text{bench}}\} \longrightarrow v_{\text{code}} \longrightarrow v_{\text{design}}.$$

The same agent identity may execute several nodes, but each node is a distinct invocation with its own inputs, artifacts, and outcome. This example illustrates both immediate artifact dependencies and the retention of experience for later workflows.

3.1 Task Decomposition and Agent Orchestration

A Heterogeneous Agent Registry. An available agent is identified by a registered name and a capability description. Its registry entry binds that name to one of four execution backends: the in-process Raven agent loop, a command-line process, an Agent Client Protocol (ACP) connection, or an OpenAI-compatible model API. The entry also records three capabilities that constrain planning. A stateful agent can continue an earlier conversation, a file-capable agent can read local paths, and a progress-reporting agent streams its intermediate activity. The default roster contains a general Raven agent and the four specialists Raven-Research, Raven-Code, Raven-Design, and Raven-Oncall. Configuration presets connect third-party agents, including Claude Code, Codex, OpenCode, Hermes Agent, and OpenClaw, through ACP. The Host Agent receives the roster with these capability tags

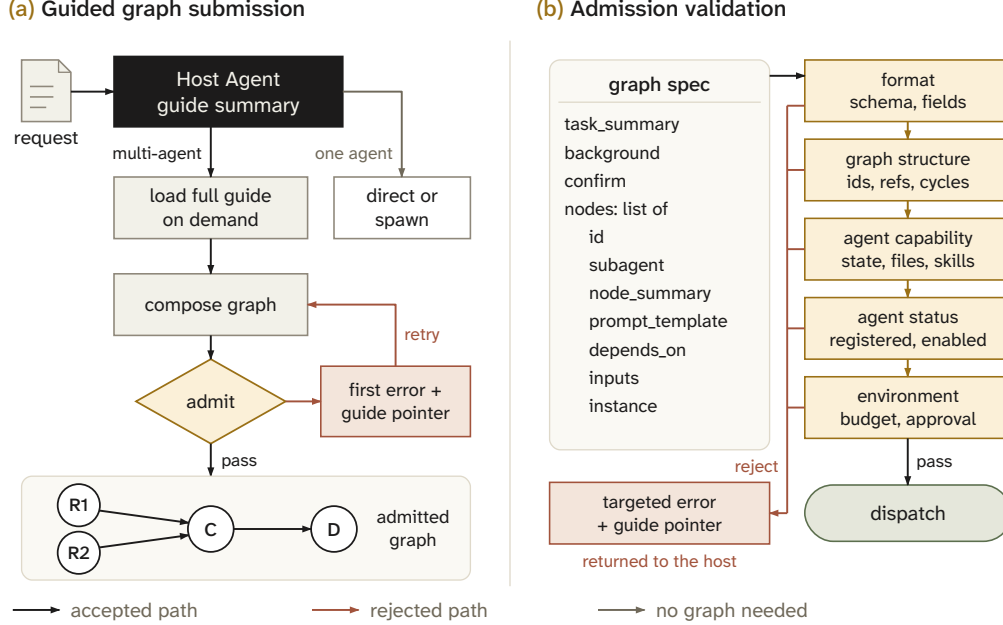


Figure 4: Graph planning and admission. (a) The host context carries only a summary of the orchestration guide. The host loads the full guide when a request needs several agents and delegates directly otherwise. A rejected graph returns its first error with a pointer back to the guide. In the admitted graph, R1 and R2 are the running example’s research nodes v_{alg} and v_{bench} , C is v_{code} , and D is v_{design} . (b) A submission is a flat node list with graph-level flags. Five groups of checks run in order before any worker is dispatched (Table 1).

through its orchestration interface. Its model selects among the entries using the current context, without a separately trained routing classifier or assumed oracle capability scores.

Staged Orchestration Guidance. A tool schema alone does not convey the rules an executable graph must satisfy. The host must respect the placeholder syntax, the identifier rules, and the capabilities of each backend. It must also decide whether a graph is needed at all. Keeping the full rules in every context would incur an input cost even on turns that do not require orchestration. Raven therefore discloses its orchestration guidance in stages (Figure 4a). Every turn carries only a short description of the orchestration guide. When the host decides that a request requires several agents, the tool description instructs it to load the full guide before composing a graph. The guide specifies node wiring, the placeholder syntax, and the concurrency limit, and it states when a single delegation is the better choice. A request that needs no graph incurs only the context cost of the short description. Once loaded, the guide remains pinned in the session context and available for subsequent graph construction. The runtime does not block a submission when the guide is absent. Instead, every rejected graph returns the validation error together with a pointer to the guide. This feedback directs the host to the relevant rules when validation fails. The planning evaluation in Section 7.1 measures the graphs produced under this guidance.

Graph Construction. The host constructs the graph by working backward from the requested deliverable. Each node must have an identifiable output and an executor able to produce it. The host then adds a dependency whenever producing one node’s input requires another node’s output. In the running example, the two research assignments ask different questions and can start independently. The coding assignment states how their findings select the algorithms to reproduce and what executable result is required. The design assignment names the expected page and the artifacts that support its claims. A request that requires only one specialist uses the single-node spawn interface, which shares the backend abstraction and the session’s node namespace.

Executable Node Specification. Let $G_{rt} = (V_{rt}, E_{rt})$ be the submitted DAG. We identify each node with its session-unique identifier, so V_{rt} and $\text{Dep}(v)$ use the same identifier universe. The record n_v stored under identifier v contains the fields

$$\begin{aligned} n_v &= (\text{id}_v, \sigma(v), \text{Tpl}_v, \text{Dep}(v), \text{Bind}_v, \text{inst}_v, \text{Cfg}_v), \\ E_{rt} &= \{(u, v) : v \in V_{rt}, u \in \text{Dep}(v) \cap V_{rt}\}. \end{aligned} \quad (29)$$

Here $\text{id}_v = v$ is the node identifier, and $\sigma(v)$ selects the registered executor, Tpl_v is the prompt template, and $\text{Dep}(v)$ lists dependencies. The input map Bind_v binds template slots to literal text, a file, or a previous node’s output. An optional instance handle inst_v continues a stateful agent conversation. Nodes that share a handle run sequentially in the same agent session. The runtime also accepts optional capability overrides Cfg_v , which specify the skills and MCP servers available for that invocation. The model-facing schema does not advertise these overrides. Omitting an override inherits the agent’s configuration, while an empty list explicitly requests none. Each node also carries a short summary for progress reporting. The exact interface fields are listed in Appendix B.2.

A submission wraps the node list with a task summary and two flags. The background flag is enabled by default. It returns control to the host as soon as the run starts, and progress, exceptions, and the final report arrive as later messages. The confirmation flag is disabled by default. When enabled, it requests user approval for the listed nodes before dispatch. If approval is denied, no node runs.

Prompt Construction and Artifact Binding. The task text is obtained by resolving the template against its declared inputs and the available artifact ledger:

$$x_v^{\text{task}} = \text{Render}(\text{Tpl}_v, \text{Bind}_v, (y_u)_{u \in \text{Dep}(v)}, d). \quad (30)$$

The authorized request packet d includes the user’s objective and constraints, and y_u is the recorded output of node u . The indexed family retains the node identifier associated with each output, even when two nodes return identical content. The host must carry the relevant parts into the node’s prompt because the executor does not have access to the host’s private conversation. A template inserts an upstream output with $\{\{ \text{id.output} \}\}$ or passes its path with $\{\{ \text{id.output_path} \}\}$. Declared inputs appear through $\{\{ \text{inputs.k} \}\}$ or $\{\{ \text{inputs.k.path} \}\}$, and files through $\{\{ \text{ref:path} \}\}$ or $\{\{ \text{ref_path:path} \}\}$. For example, $\{\{ \text{research_alg.output_path} \}\}$ gives a file-capable coding agent access to the algorithm survey. Path references avoid duplicating a long report in the initial prompt. An executor without local-file access instead needs the corresponding content. Material too long for a template, such as a detailed specification drafted by the host, is written to a file and referenced in the same way. Substituted file and node contents are marked as untrusted data to distinguish upstream artifacts from host instructions.

Admission Validation. The runtime admits a graph only after five groups of checks (Figure 4b and Table 1). Format checks reject malformed arguments and unknown fields. Graph-structure checks require unique identifiers, resolvable dependencies and inputs, confined file paths, and acyclicity. Capability checks reject wiring that the selected backend cannot honor, such as a shared instance on a stateless agent or a path reference sent to an agent without local-file access. Status checks require each named agent to be registered and enabled. Environment checks refuse recursive delegation from inside a sub-agent run, enforce a dispatch budget shared by graphs and single delegations, and collect any requested user approval. Validation stops at the first failure and returns a message that names the offending node or field. Several messages also state the repair, for example replacing a path reference with the content form for an agent without local-file access. The runner repeats the checks while it reserves node identifiers, so two concurrent submissions cannot claim the same identifier.

Admission establishes that the graph can be interpreted consistently. Because validation precedes dispatch, a rejected graph consumes a host turn without starting a partial worker execution. However, semantic validation must additionally establish whether the research assignments address the user’s question and whether the coding task has an adequate completion criterion.

3.2 Host-Mediated Graph Execution

Node Lifecycle. Once a graph is admitted, each node moves through a small set of runtime states (Figure 5a). Let $j_{rt} \in \{0, 1, \dots\}$ index scheduler events. Let $z_{j_{rt}}(v)$ denote the status of node v : pending, running, completed, exception, failed, skipped, or cancelled. A node starts pending and runs

Table 1: Admission checks applied before dispatch, grouped as in Figure 4b. Validation returns the first failure, followed by a pointer to the orchestration guide.

Group	Checks
Format	Arguments parse as complete JSON. Fields have the required types and values. No unknown field is present. Identifiers and instance handles match their patterns. Required node fields are nonblank.
Graph structure	Identifiers are unique within the graph and unused in the session. Dependencies name a node of the graph or a completed earlier node with output. Every output placeholder is backed by a dependency. Input values are well formed, and declared and referenced inputs agree. Path forms apply only to file or node inputs. File and reference paths stay within the permitted roots. The graph is acyclic.
Agent capability	Shared instances use stateful agents. Skill overrides appear only at the head of an instance chain. Path forms are not sent to agents without local-file access.
Agent status	Each named agent is registered and enabled.
Environment	The call does not come from inside a sub-agent run. Delegation is not paused, and the dispatch budget is available. The user approves the graph when confirmation is requested.

once every in-graph predecessor has completed and settled. The backend either returns an output or raises an error, and both outcomes go to a completion judge. A positive verdict marks the node as completed. A negative verdict suspends the node in the exception state if continuations remain and the host can be reached. Otherwise the node fails. A failed or skipped predecessor marks its pending successors as skipped. An exception does not propagate, because the host may still repair the node. Stopping a run cancels running and suspended nodes and skips pending ones.

Dependency-Ready Scheduling. Two additional sets track submitted work and outcomes awaiting a final verdict. Let $\text{Submitted}_{j_{rt}}$ contain calls already assigned to an execution task, including calls queued for a concurrency slot, and let $\text{Unsettled}_{j_{rt}}$ contain outcomes awaiting adjudication. The ready set is

$$\text{Ready}_{j_{rt}} = \left\{ v \in V_{rt} : \begin{array}{l} z_{j_{rt}}(v) = \text{pending}, \quad v \notin \text{Submitted}_{j_{rt}}, \\ \forall u \in \text{Dep}(v) \cap V_{rt}, \\ z_{j_{rt}}(u) = \text{completed} \wedge u \notin \text{Unsettled}_{j_{rt}} \end{array} \right\}. \quad (31)$$

Earlier-session dependencies have already been checked at admission. Excluding $\text{Submitted}_{j_{rt}}$ prevents a node waiting for a slot from being submitted twice. Excluding unsettled predecessors prevents a consumer from reading the output of a call whose completion judgment may still be rejected.

Ready nodes execute under a shared concurrency semaphore. This limit applies across all graph runs and single delegations in the session, keeping their combined execution within the available capacity. Ready nodes sharing an instance handle are grouped and executed sequentially, whereas independent instances can proceed concurrently. A settled node wakes the scheduler immediately, so the successor of a short research call need not wait for unrelated work to finish. In the running example, the coding node becomes eligible only after both research outcomes settle.

Completion Adjudication. A returned response alone does not establish task completion. An agent may explain why it could not proceed, stop at an output limit, or report success without producing the requested artifact. An independent model call therefore judges each finished node (Figure 5b). The judge reads the rendered prompt, the output, and the tail of the execution transcript. Two flags state whether the transcript evidence is complete and whether the output was cut short. The judge must answer through a constrained tool with the outcome accomplished or not accomplished. A negative verdict names a category, such as missing user input, a missing credential, a tool failure, unusable dependency output, or an output limit. It also states what is missing and cites supporting evidence. A supported negative answer to a research question counts as accomplished, whereas a report that the research could not be performed does not. A node judged unaccomplished loses its output reference, so no successor can consume it.

However, the verdict does not certify objective correctness. Under the availability fallback, a normally returned call is treated as accomplished if the judge fails or times out. Domain tests and final artifact

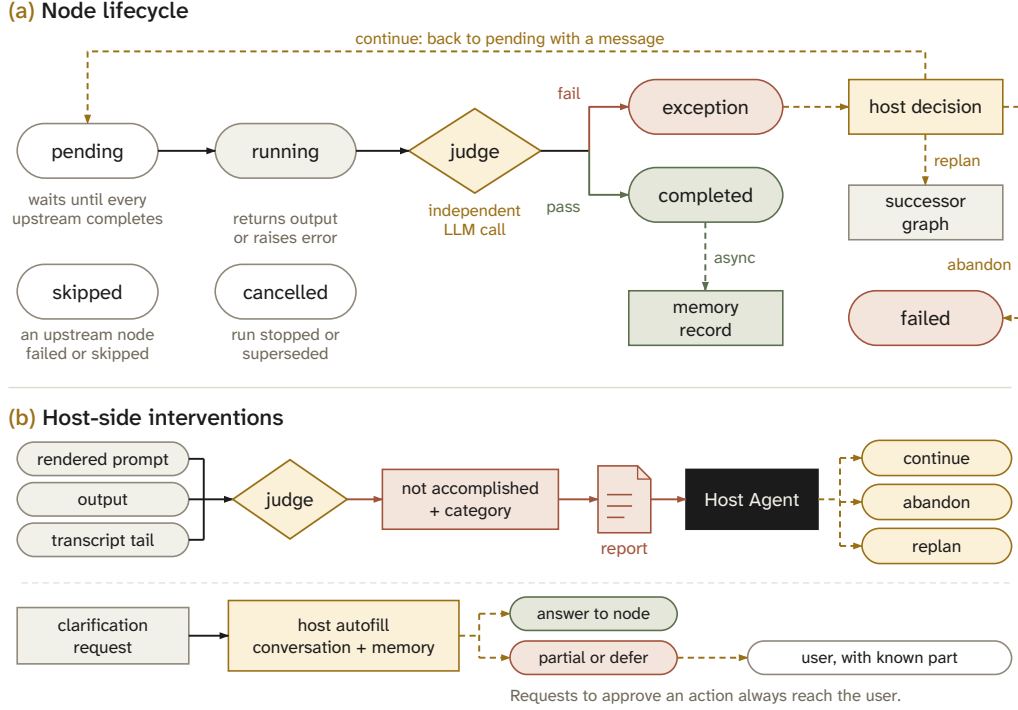


Figure 5: Node execution and host intervention. (a) A node runs once all predecessors have completed and settled. Returned output and raised errors both reach the judge. A failed verdict suspends the node for a host decision when continuations remain. Memory recording runs asynchronously and does not delay successors. (b) Top: the judge reads the rendered prompt, output, and transcript tail, and a negative verdict becomes an exception report for the host. Bottom: a worker’s clarification request is answered from the host’s conversation and memory when possible, and otherwise reaches the user.

inspection provide independent evidence. The observed completed set in Eq. (31) therefore differs from the objective successful-prefix event used in Section 2.4.

Exception Resolution. A suspended node sends an exception report to the host. The report names the node and its agent, restates the task, and gives the verdict category, the missing condition, and the evidence. It also lists the dependents that the node blocks, the attempts that remain, and the decision deadline, and it supplies ready-to-use calls of the resolution tool. The host uses the report and its conversation context to select a resolution, consulting the user when the missing information requires user input. It can continue the node with a message, abandon it, or replan the remaining workflow. Continuation returns the node to pending with the message attached. A stateful instance receives the message alone, whereas a stateless agent receives its original prompt, its previous result, and the message. Continuation retains the run and its completed artifacts, with a bounded number of additional attempts. Abandonment fails the node and skips its dependents. Replanning validates a successor graph, collects any requested approval, and starts it under a new run identifier linked to the old run. Unfinished nodes of the old run are skipped, cancelled, or marked failed as superseded, and its completed nodes remain available by reference. A report that receives no decision before its deadline fails the node. Recovery therefore changes the executable plan without overwriting earlier execution records.

Clarification Requests. A worker sometimes needs information that its prompt omits, such as a preferred output format or the location of a dataset. Agents connected through ACP can raise such questions while they run. Forwarding every question to the user would interrupt the user for details that the conversation already contains. The host therefore attempts an answer first (Figure 5b, bottom). An independent model call receives the questions, a snapshot of the host’s current conversation,

earlier answers in the session, and memories recalled from EverOS with the questions as queries. It answers a question only when the user stated the answer or a recalled preference determines it uniquely. It never answers a request to approve an action, such as pushing, deleting, sending, or paying. Unanswered and partly answered questions go to the user, annotated with what the host already knows. The questions enter this call as untrusted data, preserving the distinction between a worker’s request and user authorization.

Algorithm 1: Host-Mediated Graph Execution

Input: request packet d , graph specification, registry, artifact ledger, and concurrency allowance.

Output: a run report, terminal artifacts, and a persistent execution record.

1. Apply the admission checks in Table 1. On the first failure, return the error and a pointer to the orchestration guide without running any node.
2. If confirmation is requested, obtain user approval. Reserve the identifiers, persist the graph, and treat admitted earlier-session dependencies as completed artifacts.
3. While runnable, running, or suspended work remains:
 - (a) Propagate terminal failures to dependent nodes and apply host decisions to continue, abandon, or replan.
 - (b) Compute $\text{Ready}_{j_{rt}}$ by Eq. (31). Group shared instances and reserve submitted nodes in $\text{Submitted}_{j_{rt}}$.
 - (c) When a slot is available, assemble the node input x_v (Eqs. (30) and (34)), persist its prompt, and invoke the selected backend. Route clarification requests to host autofill, and forward the remainder to the user.
 - (d) Persist the output and transcript, and obtain the judge’s verdict. Commit an accomplished outcome. Otherwise suspend the node and send an exception report, or fail it when no continuation remains. Remove settled outcomes from $\text{Unsettled}_{j_{rt}}$ and wake the scheduler.
 - (e) Release the execution slot and schedule memory recording independently of artifact completion.
4. Return the run report, with status counts, node output files, and terminal outputs, to the Host Agent for final synthesis.

Run Report and Final Synthesis. When a run finishes, the host receives a compact report that summarizes intermediate outcomes (Figure 6b). The report gives the numbers of completed, failed, cancelled, and skipped nodes and the run directory. It lists each node with its status, instance handle, and output file, together with the error of any node that did not complete. Only the terminal outputs, from completed nodes without dependents, appear in full, up to a length cap. In the running example, the host receives the design node’s output in full and file paths for the two surveys and the reproduction. A foreground call returns an exception report as soon as a node suspends, and the host’s resolution call returns the next report or this final report. A background call returns a run identifier at once and later delivers progress, exception, and completion reports as messages. Reports that quote node output are marked as untrusted data. The host then reads further files only if its final response needs them.

Substantive final reasoning by the host is also an execution operation. In the theoretical annotation it must appear as a host-policy call with its own contract and cost. A final response that only delivers existing artifacts incurs serialization and delivery costs. This distinction keeps both synthesis and delivery within the plan’s cost accounting.

Resource Accounting. The executor is event-driven and does not impose a universal wall-clock limit on a graph. Individual backends, continuation policies, decision deadlines, and the evaluation protocol supply their own limits. Consequently, the common budget in Section 2 must be instantiated explicitly when measuring a composed run. Structural scheduling does not establish a cost advantage by itself. Planning, verification, retries, clarification, and handoffs must all be included in the measured resource use.

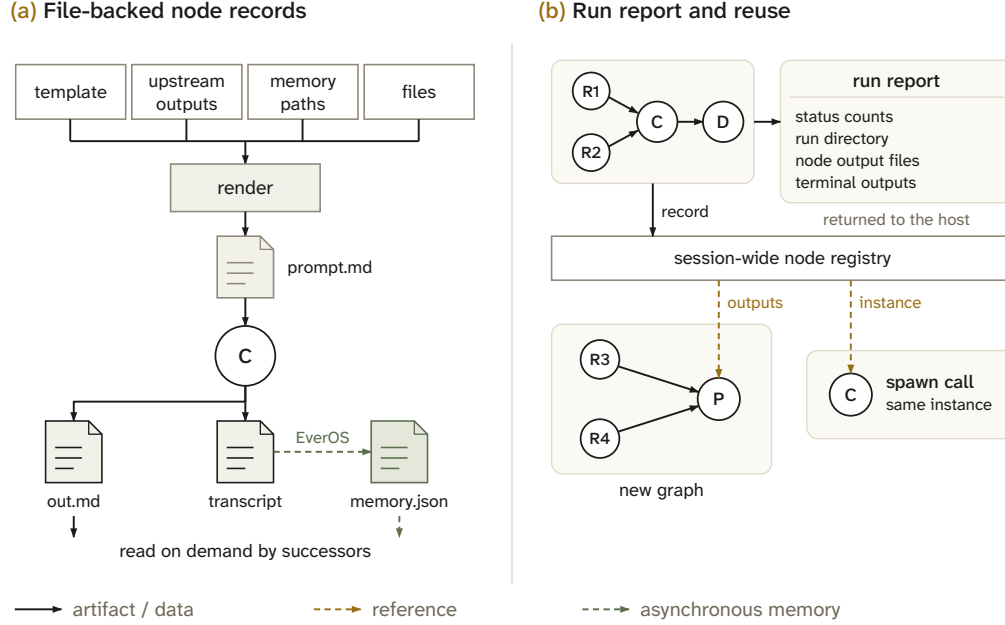


Figure 6: Artifact and memory flow. (a) A node’s prompt is rendered from its template, upstream outputs, upstream memory paths, and files. The runtime writes the prompt, output, and transcript to disk, and a memory record follows asynchronously. Successors read these files on demand. (b) The host receives a run report with paths and terminal outputs. Completed nodes enter a session-wide registry, so a later graph can reference their outputs and a single delegation can continue their instance. In (b), R3 and R4 are research nodes of a follow-up graph whose slide-deck node P references earlier outputs, and the spawn call continues the coding instance C.

3.3 Multi-Agent Memory Flow

Two Information Paths. A collaboration carries both task artifacts and retained experience. An artifact y_v is the immediate output needed to execute the submitted graph. A memory record contains what a memory backend has distilled from that invocation. In the running example, the design node needs the reproduction results regardless of whether a reusable case has already been extracted from the coding trace. Artifact completion therefore releases dependencies, while memory recording proceeds asynchronously.

File-Backed Node Records. Each handoff uses a recorded file containing the producer’s output (Figure 6a). The runtime stores each run’s graph and manifest separately from a session-wide node directory. For every node it writes the rendered prompt before dispatch, the output when the backend returns, and the transcript before the judge reads it. A continued node keeps a numbered copy of each attempt’s prompt, output, and transcript. A successor reads an upstream output inline or by path, and the host inspects a file only when it needs the content. File references avoid repeating long artifacts in the host’s prompt and in every successor’s prompt. They also give successors access to the producer’s output without an intervening paraphrase. The records identify which artifact each downstream node consumed, including after a foreground wait ends.

For a memory-enabled worker, the host also reads that invocation’s session twice: episodes on the user track and cases on the agent track (Section 5.5). These reads retrieve records by session identifier without relevance ranking. They do not request profiles or skills. When the worker’s memory source is its own backend, the host reads that backend and does not write on the worker’s behalf. When the source is a captured trace, the host stores the trace in the host’s own backend under an explicit session identity and marks the write final. The record preserves whether its source was the worker backend or the host-captured trace. The two processes can use different backends and identities.

Dependency-Scoped Propagation. For a file-capable node v , the runtime appends paths to the memory records of its transitive predecessors:

$$\text{Paths}_v = \{\text{MemPath}(u) : u \in \text{Anc}_{G_{\text{rt}}}(v)\}. \quad (32)$$

Here $\text{MemPath}(u)$ is the deterministic file location reserved for node u 's memory record. This block is generated from the graph and does not require the host to repeat every memory reference in the template. Paths give the receiving agent the option to inspect relevant experience without eagerly copying all ancestor memories into its prompt. The design node can consequently inspect the coding case and both research records. A backend without local-file access receives no unusable path block. Any information it requires must be supplied through its supported input channel. Write PathBlock_v for this rendered block, or empty text when the backend cannot read files. The base input is $x_v^{\text{base}} = x_v^{\text{task}} \mathbin{++} \text{PathBlock}_v$, where $\mathbin{++}$ concatenates text in order.

Cross-Graph Reuse. Node identifiers are unique across the whole conversation, and graph nodes and single delegations share one registry (Figure 6b). A later graph can therefore depend on a node that an earlier run completed, or read its output through a placeholder, without executing it again. Admission verifies that the earlier node completed with a recorded output. A failed, skipped, cancelled, or still-running node cannot be used as if it had produced a result. Instance handles also persist across runs. A later node that names the same handle continues the same agent session with its accumulated context. In the running example, a follow-up request for a slide deck can reuse both surveys and the reproduction by reference. A single delegation can continue the coding instance to repair one experiment. Repeating work requires a fresh identifier, which preserves the earlier record. Reuse avoids redundant subtasks and limits the host's context to references to earlier artifacts.

Cross-Task Reuse and Its Boundary. Within a graph, ancestry determines which memory-record paths are advertised. Across sessions, task-aware retrieval names the user or the agent, and the server applies its default application and project scope (Section 5.3). Cases can subsequently support skill extraction (Section 6.3), making procedures from individual executions available to future tasks. Section 3.4 extends this path with assessments that the host records for each agent after a round. The mechanism provides dependency-scoped provenance and bounded context transfer while retaining experience for future tasks. Its effect on final-task success requires evaluation beyond graph agreement.

3.4 Shared Group Memory

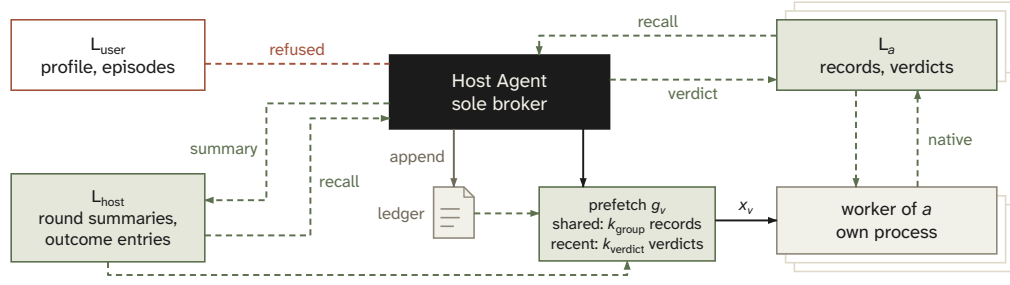
A worker observes its own execution but lacks direct access to downstream use of its output and subsequent user feedback. The host observes both through run reports and later user messages. To incorporate this broader feedback, shared group memory converts these observations into assessments that inform future orchestration and worker invocations (Figure 7 and Algorithm 2). We define a *round* as a finished graph run or a single delegation, represented as a one-node graph. The completion judge evaluates individual nodes (Section 3.2), while group memory records the host's assessment of each agent's contribution across the round.

The layer reuses the active EverOS backend and stores three record types: a deterministic round summary, per-agent assessments called *verdicts*, and an outcome entry after user feedback. Summaries and outcome entries belong to the host's shared library, while verdicts belong to the assessed agents' libraries. Bounded retrieval supplies this experience before host planning and worker execution. Artifact transfer follows the graph and cross-graph references described in Section 3.3.

Memory Ownership and Access. A library is the collection of records under one EverOS owner identifier (Section 5.1). Let L_{user} contain the user's profile and episodes. The host's shared library L_{host} uses a separate owner derived from the same user identifier. For each agent a in the configured subset $\mathcal{A}_{\text{map}}$, let L_a use the worker's existing owner identifier $\text{own}(a)$. Depositing verdicts under this owner makes them available through the worker's native recall. All instances of an agent share its library, as do agents mapped to the same owner. Reuse therefore operates at the owner level, without isolation between instances that share an owner. Unmapped agents receive no group memory prefetch or verdicts.

The host performs all group memory reads and writes under the policy in Table 2. Workers receive selected records through prefetch and retain their ordinary access to their own libraries. Group

(a) Host-brokered libraries



(b) Two-phase assessment across host turns

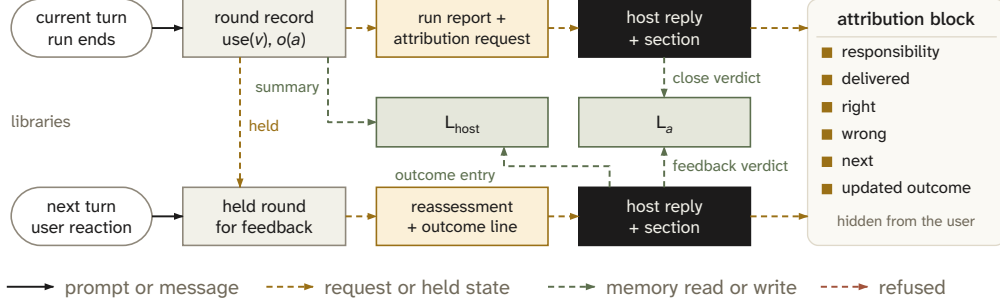


Figure 7: Shared group memory. (a) The host mediates access to owner-partitioned EverOS libraries. It retrieves experience before planning, deposits round summaries and outcomes in its shared library, and writes agent assessments to the corresponding agent libraries. Dispatch prefetch combines shared experience with recent assessments from a provenance ledger, with respective caps k_{group} and k_{verdict} . Personal user memory is excluded, while workers retain their ordinary memory access. Offset outlines indicate one library and worker per mapped agent. (b) Assessment has two phases: a close verdict after execution and a feedback verdict on the next user turn. Attribution is extracted from the host’s reply and omitted from the user-visible response and session log.

Table 2: Access within the group memory layer. The host performs all retrievals and deposits. Ordinary host and worker memory calls are outside this policy.

Actor	L _{host}	L _{user}	Agent Libraries
Host	read, write	none	read and write every mapped library
Mapped worker	read via prefetch	none	its own verdicts via prefetch
Unmapped agent	none	none	none

memory excludes L_{user} and does not expose agent libraries under other owners to workers. Verdict targets are resolved from the configured owner map and restricted to agents dispatched in the round. Generated assessment text cannot select a target owner.

Recall Before Orchestration. For an incoming host message x_{in} , define the queried libraries as $\mathcal{Q} = \{L_{\text{host}}\} \cup \{L_a : a \in \mathcal{A}_{\text{map}}\}$. The host receives

$$g_{\text{orch}} = \text{Fence}_{B_{\text{orch}}} \left(\text{Merge} \left(\left(\text{Top}_{k_{\text{group}}} (L, x_{\text{in}}) \right)_{L \in \mathcal{Q}} \right) \right). \quad (33)$$

Here the positive integer k_{group} is a retrieval cap, and $\text{Top}_{k_{\text{group}}} (L, q)$ returns up to k_{group} records ranked for query q within L (Section 5.3). Merge groups results by source library and removes duplicate lines, and $\text{Fence}_{B_{\text{orch}}}$ renders them as untrusted context within a character budget B_{char} , instantiated as B_{orch} here and B_{worker} below. Without a subscript, Fence applies no character budget. Retrieval covers every mapped agent, including those not yet dispatched, so prior experience is

available when selecting an executor. Agent libraries contribute both native worker records and host verdicts. The block is appended to the incoming message and excluded from the session log to prevent automatic replay on later turns.

Prefetch at Dispatch. For a node v assigned to a mapped agent $\sigma(v)$, the host augments the base input x_v^{base} , including the path block from Section 3.3:

$$\begin{aligned} g_v &= \text{Fence}_{B_{\text{worker}}} \left(\text{Merge} \left(\left(\text{Top}_{k_{\text{group}}} (\text{L}_{\text{host}}, x_v^{\text{base}}), \text{Recent}_{k_{\text{verdict}}} (\sigma(v)) \right) \right) \right), \\ x_v &= x_v^{\text{base}} \mathbin{++} g_v. \end{aligned} \quad (34)$$

The operator $\text{Recent}_{k_{\text{verdict}}}(a)$ selects the most recent verdicts about agent a , up to a positive integer cap k_{verdict} , from the host’s provenance ledger. Shared experience is selected by query relevance, while verdicts are selected by recency. This gives recent feedback a retrieval path independent of its similarity to the new task or its ranking against the worker’s native records. The block follows the task so that the worker’s memory records the assignment as its task context. Single delegations use the same rule. If this layer is disabled or the agent is unmapped, g_v is empty and $x_v = x_v^{\text{base}}$.

The final dispatched input x_v is the object corresponding to Eq. (5). Applying that theory to memory-enabled execution requires an annotated expansion $G_{\text{ann}} = (V_{\text{ann}}, E_{\text{ann}})$ of G_{rt} . Semantic retrieval and other context-producing computation become explicit operations with their own contracts, costs, and recorded outputs. The theory is instantiated with $G = G_{\text{ann}}$, so $K_{\text{op}} = |V_{\text{ann}}| + |E_{\text{ann}}|$, resource allocations, and local error sums refer to the expanded plan. Authorized fixed context is bound in d , and later retrieval results enter through declared messages. The final Φ_v only projects or serializes these inputs. This annotation preserves the theory’s assembly restriction and accounts for retrieval cost. Supplying a memory block alone does not establish these semantic guarantees.

Deterministic Round Outcomes. For round r with graph $G_{\text{rt}} = (V_{\text{rt}}, E_{\text{rt}})$, let $z(v)$ denote the final status of node v and $\text{Succ}(v) = \{w \in V_{\text{rt}} : v \in \text{Dep}(w)\}$ its successors within the round. Stopped runs are excluded from recording because cancellation and skipping may reflect the stop decision. For other finished rounds, define

$$\text{use}(v) = \begin{cases} \text{terminal}, & \text{Succ}(v) = \emptyset, \\ \text{consumed}, & \exists w \in \text{Succ}(v) : z(w) \in \{\text{completed}, \text{failed}\}, \\ \text{unused}, & \text{otherwise.} \end{cases} \quad (35)$$

Here consumed indicates that a successor executed, even if it failed, and terminal indicates that no in-round successor exists. These labels describe execution structure and do not establish output quality or user acceptance.

Let $\mathcal{A}_r = \{\sigma(v) : v \in V_{\text{rt}}\}$ be the participating agents and $V_a = \{v \in V_{\text{rt}} : \sigma(v) = a\}$. For $a \in \mathcal{A}_r$, the initial outcome is the first matching case below:

$$o(a) = \begin{cases} \text{cancelled}, & \exists v \in V_a : z(v) = \text{cancelled}, \\ \text{failed}, & \exists v \in V_a : z(v) = \text{failed}, \\ \text{cancelled}, & \exists v \in V_a : z(v) \neq \text{completed}, \\ \text{adopted}, & \forall v \in V_a : \text{use}(v) \neq \text{unused}, \\ \text{partially_adopted}, & \exists v \in V_a : \text{use}(v) \neq \text{unused}, \\ \text{rejected}, & \text{otherwise.} \end{cases} \quad (36)$$

The third case maps skipped nodes to cancelled. Adoption labels apply only when all of the agent’s nodes completed, with terminal outputs counted as adopted. At this phase, rejected denotes unused outputs. The host immediately deposits a round summary containing node assignments, statuses, dependencies, task excerpts, and agent outcomes in L_{host} . Task excerpts provide semantic context for later retrieval.

Two-Phase Agent Assessment. A verdict associates the host’s assessment with an agent, round, and phase. For $a \in \mathcal{A}_r \cap \mathcal{A}_{\text{map}}$, write

$$\text{Verd}_{r,a}^{(p)} = (\text{id}_H, a, r, p, o_{r,a}^{(p)}, \text{body}_{r,a}^{(p)}), \quad p \in \{\text{close}, \text{feedback}\}, \quad o_{r,a}^{(\text{close})} = o(a), \quad (37)$$

where id_H identifies the host as author and $\text{body}_{r,a}^{(p)}$ is the assessment text. Each verdict is deposited in L_a , with its author, subject, round, and outcome encoded in a leading header.

At round closure, the run report requests one attribution block per distinct participating agent. The host appends these blocks to its existing reply, requiring no additional model call. Each block addresses the agent directly and describes its responsibility, delivered work, strengths, failures and their causes, and instructions for future tasks. Assessments must be grounded in observations, while node statuses come from the run record. The layer preserves usable block text in the close verdict and removes the attribution section before displaying or logging the reply. Missing or unparseable blocks yield deterministic summaries of node status and output use, with distinct headers identifying the fallback reason.

The layer also retains each closed round for one reassessment on the next user turn. The host supplies updated attribution blocks and proposed outcome labels $\hat{o}_{r,a}$, with missing or invalid labels represented by $\perp$. Let Labels contain the five distinct labels in Eq. (36). For each reassessed mapped agent,

$$o_{r,a}^{(\text{feedback})} = \begin{cases} \hat{o}_{r,a}, & \hat{o}_{r,a} \in \text{Labels}, \\ o_{r,a}^{(\text{close})}, & \text{otherwise.} \end{cases} \quad (38)$$

Only this phase permits model-generated text to revise the deterministic outcome. The feedback verdict is appended alongside the close verdict under the same round identifier. An outcome entry in L_{host} records the reassessed labels and their initial values when changed. The round is then released from pending reassessment. Without a subsequent user turn, only the close verdict is retained.

In the running example, suppose Raven-Research is mapped. It receives one close verdict covering both research nodes. If the user later identifies an outdated benchmark version, the host can revise its outcome to rejected and include a corrective instruction in the feedback verdict. Subsequent prefetch can expose this verdict to another Raven-Research instance even when the task differs, subject to the recency and character budgets. The host can also retrieve it when planning a related task.

Algorithm 2: Shared Group Memory Across Host Turns

Input: host messages, dispatched rounds, owner map own , libraries $\mathcal{Q}$, and provenance ledger.

Output: host and worker context blocks, round summaries, agent verdicts, and outcome entries.

1. Before host planning, append g_{orch} from Eq. (33) to the incoming message without logging the block.
2. At dispatch to a mapped agent, send x_v from Eq. (34).
3. At completion of a non-stopped round, compute output-use labels and agent outcomes by Eqs. (35) and (36), deposit the round summary in L_{host} , and retain the round for reassessment.
4. Request attribution in the host’s reply to the run report. Extract and deposit $\text{Verd}_{r,a}^{(\text{close})}$ for each participating mapped agent, using labeled fallbacks where needed. Strip attribution before displaying or logging the reply.
5. On the next user turn, request reassessment, apply Eq. (38), append feedback verdicts and a shared outcome entry, and release the round.
6. Enforce Table 2 on every read and write, and record each deposit in the provenance ledger.

Provenance and Limitations. EverOS records identify an owner but have no separate author field, and extraction can shorten or rewrite their text. Verdict headers therefore carry attribution within the content. To preserve the original assessments, a host-side append-only ledger retains the authoritative deposit text, target owner, record kind, round, and covered nodes. Each deposited record ends with a ledger reference, and the ledger supplies verdicts for recency-based prefetch. The layer neither expires records nor reviews workers’ native writes. Verdicts remain observational judgments by the host and may be incorrect. Group memory is disabled by default and is not evaluated in this report. Its effect on orchestration and task performance remains to be established.

4 Agent Harness Self-Evolution

Harness self-evolution seeks to improve the execution policy surrounding a frozen language model. Building on HarnessBank [39], the method exposes the policy through four strategy interfaces, diagnoses execution failures, constructs new or recombined harnesses, screens their behavior, and retains evaluated candidates in a semantic gene bank. Sections 4.1 to 4.4 specify the complete method, and Section 7.2 summarizes the published harness-evolution experiments. A Task Agent executes tasks, an Evolver Agent proposes modifications, and a fixed Evaluator measures their outcomes. The resulting agent is available to the Host Agent for subsequent tasks. Figure 8 connects the editable policy to the search procedure. We first define the modules and admissible edits, then the diagnostic evidence, archive search, and verification rules.

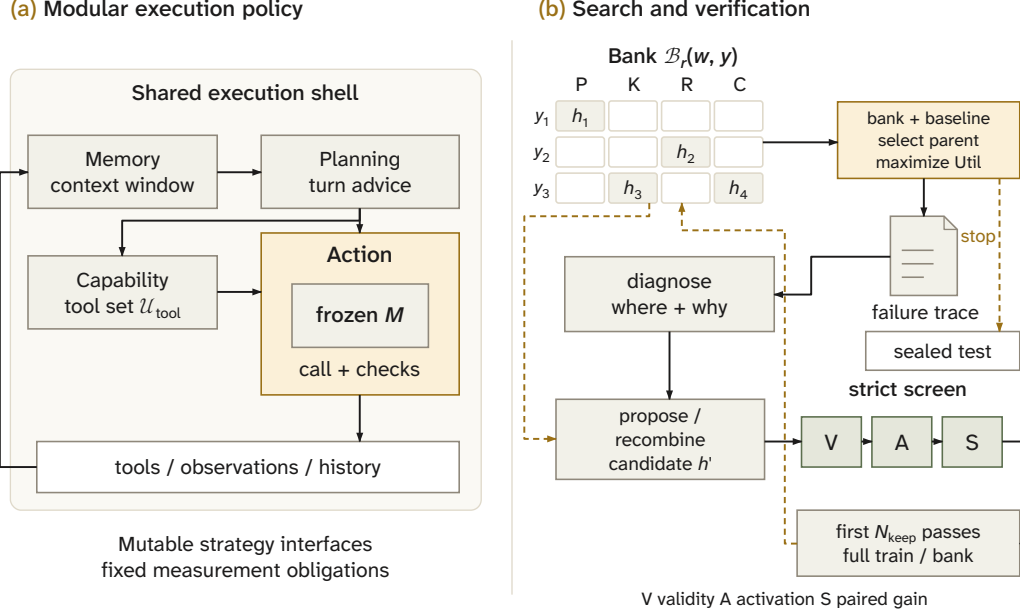


Figure 8: Harness self-evolution. (a) Four strategy interfaces expose decisions around a frozen model. The shared execution shell handles dispatch and accounting. (b) The best training-evaluated harness supplies failure traces for diagnosis and candidate generation. The bank stores one complete harness per edit-category/pathology cell. P/K/R/C denote prompt, knowledge, runtime, and configuration edits. The first N_{keep} candidates passing validity, activation, and paired-gain screening receive full-training evaluation before cell competition, where N_{keep} is the per-round survivor cap. The final training-selected harness is frozen before held-out testing.

4.1 Modular Harness Design

The Policy Surrounding a Model. For the model–harness pair in Eq. (1), let M denote the frozen Task Agent model and represent its configurable harness as

$$h = (\text{id}_h, \text{Mem}_h, \text{Plan}_h, \text{Cap}_h, \text{Act}_h; \theta_h). \quad (39)$$

The identifier id_h distinguishes candidates even when their executable components coincide. It keys the candidate’s descriptor, activation specification, creation order, and designated evaluation records. The four strategy components govern execution. θ_h contains their prompts, domain instructions, and configuration, rather than the weights of M . **Memory** assembles the working context and decides what to remove or summarize under context pressure. **Planning** prepares turn guidance and incorporates domain advice. **Capability** selects the tool definitions exposed to the model. **Action** obtains a usable model response and judges actions or completion. These roles implement decisions within one agent. The shared execution shell handles tool dispatch, iteration accounting, persistence, events, and bounded retry control.

The interfaces connect through the context presented to the model. Let x_{in} be a request, $\text{Hist}_{\text{local}}$ its available interaction history, and $B_{\text{ctx}} > 0$ the context allowance. The initial message sequence is

$$x^{(0)} = \text{Plan}_h(x_{\text{in}}, \text{Mem}_h(\text{Hist}_{\text{local}}, x_{\text{in}}, B_{\text{ctx}})). \quad (40)$$

At model-call iteration $j \geq 0$, with current messages $x^{(j)}$, the next two decisions are abstracted as

$$\mathcal{U}_{\text{tool}}^{(j)} = \text{Cap}_h(x^{(j)}, j), \quad u^{(j)} \sim \text{Act}_h(\cdot \mid M, x^{(j)}, \mathcal{U}_{\text{tool}}^{(j)}). \quad (41)$$

Here $\mathcal{U}_{\text{tool}}^{(j)}$ is the available tool set and $u^{(j)}$ is a response containing tool requests or a proposed final answer. The conditional distribution includes model sampling. The shell dispatches tool requests and appends observations to the history to construct the next context. A proposed final answer is subject to the Action role’s completion judgment. Under context pressure, Memory proposes a reduced window and the shell controls the retry. These equations abstract the information flow across the strategy interfaces. The default Planning preparation preserves its input messages, while domain-specific components can add iterative advice.

Domain Specialization and Mutable Boundaries. A specialized agent combines these strategies with domain tools, skills, and instructions. Research can emphasize evidence sufficiency, while coding can emphasize execution feedback and final verification. The selected memory backend supplies context to Mem_h , whereas the strategy itself controls the working window. This distinction permits changing the use of memory without redefining the storage system in Section 5.

Building on this modular design, evolution starts from a baseline harness h_0 . Let $\mathcal{X}_{\text{edit}}$ be the allowed edit locations and $\mathcal{K}_{\text{fixed}}$ the immutable locations, with $\mathcal{X}_{\text{edit}} \cap \mathcal{K}_{\text{fixed}} = \emptyset$. The immutable kernel protects evaluation, bookkeeping, and required execution interfaces. For a concrete patch Δ^{patch} , $\text{scope}(\Delta^{\text{patch}})$ is the set of locations it changes and $\text{Apply}(h, \Delta^{\text{patch}})$ materializes a harness with a fresh candidate identity. The identity is an analytical label and is excluded from executable edit locations and protected contents. Write $\text{Exec}(h)$ for the executable components of h , excluding its identity. The admissible search space is

$$\begin{aligned} \mathfrak{H}(\mathcal{X}_{\text{edit}}) = \{h : \exists \Delta^{\text{patch}}, \\ \text{Exec}(h) = \text{Exec}(\text{Apply}(h_0, \Delta^{\text{patch}})), \\ \text{scope}(\Delta^{\text{patch}}) \subseteq \mathcal{X}_{\text{edit}}, \quad h|_{\mathcal{K}_{\text{fixed}}} = h_0|_{\mathcal{K}_{\text{fixed}}}, \\ \text{InterfaceValid}(h) = 1\}. \end{aligned} \quad (42)$$

Restriction $h|_{\mathcal{K}_{\text{fixed}}}$ denotes the protected contents. InterfaceValid checks construction and interface compatibility. We assume h_0 satisfies these requirements. A patch applied to an evolved parent must leave the resulting harness in the same search space: its cumulative modifications still obey the baseline’s edit restrictions. Admissibility establishes executability, while task-level improvement requires evaluation.

4.2 Diagnosis-Driven Harness Evolution

Objective and Measurement Unit. Let $\mathcal{D}_{\text{tr}}$ be the finite training task set available to evolution and $\mathcal{D}_{\text{te}}$ a disjoint, sealed test set. A fixed protocol Π_{eval} specifies the model and environment versions, scorer, initial state, sampling policy, resource ceilings, and infrastructure-retry rules. Each task receives $K_{\text{att}} \geq 1$ scored attempts, with $[K_{\text{att}}] = \{1, \dots, K_{\text{att}}\}$. For task i , attempt k , and harness h , the Evaluator records score $s_{i,k}(h) \in [0, 1]$, execution trajectory $\tau_{i,k}(h)$, and metadata $\text{meta}_{i,k}(h)$, including validity, cost, and activation events. For any nonempty task set $\mathcal{D}$, define

$$\bar{s}_i(h) = \frac{1}{K_{\text{att}}} \sum_{k=1}^{K_{\text{att}}} s_{i,k}(h), \quad \text{Util}(h; \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{i \in \mathcal{D}} \bar{s}_i(h), \quad (43)$$

$$\text{Log}(h; \mathcal{D}) = ((s_{i,k}(h), \tau_{i,k}(h), \text{meta}_{i,k}(h)))_{i \in \mathcal{D}, k \in [K_{\text{att}}]}. \quad (44)$$

The log is an indexed collection, so identical outcomes do not collapse into a single record. $\bar{s}_i$ averages attempts within a task and Util weights tasks equally. For binary scores, this estimates mean per-attempt success, not the probability of succeeding at least once in K_{att} attempts. The search maximizes training utility over $\mathfrak{H}(\mathcal{X}_{\text{edit}})$ under a finite development budget. The returned harness

maximizes recorded utility among the baseline and archived candidates, without a guarantee of global optimality.

Repeated evaluations produce distinct records. Let $\mathcal{D}_r$ denote the screening subset of $\mathcal{D}_{\text{tr}}$ used at round $r \geq 0$. Unmarked quantities refer to the designated full-training log (or the final test log when $\mathcal{D} = \mathcal{D}_{\text{te}}$). Superscript sc denotes a fresh screening evaluation, for example $s_{i,k}^{\text{sc}}(h)$, $\text{Log}^{\text{sc}}(h; \mathcal{D}_r)$, and $\text{Util}^{\text{sc}}(h; \mathcal{D}_r)$, defined by the same formulas. The screening and confirmation scores for an overlapping task need not coincide. Only the full-training record enters archive ranking.

The operation $\text{Eval}_{\Pi_{\text{eval}}}(h, \mathcal{D})$ produces this log under the fixed protocol. Agent failures remain scored outcomes and cannot be removed from its $|\mathcal{D}|K_{\text{att}}$ denominator. Infrastructure failures follow the declared repair-and-retry rule. An evaluation that remains invalid under the protocol is excluded from selection. Score comparisons use only complete, valid logs. The procedure below retains the recorded full-training log of each harness and reuses its restriction to a screening subset. This requires a stable evaluation environment. If the environment changes, compared candidates and incumbents require a common reevaluation protocol. Utilities measured under different environments are not ranked together.

From a Failure Trace to an Intervention. Using these evaluation records, the Evolver Agent converts the parent’s log into a testable intervention hypothesis. A diagnosis identifies the observed symptom, its supporting trace events, the edit location, and the behavior expected to change. For example, an empty final response after reasoning-budget exhaustion motivates a recovery intervention, while an untested code change motivates a finalization check. The diagnosis must connect the observed failure to a mechanism that the Evolver Agent can modify.

Each proposal contains an executable patch, a semantic descriptor, and a declared activation specification. Fixed before candidate evaluation, the activation specification identifies the recorded event used to determine whether the targeted mechanism executed. The Evaluator records activation and scores independently of the Evolver Agent’s explanation. The proposer may use a different model from the frozen Task Agent. Neither proposal generation nor patch application updates the Task Agent’s weights.

Development and Deployment Cost. Development includes diagnosis, proposal generation, screening, and confirmation. Deployment cost measures only execution of the selected harness. Both candidate and parent operate under the same externally fixed resource ceilings in Π_{eval} , although configuration edits may change how that allowance is used. The utility in Eq. (43) contains no implicit cost penalty, so token use and latency must be measured separately. The reference experiments use $K_{\text{att}} = 3$. Screening size n , proposal cap J , survivor cap $N_{\text{keep}} \leq J$, round cap R_{round} , and stagnation patience P are positive integer run settings, with $2 \leq n \leq |\mathcal{D}_{\text{tr}}|$.

4.3 Harness Gene Bank and Recombination

Semantic Archive. To support reuse across evolution rounds, the Harness Gene Bank retains evaluated candidates associated with different failure modes, including candidates below the current best utility. Let $\mathcal{W}_{\text{edit}} = \{\text{prompt, knowledge, runtime, config}\}$ and let $\mathcal{Y}_{\text{fail}}$ contain admissible pathology labels. At round $r \geq 0$, the bank is

$$\mathcal{B}_r : \mathcal{W}_{\text{edit}} \times \mathcal{Y}_{\text{fail}} \longrightarrow \mathfrak{H}(\mathcal{X}_{\text{edit}}) \cup \{\perp\}, \quad \mathcal{B}_0(\text{cell}) = \perp. \quad (45)$$

A cell $\text{cell} = (w, y)$ records the edit category and the diagnosed failure that motivates the modification. The observed vocabulary of y can grow, and newly introduced cells start empty, holding $\perp$. These edit categories are distinct from the four strategy roles in Eq. (39). For example, modifying a Memory strategy can be a runtime edit.

Each identified candidate carries a fixed descriptor $\text{desc}(h) = (w, y)$. The edit category is checked against the patch, while the pathology is inferred from the diagnosis. For a composite intervention, its primary cell and donor lineages are recorded before evaluation. Labels are not changed in response to scores. Thus, desc records the intervention’s assigned category and diagnosis. It is not uniquely determined by the executable content. The bank stores the complete harness and its evaluated log. One elite per cell implements quality-diversity search over semantic niches [43]. Pathology labels remain hypotheses, not established causes.

Parent Selection and Candidate Construction. Define the set of stored harnesses $\text{Im}^+(\mathcal{B}_r) = \{\mathcal{B}_r(\text{cell}) : \mathcal{B}_r(\text{cell}) \neq \perp\}$. The parent p_r is

$$p_r = \text{Best}(\{h_0\} \cup \text{Im}^+(\mathcal{B}_r)), \quad \text{Best}(S) \in \arg \max_{h \in S} \text{Util}(h; \mathcal{D}_{\text{tr}}). \quad (46)$$

For every nonempty finite S , Best resolves equal utility by a fixed creation order, with h_0 first. All candidates in this comparison have valid cached training logs. The empty initial bank therefore selects h_0 .

Given the parent, its diagnostic evidence, and the bank, proposal generation returns an ordered list

$$\text{Proposals}_r = \text{Evolve}(p_r, \text{Log}(p_r; \mathcal{D}_{\text{tr}}), \mathcal{B}_r; J) = (\text{prop}_1, \dots, \text{prop}_{J_r}), \quad 0 \leq J_r \leq J, \quad (47)$$

where $\text{prop}_j = (\Delta_j^{\text{patch}}, \text{cell}_j, \beta_j)$ contains a patch, cell, and activation specification. The materialized candidate is $h'_j = \text{Apply}(p_r, \Delta_j^{\text{patch}})$, with $\text{desc}(h'_j) = \text{cell}_j$. Invalid patches are rejected before task rollouts. Proposals either synthesize an intervention from the diagnosis or recombine changes from archived donor lineages with the parent. Donor provenance is retained and overlapping edits must be resolved before materialization. A recovery rule and a final-verification rule, for example, can act at different stages. The combined harness requires its own evaluation because the components may interact and their gains need not be additive. Both proposal types follow the same screening procedure.

Competitive Archive Update. Screening considers proposals in their recorded order and retains the first N_{keep} that pass. Each survivor is evaluated on all training tasks. Let Confirmed_r contain the survivors whose full-training logs are protocol-valid. The competitors for cell cell are

$$\text{Comp}_r(\text{cell}) = (\{\mathcal{B}_r(\text{cell})\} \setminus \{\perp\}) \cup \{h \in \text{Confirmed}_r : \text{desc}(h) = \text{cell}\}. \quad (48)$$

The update is

$$\mathcal{B}_{r+1}(\text{cell}) = \begin{cases} \text{Best}_{\text{cell}}(\text{Comp}_r(\text{cell})), & \text{Comp}_r(\text{cell}) \neq \emptyset, \\ \perp, & \text{Comp}_r(\text{cell}) = \emptyset. \end{cases} \quad (49)$$

Here $\text{Best}_{\text{cell}}$ maximizes full-training utility, retains the incumbent on a tie, and otherwise retains the earliest candidate in proposal order among tied maxima. Equivalently, process confirmed candidates sequentially and replace only an empty cell or a strictly weaker incumbent. This resolves competition when several candidates target the same cell.

Archive admission and parent selection use different comparisons. A candidate can outperform its parent on the screening subset yet fall below it on the full training set. It may still fill a new cell or replace a weaker cell incumbent, but becomes a parent only if Eq. (46) selects it. With fixed cached utilities, occupied-cell utility and the best available utility cannot decrease. However, neither property guarantees improvement on unseen tasks. HarnessDev reports this gap empirically: harness gains obtained by evolution can be unstable, transfer only partially to held-out tasks, and depend strongly on the model that executes the harness [70].

4.4 Gated Harness Screening and Selection

Validity and Activation. At round r , sample $\mathcal{D}_r \subseteq \mathcal{D}_{\text{tr}}$ uniformly without replacement, with $|\mathcal{D}_r| = n$, and use it for every candidate-parent comparison in that round. Record the sample and proposal order. The parent's measurements are the corresponding entries of its full-training log. Each candidate receives K_{att} attempts per sampled task under Π_{eval} . Let $\text{ValidLog}(\text{Log}) \in \{0, 1\}$ indicate a complete protocol-valid log. The gates are

$$g_{\text{valid}}(h'; \mathcal{D}_r) = \text{ValidLog}(\text{Log}^{\text{sc}}(h'; \mathcal{D}_r)), \quad (50)$$

$$g_{\text{act}}(h'; \mathcal{D}_r) = \mathbb{I} \left[\sum_{i \in \mathcal{D}_r} \sum_{k=1}^{K_{\text{att}}} \text{active}_{i,k}(h') > 0 \right]. \quad (51)$$

The activation indicator is $\text{active}_{i,k}(h') = \beta(\tau_{i,k}^{\text{sc}}(h'), \text{meta}_{i,k}^{\text{sc}}(h')) \in \{0, 1\}$, where β is the declared activation predicate of the proposal that produced h' . It detects an instrumented intervention in the candidate's increment over h_0 , such as execution of a recovery branch or actual injection of a new instruction. The predicate must exclude unrelated log events. Recombined mechanisms retain

their provenance and separate event records. The existential gate establishes that an instrumented intervention ran at least once. It neither requires every donor mechanism to execute nor establishes the causal contribution of the newest edit. The subsequent utility comparison includes *all* sampled tasks, including those without activation.

Paired Improvement Screening. Validity and activation alone do not establish improvement. For a protocol-valid candidate h' and parent p_r , define one paired difference per task:

$$\gamma_i = \bar{s}_i^{\text{sc}}(h') - \bar{s}_i(p_r), \quad \bar{\gamma}_r = \frac{1}{n} \sum_{i \in \mathcal{D}_r} \gamma_i, \quad (52)$$

$$\hat{\sigma}_\gamma^2 = \frac{1}{n-1} \sum_{i \in \mathcal{D}_r} (\gamma_i - \bar{\gamma}_r)^2, \quad \hat{\text{se}}_r = \frac{\hat{\sigma}_\gamma}{\sqrt{n}}. \quad (53)$$

Here $\bar{\gamma}_r = \text{Util}^{\text{sc}}(h'; \mathcal{D}_r) - \text{Util}(p_r; \mathcal{D}_r)$ is mean gain, $\hat{\sigma}_\gamma^2$ is the sample variance of task-level gains, and $\hat{\text{se}}_r$ is their estimated standard error. The notation suppresses dependence on the candidate for readability. Averaging the K_{att} attempts first keeps the task as the measurement unit, so the sample size is n , not nK_{att} . Pairing matches task identity and does not assume that attempt indices share identical random draws.

Define the paired improvement statistic and its operational boundary cases by

$$t_{\text{pair}} = \begin{cases} \bar{\gamma}_r / \hat{\text{se}}_r, & \hat{\text{se}}_r > 0, \\ +\infty, & \hat{\text{se}}_r = 0, \bar{\gamma}_r > 0, \\ -\infty, & \hat{\text{se}}_r = 0, \bar{\gamma}_r < 0, \\ 0, & \hat{\text{se}}_r = 0, \bar{\gamma}_r = 0. \end{cases} \quad (54)$$

The nondegenerate expression has the form of a paired Student t -statistic, since its denominator uses an estimated variance. The screening rule uses the fixed standard-normal cutoff $\lambda_{\text{screen}} = 1.96$ without interpreting it as an exact finite-sample two-sided 5% test. A Student t interpretation requires assumptions on the task differences and a critical value with $n - 1$ degrees of freedom. Zero-variance cases follow the explicit convention above and are flagged in the log. No calibrated p -value is assigned to an infinite statistic.

For a declared $\lambda_{\text{screen}} > 0$, the paired-gain screen is

$$g_{\text{sig}}(h', p_r; \mathcal{D}_r) = \mathbb{I}[\bar{\gamma}_r > 0] \mathbb{I}[t_{\text{pair}} \geq \lambda_{\text{screen}}]. \quad (55)$$

The composite decision rejects invalid or inactive candidates before assessing paired gain:

$$g_{\text{screen}}(h', p_r; \mathcal{D}_r) = \begin{cases} g_{\text{sig}}(h', p_r; \mathcal{D}_r), & g_{\text{valid}} = g_{\text{act}} = 1 \text{ and } n \geq 2, \\ 0, & \text{otherwise.} \end{cases} \quad (56)$$

Both operational gates in this expression are evaluated on $(h', \mathcal{D}_r)$. The parent log must also be valid. An invalid log never reaches the arithmetic in Eqs. (52) and (53). Algorithm 3 makes the rejection order explicit. In the algorithms, $\text{Log}_{h'}^{\text{sc}}$ abbreviates the fresh screening record $\text{Log}^{\text{sc}}(h'; \mathcal{D})$ for the supplied subset, and Log_h abbreviates the cached record $\text{Log}(h; \mathcal{D}_{\text{tr}})$. The supplied proposal's activation predicate is β . The screening algorithm suppresses the round subscript on its gain statistic because it handles one comparison at a time. These equations define the candidate screening procedure. Appendix B.4 describes the experiment source and the conventions used to complete the algorithm specification.

Complete Search and Final Selection. Combining the screening and archive rules, Algorithm 4 holds the parent and archive fixed while a round's proposals are screened. Only complete full-training evaluations can update the bank. If no proposal passes, the bank remains unchanged and the stagnation counter advances. With R_{round} rounds, at most J proposals and N_{keep} confirmations per round, the procedure uses at most $K_{\text{att}}[(1 + R_{\text{round}}N_{\text{keep}})|\mathcal{D}_{\text{tr}}| + R_{\text{round}}Jn]$ scored task attempts, excluding infrastructure retries and proposer computation. An additional token or time ceiling can stop development early. An unfinished evaluation is never used for selection. In the algorithm, Survivors_r is the ordered list of screening survivors and stale counts consecutive rounds without a cell update.

Algorithm 3: Candidate Screening

Input: proposal $\text{prop} = (\Delta^{\text{patch}}, \text{cell}, \beta)$, parent p with valid training log, subset $\mathcal{D}$, protocol Π_{eval} , admissible space $\mathcal{H}(\mathcal{X}_{\text{edit}})$, threshold λ_{screen} .

Output: candidate h' and screening log, or rejection $\perp$ with a recorded reason.

- 1 **if** $|\mathcal{D}| < 2$, **return** $\perp$ (insufficient tasks).
- 2 Materialize $h' \leftarrow \text{Apply}(p, \Delta^{\text{patch}})$ and attach $\text{desc}(h') = \text{cell}$, predicate β , and lineage.
- 3 **if** application fails, $h' \notin \mathcal{H}(\mathcal{X}_{\text{edit}})$, or its descriptor/activation contract is invalid, **return** $\perp$ (inadmissible proposal).
- 4 $\text{Log}_{h'}^{\text{sc}} \leftarrow \text{Eval}_{\Pi_{\text{eval}}}(h', \mathcal{D})$, including the declared bounded infrastructure retries.
- 5 **if** $\text{ValidLog}(\text{Log}_{h'}^{\text{sc}}) = 0$, **return** $\perp$ (invalid measurement) without computing a gain.
- 6 **if** $\sum_{i \in \mathcal{D}} \sum_{k=1}^{K_{\text{att}}} \beta(\tau_{i,k}^{\text{sc}}(h'), \text{meta}_{i,k}^{\text{sc}}(h')) = 0$, **return** $\perp$ (inactive intervention).
- 7 Compute $\gamma_i, \bar{\gamma}, \hat{\sigma}_{\gamma}^2$ against the parent’s restricted log using Eqs. (52) and (53).
- 8 Compute t_{pair} by Eq. (54) and record whether the variance is zero.
- 9 **if** $\bar{\gamma} \leq 0$ or $t_{\text{pair}} < \lambda_{\text{screen}}$, **return** $\perp$ (insufficient paired gain).
- 10 **return** $(h', \text{Log}_{h'}^{\text{sc}})$.

Algorithm 4: Semantic Gene-Bank Search

Input: valid h_0 , admissible space $\mathcal{H}(\mathcal{X}_{\text{edit}})$, training set $\mathcal{D}_{\text{tr}}$, protocol Π_{eval} with K_{att} attempts, settings $(n, J, N_{\text{keep}}, R_{\text{round}}, P, \lambda_{\text{screen}})$.

Output: training-selected harness $\hat{h}$ and its cached evaluation record. Test tasks are not inputs to the search.

- 1 $\text{Log}_{h_0} \leftarrow \text{Eval}_{\Pi_{\text{eval}}}(h_0, \mathcal{D}_{\text{tr}})$. **If** $\text{ValidLog}(\text{Log}_{h_0}) = 0$, terminate with an evaluation error.
- 2 Cache Log_{h_0} , $\text{Util}(h_0; \mathcal{D}_{\text{tr}})$. Initialize $\mathcal{B}_0 \leftarrow \perp$ in every cell and set $r \leftarrow 0$, stale $\leftarrow 0$.
- 3 **while** $r < R_{\text{round}}$ and stale $< P$:
- 4 $p_r \leftarrow \text{Best}(\{h_0\} \cup \text{Im}^+(\mathcal{B}_r))$.
- 5 Generate ordered Proposals_r by Eq. (47) and sample $\mathcal{D}_r$ of size n without replacement.
- 6 $\text{Survivors}_r \leftarrow []$, $\text{Confirmed}_r \leftarrow \emptyset$.
- 7 **for each** prop_j in Proposals_r , in order:
- 8 **if** $|\text{Survivors}_r| = N_{\text{keep}}$, **break**.
- 9 Run Algorithm 3 with $(\text{prop}_j, p_r, \mathcal{D}_r, \Pi_{\text{eval}}, \mathcal{H}(\mathcal{X}_{\text{edit}}), \lambda_{\text{screen}})$.
- 10 **if** accepted, append the returned candidate to Survivors_r .
- 11 **for each** h' in Survivors_r , in order:
- 12 $\text{Log}_{h'} \leftarrow \text{Eval}_{\Pi_{\text{eval}}}(h', \mathcal{D}_{\text{tr}})$.
- 13 **if** $\text{ValidLog}(\text{Log}_{h'}) = 1$: cache its log and utility, and set $\text{Confirmed}_r \leftarrow \text{Confirmed}_r \cup \{h'\}$.
- 14 Update $\mathcal{B}_{r+1}$ by Eqs. (48) and (49).
- 15 stale $\leftarrow 0$ if any cell changes, otherwise stale $\leftarrow$ stale + 1.
- 16 $r \leftarrow r + 1$.
- 17 $\hat{h} \leftarrow \text{Best}(\{h_0\} \cup \text{Im}^+(\mathcal{B}_r))$. **Return** $\hat{h}$ and its cached record.

After selection, freeze $\hat{h}$ and evaluate it and h_0 under the same held-out protocol. The reported difference is $\text{Util}(\hat{h}; \mathcal{D}_{\text{te}}) - \text{Util}(h_0; \mathcal{D}_{\text{te}})$. Test outcomes do not revise proposals, thresholds, archive cells, or the selected harness. Repeated adaptive screening on training tasks guides the search but does not provide a family-wise significance guarantee. Activation evidence, archive competition, and the held-out comparison address separate questions: whether the intervention executed, how the candidate ranked on training tasks, and how the selected harness performed on unseen tasks.

5 Memory System

An agent’s interaction history contains information that can remain useful beyond the active context, including user constraints, evidence supporting earlier decisions, and procedures learned during execution. To preserve this information across turns and sessions, Raven uses EverOS [23, 15]. Its memory lifecycle separates episodic trace formation, semantic consolidation, and query-dependent recollection, with distinct user and agent memory tracks and asynchronous persistence. Raven connects user memory to context assembly and agent experience to Skill Forge. We first describe how memories acquire structure, then explain their retrieval and use in Raven. Figure 9 summarizes

the formation and consolidation process, while Figure 10 shows the retrieval path used for Raven’s user memory. Implementation defaults are collected in Appendix B.3.

5.1 Episodic Trace Formation

A useful memory must preserve enough context to interpret an event without replaying the entire conversation. Let $\text{Hist} = (\text{msg}_1, \dots, \text{msg}_{T_{\text{msg}}})$ be an ordered interaction stream of T_{msg} messages. Each message records its content, sender, role, timestamp, and any associated tool interaction. A boundary detector groups contiguous messages into segments $\text{seg}_i = (\text{msg}_{\text{end}_{i-1}+1}, \dots, \text{msg}_{\text{end}_i})$, where i indexes segments, end_i is the last message index of segment i , and $\text{end}_0 = 0$. An LLM examines a sliding context window and closes a segment when it detects a semantic boundary such as a topic change. Grouping related exchanges preserves the context needed to resolve references and explain decisions. Messages after the last accepted boundary remain buffered until a later boundary or an explicit flush.

We describe the information associated with segment i as a *MemCell*:

$$\text{Cell}_i = (\text{ep}_i, \mathcal{F}_i, \mathcal{P}_i, \text{meta}_i). \quad (57)$$

Here ep_i is an episode narrative, $\mathcal{F}_i$ is a set of atomic facts, $\mathcal{P}_i$ is a set of forward-looking statements, and meta_i is grounding metadata. The narrative rewrites the interaction in the third person, resolves coreferences, and removes conversational repetition while retaining the event and its context. Atomic facts split that narrative into statements that can be matched individually to a later query. For example, a discussion about setting up a project can yield a narrative explaining the environment decision and a fact stating the selected Python version. The narrative explains the decision, while the fact provides a precise retrieval target.

Foresight represents information whose usefulness depends on time. Write an element of $\mathcal{P}_i$ as $(p, t_{p,0}, t_{p,1})$, where p is a plan or inferred temporary state, and $[t_{p,0}, t_{p,1}]$ is its estimated validity interval. At time t_{now} , the time-valid subset is

$$\mathcal{P}_i(t_{\text{now}}) = \{(p, t_{p,0}, t_{p,1}) \in \mathcal{P}_i : t_{p,0} \leq t_{\text{now}} \leq t_{p,1}\}. \quad (58)$$

An upcoming deadline, for instance, has a different temporal status from a durable preference. The validity interval allows expired predictions to be filtered out. EverOS can derive foresight from the episode or the shared source segment.

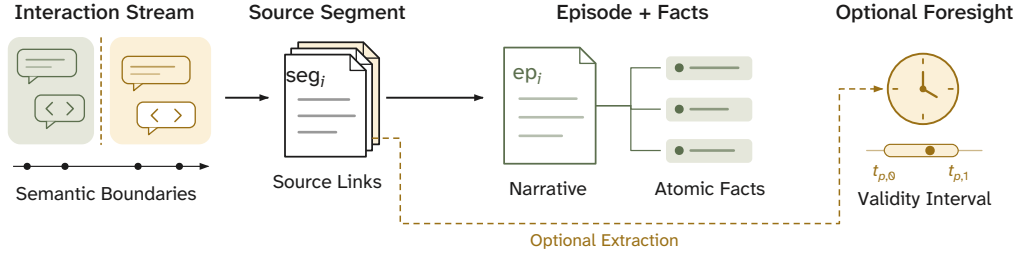
The metadata meta_i connects the extracted representations to their origin. It includes time and source identifiers, together with session and ownership information. EverOS separates application and project scope, then partitions records by owner type and owner identifier. User records belong to a user, while cases and skills belong to an agent. A session identifier connects records from one interaction within these partitions. Thus, a session describes where an experience occurred, while ownership determines the collection in which it can be recalled. Raven’s ordinary recall names exactly one user or agent identifier and uses the backend’s default application and project scope.

In EverOS, Eq. (57) describes a logical association among representations that are stored separately. A shared boundary stage creates the source segment and its identifier. The user pipeline synthesizes and writes the episode, after which background strategies extract facts and update the higher-level representations. Facts retain a parent link to the episode. In the default agent mode, the same source segment also enters the agent pipeline, preserving tool calls and their results for case extraction. This shared origin connects what the user discussed with what the agent attempted. Raven submits completed turns and flushes the remaining buffer, so its turn boundaries also influence memory granularity.

5.2 Semantic Consolidation

Individual episodes describe local events. To identify recurring topics and persistent attributes across interactions, consolidation groups related events into a *MemScene*. EverOS implements the user-side grouping as online clustering of episode embeddings. Let $\mathbf{z}_i = \text{Enc}(\text{ep}_i) \in \mathbb{R}^{d_{\text{emb}}}$ be the embedding of episode i , where Enc is the embedding model and d_{emb} its output dimension. A scene scene_j stores its member episode identifiers, centroid $\boldsymbol{\mu}_j \in \mathbb{R}^{d_{\text{emb}}}$, member count n_j , and latest member timestamp $t_{j,\text{last}}$. These quantities summarize the evidence already assigned to that scene.

(a) From Interaction to Memory



(b) From Episodes to Persistent Knowledge

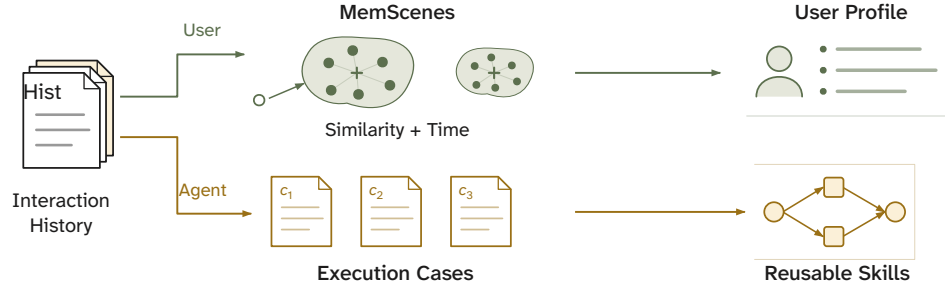


Figure 9: Memory formation and consolidation in EverOS. (a) Semantic boundaries group dialogue and tool interactions into a source segment seg_i . Its episode ep_i anchors atomic facts. The dashed branch derives optional foresight with validity interval $[t_{p,0}, t_{p,1}]$. (b) Across interaction history Hist, episodes, shown as points, form MemScenes around centroids marked by crosses. These scenes select evidence for profile updates. The agent track consolidates execution cases c_1, c_2, c_3 into reusable procedures, depicted by a small dependency graph. The diagram shows representation and data dependencies. Persistence and indexing proceed separately.

Assignment combines semantic similarity with temporal proximity. For episode timestamp t_i , let $\mathcal{J}_{\text{valid}}$ index existing scenes with nonempty centroids of the embedding dimension. Define the eligible scene indices and the similarity score as

$$\mathcal{J}_i = \{j \in \mathcal{J}_{\text{valid}} : |t_i - t_{j,\text{last}}| \leq \Delta_{\text{time}}\},$$

$$\text{sim}(\mathbf{z}_i, \boldsymbol{\mu}_j) = \frac{\mathbf{z}_i^\top \boldsymbol{\mu}_j}{(\|\mathbf{z}_i\|_2 + \epsilon_{\text{num}})(\|\boldsymbol{\mu}_j\|_2 + \epsilon_{\text{num}})}. \quad (59)$$

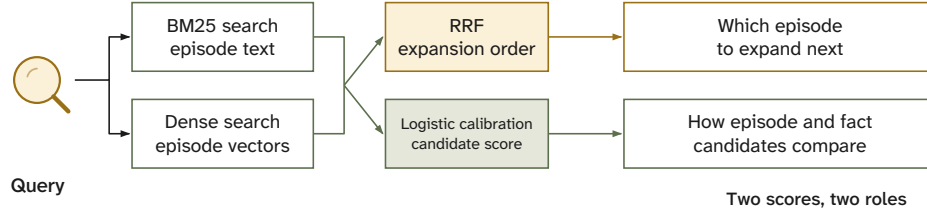
Here Δ_{time} is the allowed time gap and $\epsilon_{\text{num}} = 10^{-9}$ prevents division by zero. Scenes with no centroid are excluded. When $\mathcal{J}_i$ is nonempty, the highest-scoring eligible scene is $j^* \in \arg \max_{j \in \mathcal{J}_i} \text{sim}(\mathbf{z}_i, \boldsymbol{\mu}_j)$, with ties resolved by the first scene encountered in the backend’s candidate order. Episode i joins that scene if $\text{sim}(\mathbf{z}_i, \boldsymbol{\mu}_{j^*}) \geq \theta_{\text{sim}}$, where θ_{sim} is the similarity threshold. Otherwise, a new singleton scene is created with centroid $\mathbf{z}_i$, count one, and timestamp t_i . For an accepted assignment, the updates are

$$\boldsymbol{\mu}_{j^*}^+ = \frac{n_{j^*} \boldsymbol{\mu}_{j^*} + \mathbf{z}_i}{n_{j^*} + 1}, \quad n_{j^*}^+ = n_{j^*} + 1, \quad t_{j^*,\text{last}}^+ = \max(t_{j^*,\text{last}}, t_i). \quad (60)$$

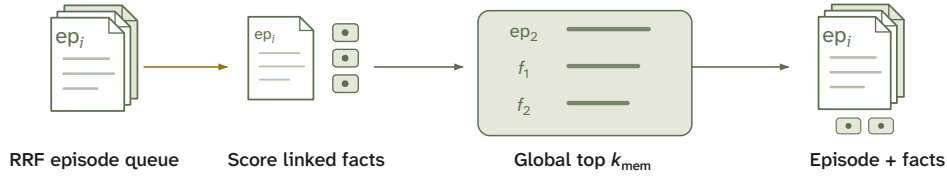
The superscript $+$ denotes the state after insertion. The member identifier is appended as well. The time window constrains scene assignment without limiting episode retention or searchability. Consequently, a recurring topic can occupy several scenes when its occurrences are separated in time.

Profile evolution uses the resulting groups to select evidence for a compact account of the user. Let Prof_u denote the current profile of user u , with explicit attributes and inferred traits. Explicit attributes include stated preferences and goals, while inferred traits summarize patterns across interactions. EverOS uses recently updated scenes to select the underlying source segments for a profile update. If t_{Prof_u} is the previous profile timestamp and j_{cur} is the scene updated by the current insertion, this

(a) Episode Recall and Scoring



(b) Fact Expansion and Response



A fact can displace its parent. The response restores episode context

Figure 10: Raven’s user-memory retrieval through EverOS HYBRID. (a) BM25 and dense search retrieve episodes. Reciprocal rank fusion (RRF) determines their expansion order. A separate calibrated score ranks them for final selection. (b) The query scores facts belonging to each expanded episode. Episodes and facts compete within a bounded candidate set. A selected fact is returned under its parent episode, which restores its context. The positive integer k_{mem} is the backend candidate-set cap.

selection is

$$\mathcal{J}_u^+ = \{j : t_{j,\text{last}} > t_{\text{Prof}_u}\} \cup \{j_{\text{cur}}\}, \quad \text{Prof}_u^+ = \text{UpdateProfile}(\text{Prof}_u, \text{Segments}(\mathcal{J}_u^+)). \quad (61)$$

All scenes in this expression belong to user u in the selected application and project. The operator $\text{Segments}(\mathcal{J})$ collects the available source segments linked to scenes indexed by $\mathcal{J}$, ordered by time. UpdateProfile is an LLM extraction step conditioned on those segments and the previous profile. With no prior profile, the extractor initializes one from the available source segments. Including j_{cur} keeps the current cluster eligible even when an older interaction is ingested after a newer profile update. Without embeddings, EverOS can select recent source segments by timestamp and update the profile without geometric clustering.

Beyond profile updates, EverOS provides optional episode reflection for consolidation at a longer timescale. Reflection selects a cluster, synthesizes a merged episode, extracts facts from that merged narrative, and marks the superseded episodes and facts as deprecated. Online assignment preserves individual events, whereas reflection can replace fragmented retrieval units with a consolidated account. This operation is separate from the ordinary cluster and profile updates described above.

5.3 Reconstructive Retrieval

EverOS supports several retrieval methods. Raven’s user-memory call uses the HYBRID method, which searches episodes first and then lets their atomic facts compete for a bounded result set. Retrieval remains within the selected user partition, following the search and response flow in Figure 10.

Episode Recall and Expansion Order. Let the positive integer k_{mem} bound the backend’s episode-and-fact candidate set. For query q , EverOS retrieves episodes through BM25 search over the

episode text and dense search over episode embeddings. Lexical search matches query terms, while dense search can also match paraphrases. Let $L_\ell^{\text{hit}}(q)$ be the ranked episode list from channel $\ell \in \{\text{lex}, \text{dense}\}$, with one-based ranks. RRF gives episode ep_i the expansion priority

$$\text{RRF}_i(q) = \sum_{\substack{\ell \in \{\text{lex}, \text{dense}\} \\ \text{ep}_i \in L_\ell^{\text{hit}}(q)}} \frac{1}{c_{\text{RRF}}^{\text{mem}} + \text{rank}_{L_\ell^{\text{hit}}(q)}(\text{ep}_i)}, \quad (62)$$

where $c_{\text{RRF}}^{\text{mem}} > 0$ controls rank damping. A channel contributes only when it returns the episode. RRF determines expansion order, while calibrated scores determine final relevance.

Final selection uses a separate calibrated relevance score. Let $\text{dense}_i(q)$ be episode ep_i 's dense similarity and $\text{lex}_i(q)$ its BM25 score, with zero for a channel that did not recall it. The backend's logistic calibration has the form

$$\text{Rel}_i(q) = \text{sigmoid}(w_{\text{dense}} \text{dense}_i(q) + w_{\text{lex}} \text{lex}_i(q) + b_{\text{off}}), \quad \text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}. \quad (63)$$

Here w_{dense} , w_{lex} , and b_{off} are fixed backend coefficients. Calibration puts episode and fact candidates on a common ranking scale.

Fact Expansion and Response. For each recalled episode, EverOS fetches linked atomic facts by parent identifier and scores them against the query embedding. The initial BM25 search is over episodes, so a fact $f \in \mathcal{F}_i$ uses its parent's lexical score $\text{lex}_i(q)$, together with its own dense similarity $\text{dense}_f(q)$:

$$\text{Rel}_f(q) = \alpha_{\text{mix}} \text{sigmoid}(w_{\text{dense}} \text{dense}_f(q) + w_{\text{lex}} \text{lex}_i(q) + b_{\text{off}}) + (1 - \alpha_{\text{mix}}) \text{Rel}_i(q). \quad (64)$$

The mixing weight satisfies $\alpha_{\text{mix}} \in [0, 1]$. The default setting is $\alpha_{\text{mix}} = 1$, allowing a precise fact match to outrank its parent episode. The backend initializes a set of at most k_{mem} candidates and expands episodes in RRF order. The best facts from an expanded episode may enter if space remains or their calibrated score exceeds the lowest retained score. When one enters, the parent episode is removed from this competition to avoid spending two slots on the same evidence. Expansion stops after all candidates are considered or the selected identifiers stay unchanged for a bounded number of steps.

The response restores episode context after this competition. Selected facts are grouped beneath their parent episode. If that episode was removed from the candidate set, the backend recovers it from the episode recall pool and assigns it the highest score among its retained facts. The API therefore returns episode objects with nested `atomic_facts`, possibly fewer than k_{mem} distinct episodes. Raven renders episode summaries, falling back to the episode body.

5.4 User Context Assembly

To use the retrieved information during execution, Raven combines recalled semantic memory with a host-maintained Markdown archive in the workspace. The archive contains profile sections and episode notes. Its consolidator appends an episode note when history is archived and revises the profile when enough new notes share a tag. For an ordinary turn, the host selects the two profile sections with the greatest lexical overlap with the incoming message, together with all Notes sections. This gives the host a compact source of persistent user context alongside query-dependent backend recall.

Let the positive integer k_{recall} be the adapter's final result limit, distinct in role from the backend cap k_{mem} . EverOS user recall requests episodes and the user's profile. The backend retrieves episodes by the HYBRID method above and fetches the profile separately. Raven renders the profile as key-value lines within a character budget. It then places the profile in the same result list as episodes with an adapter-assigned score of 1.0 and retains at most k_{recall} items. This score is a placement convention, not a relevance estimate or confidence in the profile. The adapter's result limit includes any retained profile, while the server's episode limit excludes it.

Let $\text{Archive}(q)$ be the selected host archive text and $\text{Recall}(q)$ the rendered user recall. Raven assembles the memory segment as

$$\text{MemContext}(q) = \text{Archive}(q) ++ \text{Fence}(\text{Recall}(q)), \quad (65)$$

where $++$ denotes ordered text concatenation and Fence marks recalled content as untrusted contextual data. This segment can contain both host and EverOS profile text. If either layer is unavailable, it contributes an empty segment and the turn continues. When the backend lacks an embedding model, the adapter requests keyword retrieval.

5.5 Agent Experience and Cross-Session Reuse

For procedural reuse, the agent memory track extracts procedures from execution evidence. Given the source segment produced during formation, a case extractor records the task intent, attempted approach, reusable insight, and an estimated quality score. Related cases are grouped by task intent and used to create or revise skills with links to their supporting cases. User profiles describe the user’s context, whereas cases and skills capture how an agent approached a class of tasks. Agent recall supplies these records to Skill Forge, whose admission, clustering, and skill-update rules are specified in Section 6.3. They do not enter the user memory segment in Eq. (65).

Cross-session reuse requires memory content to survive independently of the active process and its search index. EverOS stores episodes, facts, profiles, cases, and skills as Markdown records. SQLite maintains operational state, source-segment payloads, and cluster membership, while a derived index supports lexical and vector retrieval. File changes are propagated to the index asynchronously. The backend uses LanceDB by default, supports Milvus as an alternative, and maintains approximate nearest-neighbor indexes for sufficiently large vector collections. These indexing choices change retrieval execution without changing the memory representations above. The Markdown records can be used to rebuild the search index. Raven serializes writes within a session and retries failed stores. Queue limits and retry settings are recorded in Appendix B.3.

For multi-agent execution, Raven also reads the records associated with a particular worker session. This is a session-filtered read of user episodes and agent cases. When a worker uses its own memory backend, the host reads that backend. When memory is formed from a captured worker trace, the host submits the trace with the worker’s identities and session identifier. The resulting memory artifacts can be made available to downstream graph nodes through the paths described in Section 3.3. Task completion releases execution dependencies while memory recording continues asynchronously.

6 Skill Forge

Skill Forge retrieves reusable procedures for an agent and updates them using execution experience. A skill describes a task class, instructions for performing it, and any required resources. The method has three stages: build a curated catalog, select procedures for the current task, and revise experience-derived skills for later use. The catalog and reference retrieval pipeline build on our prior work, SkillCorpus [64]. SkillCorpus introduced the catalog construction in Section 6.1, including corpus deduplication, quality assessment, and release admission. It also released the reference stack of query rewriting, recall, reranking, and LLM selection on which Section 6.2 builds, and Section 7.7 summarizes its published experiments. Raven extends this work with an online router that fuses the SkillHub catalog with local and EverOS skills (Section 6.2). Stored turns enter EverOS, whose agent pipeline converts execution cases into procedures (Section 6.3). Figure 11 shows how corpus admission, task-specific selection, and experience-driven revision connect.

6.1 Curated Skill Catalog

SkillHub is the live skill catalog that Raven queries. Its corpus and retrieval design build on SkillCorpus [64], whose curation pipeline this subsection summarizes.

Skill Representation. We denote a skill record by a sans-serif s , distinguishing it from the analytical state s in Section 2, and represent it as

$$s = (\text{id}_s, \text{name}_s, \text{desc}_s, \text{body}_s, \mathcal{R}_s^{\text{res}}, \text{meta}_s), \quad (66)$$

where id_s is its source-native identifier, name_s its display name, desc_s its applicability and trigger description, body_s its procedural body, $\mathcal{R}_s^{\text{res}}$ its bundled resources, and meta_s its metadata. Metadata can include origin, curation scores, owner, and supporting-case identifiers. In the SKILL.md representation, name and description occupy frontmatter, the procedure is Markdown, and optional directories

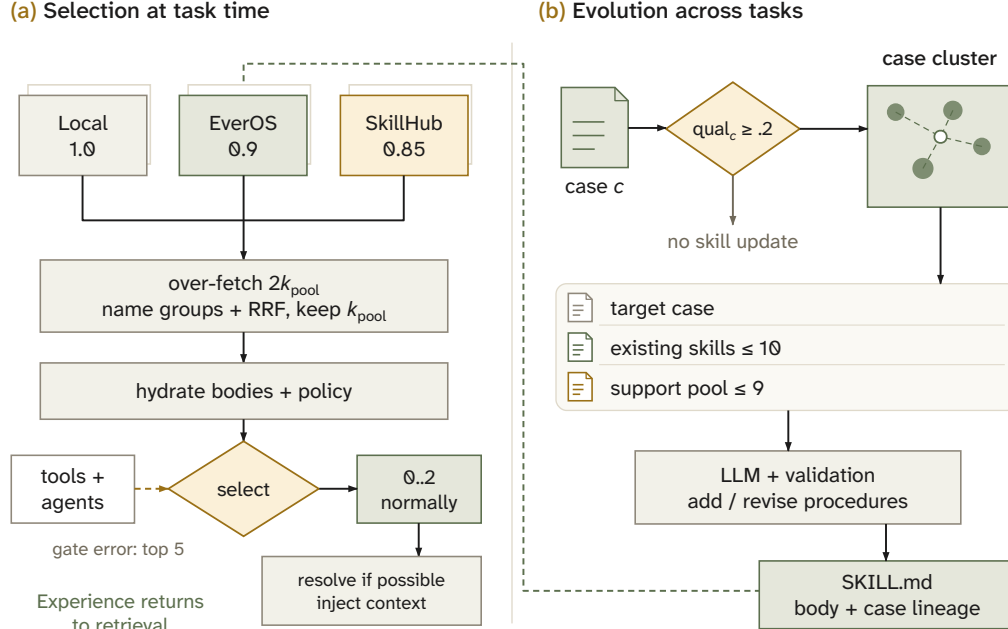


Figure 11: Skill selection and experience-driven updates. (a) Ranked source hits are grouped by name and fused before body retrieval, runtime policy checks, and LLM selection. The normal gate selects a bounded list, while a gate failure uses a separate ranked fallback. Resource resolution is attempted after selection. (b) A case enters an intent cluster and conditions skill operations together with existing skills and a bounded set of supporting cases. Persisted revisions become candidates for later EverOS retrieval after indexing. The positive integer k_{pool} is the fused skill-pool cap. Numeric labels show the implementation defaults in Appendix B.3.

hold scripts, references, or assets. Separating these fields permits compact discovery while retaining the instructions used at execution time. A name is not a globally unique identifier.

Corpus Construction and Deduplication. SkillCorpus aggregates public skills through a source registry and applies six curation stages. The pipeline parses each entry, filters malformed or length-ineligible entries, removes duplicates, assesses quality and assigns issue flags, applies release admission rules, and attaches source priors, retrieval embeddings, and index entries. The reference release contains 96,401 active skills. This count describes a fixed corpus snapshot and may differ from the live SkillHub catalog.

Deduplication first collapses content- and name-fingerprint matches. The semantic stage embeds the remaining entries in a 1,024-dimensional space and compares pairs by cosine similarity. Similarity above 0.995 triggers automatic merging, while pairs in $(0.90, 0.995]$ receive LLM adjudication. Removing duplicate procedures reduces redundant retrieval and potential near-duplicate collisions in retriever training. This corpus-level procedure differs from the online router’s exact-name collapse in Section 6.2.

Quality Facets and Release Admission. After deduplication, the judge assigns integer utility, robustness, and safety subscores from $\{0, \dots, 10\}$. Dividing them by ten yields $\text{util}_s, \text{rob}_s, \text{safe}_s \in [0, 1]$. Utility evaluates the description’s scope and triggers. Robustness evaluates the body and its consistency with the description. Safety evaluates risks in following the instructions. Each of the nineteen issue flags imposes a cap on its assigned quality facet. Soft flags apply through these caps, without a separate score deduction.

For a parsed, deduplicated, judged record, let $\mathcal{F}_s^{\text{flag}}$ be its triggered flags, $\text{mal}_s \in \{0, 1\}$ a malware-pattern match, and $g_{\text{lic}}(s) \in \{0, 1\}$ acceptance under the release’s source license policy. The hard-flag set $\mathcal{F}_{\text{hard}}^{\text{flag}}$ contains `prompt_injection`, `cmd_injection`, `unsafe_exec`, `auth_bypass`,

and `csam_risk`. Release admission is

$$g_{\text{rel}}(s) = g_{\text{lic}}(s) \mathbb{I}[\text{mal}_s = 0] \mathbb{I}[\mathcal{F}_s^{\text{flag}} \cap \mathcal{F}_{\text{hard}}^{\text{flag}} = \emptyset] \mathbb{I}[\text{safe}_s \geq 0.3], \quad (67)$$

Only records that pass structural validation and deduplication reach this decision.

For admitted skills, safety attenuation and content quality are

$$\text{atten}(x) = \begin{cases} 0.5 + 0.5(x - 0.3)/0.4, & 0.3 \leq x \leq 0.7, \\ 1, & 0.7 < x \leq 1, \end{cases} \quad (68)$$

$$\text{qual}_{\text{content}}(s) = \text{atten}(\text{safe}_s) [0.50 \text{ util}_s + 0.35 \text{ rob}_s + 0.15 \text{ safe}_s]. \quad (69)$$

Thus $\text{atten}(0.3) = 0.5$ and $\text{atten}(0.7) = 1$. It is not evaluated for a safety-rejected record. Let $\text{src}(s)$ identify the corpus source and $p_{\text{src}}(\text{src}(s))$ be its prior in Appendix B.5. Define the structural bonus $\text{bonus}_s = 0.05 \mathbb{I}[\text{scripts}/\text{present}] + 0.02 \mathbb{I}[\text{references}/\text{present}]$. The admitted skill’s composite quality is

$$\text{qual}_{\text{cur}}(s) = \text{clip}_{[0,1]}(0.85 \text{ qual}_{\text{content}}(s) + 0.15 p_{\text{src}}(\text{src}(s)) + \text{bonus}_s), \quad (70)$$

where $\text{clip}_{[a,b]}(z) = \min(b, \max(a, z))$. A prior or structural bonus cannot override rejection by Eq. (67). The prior uses source-level content-quality statistics, not the final composite score, avoiding a circular definition. The resulting scores reflect curation rules and are not calibrated probabilities of downstream success. Each released skill also receives one of sixteen task-domain labels for organization.

Catalog Retrieval Artifacts. To search the resulting catalog, SkillCorpus provides two fine-tuned models, using Qwen3-Embedding-0.6B for retrieval and Qwen3-Reranker-0.6B for reranking. They use a retrieval field distilled to 3,000 characters from each body. The reference stack recalls and reranks candidates, then presents full bodies to an LLM selector returning zero to two skills. Raven’s integration combines several sources and supplies bounded body excerpts to its selection gate. Consequently, the corpus, its packaged retrieval models, and Raven’s online selector are distinct components of an evaluation.

6.2 Task-Aware Skill Retrieval and Selection

Query, Sources, and Budgets. Let x_{in} be the current request, $\mathcal{U}_{\text{tool}}$ the receiving agent’s available tools, and $\mathcal{A}_{\text{spec}} \subseteq \mathcal{A}$ its available specialist roster. The latter need not contain every executor in the theoretical pool, including the host’s nondelegating policy. An optional rewriter returns (need, q) , with $\text{need} \in \{0, 1\}$ and retrieval query q . If the rewriter returns a valid skip decision, the router supplies no skill segment. Otherwise, q retains the task class, domain, and required capabilities while removing irrelevant identifiers. A rewriter failure keeps retrieval enabled with x_{in} as the query. The gate subsequently sees this retrieval query, so query rewriting affects both retrieval and selection.

The positive integer k_{pool} caps the fused candidate pool. The retrieval sources are

$$\mathcal{J}_{\text{src}} = \{\text{local}, \text{memory}, \text{hub}\}.$$

Configured sources are queried concurrently for up to $2k_{\text{pool}}$ hits each, where `hub` denotes SkillHub. Local retrieval uses the indexed local catalog. The memory adapter recalls skills within its configured agent scope. SkillHub retrieval initially returns metadata. The router can pass session history, but an adapter may ignore it. A failed source contributes an empty list while other sources remain usable. Appendix B.3 collects the budgets for gated and ungated retrieval, fusion weights, and extraction settings. Candidate budgets limit the number of entries, not the token count of the final skill bodies.

Name-Group Fusion and Representative Selection. To combine the source rankings, let $L_j^{\text{hit}} = (h_{j1}, \dots, h_{jm_j})$ be source j ’s ranked hit list. A hit contains a qualified identifier $\text{qid}(h) = \text{source/native_id}$, display name $\text{name}(h)$, body, metadata, and source-native score $\text{score}_{\text{native}}(h)$. The display name is the `name_s` field of the underlying skill record. Positions $r = 1, \dots, m_j$ are one-based ranks. For every name n present in the lists, the weighted reciprocal-rank score is

$$\text{RRF}(n) = \sum_{j \in \mathcal{J}_{\text{src}}} \sum_{r=1}^{m_j} \mathbb{I}[\text{name}(h_{jr}) = n] \frac{w_j}{c_{\text{RRF}}^{\text{skill}} + r}. \quad (71)$$

Weights $w_j \geq 0$ are source preferences and $c_{\text{RRF}}^{\text{skill}} > 0$ controls rank damping. The formula sums hit occurrences, including repeated names within one source if supplied. It groups exact name matches without testing semantic equivalence.

Let $\text{Hits}_n = \{h_{jr} : \text{name}(h_{jr}) = n\}$ be the observed hits carrying a name. The representative whose content will be rendered is selected separately:

$$\text{rep}(n) \in \arg \max_{h \in \text{Hits}_n} \text{score}_{\text{native}}(h), \quad \text{Pool}_0 = \text{Take}_{k_{\text{pool}}}([\text{rep}(n)]_{n \text{ ordered by } \text{RRF}(n) \downarrow}). \quad (72)$$

Here $\text{Take}_{k_{\text{pool}}}$ retains the first k_{pool} entries of an ordered list. Ties in representative scores retain the first encountered hit. Ties in fused scores follow the order in which names first appear, determined by registered-source order and within-source rank. Thus RRF avoids comparing native score scales when ordering names, while representative selection still compares those scores. Rescaling a source’s scores can change the injected version without changing its name’s RRF position. The representative’s qualified identifier and contributing-source metadata preserve this distinction.

Body Retrieval, Policy Checks, and Model Selection. After fusion, the runtime removes blocked representatives from Pool_0 , then fetches missing SkillHub bodies and applies the configured detail policy. A failed detail fetch or policy refusal removes that candidate. Filtering occurs after the top- k_{pool} fusion, without refilling from lower-ranked names. Let Pool be the remaining ordered pool, with $|\text{Pool}| \leq k_{\text{pool}}$. Catalog release admission (Eq. (67)) and runtime admission are separate checks. Their default safety thresholds differ, as recorded in Appendix B.3.

To select procedures from this filtered pool, the LLM gate receives q , qualified identifiers, and descriptions and normalized body excerpts under separate character limits. Available tool and specialist rosters are supplied when obtainable. The prompt asks the gate to plan the task, reject procedures requiring unavailable capabilities, and avoid procedures whose work is already covered by a specialist. Procedures shaping the current agent’s planning or verification can still be useful. The gate assesses compatibility from the supplied excerpts and rosters. However, this judgment does not guarantee that all requirements for executing the selected skill are satisfied.

Let IDs be the ordered identifier list from a successfully parsed gate response, $k_{\text{sel}} \geq 1$ the normal selection cap, and $k_{\text{fb}} \geq 1$ the failure-fallback cap, both integers. Define $\text{Lookup}(\text{IDs}, \text{Pool})$ to replace each known identifier by its pool entry, ignore unknown identifiers, and preserve list order. The selected list is

$$S_{\text{sel}} = \begin{cases} \text{Take}_{k_{\text{sel}}}(\text{Lookup}(\text{IDs}, \text{Pool})), & \text{gate response parses successfully,} \\ \text{Take}_{k_{\text{fb}}}(\text{Pool}), & \text{gate call or response parsing fails,} \\ \text{Pool}, & \text{gate is disabled.} \end{cases} \quad (73)$$

The response fields are specified in Appendix B.2. A successful empty identifier list remains empty without triggering fallback. With the default caps, fallback can select more entries than a normal response. The current parser preserves repeated known identifiers, so the cap bounds list entries, not necessarily distinct procedures. Gate output membership is checked against Pool . The accompanying plan is explanatory text, not an executable task graph.

Resource Resolution and Context Injection. After selection, local references are resolved against the skill’s directory. Selected SkillHub resources are downloaded only when the installation policy permits, using the previously fetched metadata. A skipped or failed installation retains the available body with unresolved resource references. Memory-derived skills have no bundled files in this path. The runtime renders selected bodies, qualified identifiers, and any resolved resource locations into the agent’s context. Selection, resource availability, and actual execution are therefore different events. Recording an injected identifier establishes selection, but successful execution requires separate evidence.

6.3 Experience-Driven Skill Self-Evolution

Update Pipeline. To turn execution experience into reusable procedures, EverOS’s agent pipeline performs the updates below when Raven submits interaction segments for memory formation (Section 5.1). The procedure uses the backend’s default thresholds. During retrieval, Eq. (71) groups local and EverOS skills with identical names into one candidate.

Algorithm 5: Task-Aware Skill Selection

Input: request x_{in} , session history, capabilities $(\mathcal{U}_{tool}, \mathcal{A}_{spec})$, configured sources and clients, rewriter/gate switches, budgets $(k_{pool}, k_{sel}, k_{fb})$, runtime policy.

Output: skill context and selected identifiers with provenance.

- 1 **if** no router is configured, **return** an empty segment.
- 2 Initialize $(need, q) \leftarrow (1, x_{in})$. If enabled and the request is nonblank, run the rewriter. Retain this initialization on failure.
- 3 **if** $need = 0$, **return** an empty segment. If the rewritten query is empty, use x_{in} .
- 4 Query sources concurrently for $2k_{pool}$ hits each. Assign $L_j^{hit} \leftarrow []$ for each failed source.
- 5 Compute name scores and representatives by Eqs. (71) and (72), obtaining $Pool_0$.
- 6 Remove blocked entries, retrieve missing SkillHub bodies, and apply detail policy, dropping failed or refused entries. Call the result $Pool$.
- 7 **if** $Pool = []$, **return** an empty segment.
- 8 If enabled, run the gate on $(q, Pool, \mathcal{U}_{tool}, \mathcal{A}_{spec})$ and distinguish a parsed identifier list from call/parse failure.
- 9 Compute S_{sel} by Eq. (73). No further truncation to k_{sel} is applied to the failure fallback.
- 10 For selected entries, attempt resource resolution under policy. Preserve available bodies when SkillHub installation is skipped or fails.
- 11 Render the selected entries and available paths, then return the context with qualified identifiers and source provenance.

Execution Cases and Ownership. Inside the EverOS pipeline, an extractor produces zero or one substantive case from an agent-mode memory cell:

$$c = (id_c, t_c, intent_c, approach_c, insight_c, qual_c). \quad (74)$$

Here t_c is the case timestamp, $intent$ describes the task class, $approach$ describes the execution strategy, $insight$ states the reusable lesson, and $qual_c \in [0, 1]$ is estimated case quality. It is distinct from catalog quality $qual_{cur}(s)$ and skill confidence. The same case is persisted separately for each participating assistant owner. Let $own(a)$ denote the owner partition of agent a within its application and project scope. This replication makes a shared execution available to its participants. It does not constitute independent successes or causal credit for each participant.

Incremental Task-Intent Clustering. A case below the quality threshold θ_{skip} , or a deployment without embedding capability, produces no skill update. Otherwise, embed its intent as $\mathbf{z}_c \in \mathbb{R}^{d_{emb}}$. An existing cluster C has centroid $\mu_C \in \mathbb{R}^{d_{emb}}$, positive count n_C , members, timestamp, and previews. The clustering operator recalls a bounded list of centroids by

$$\text{sim}(\mathbf{z}_c, \mu_C) = \frac{\mathbf{z}_c^\top \mu_C}{(\|\mathbf{z}_c\|_2 + \epsilon_{num})(\|\mu_C\|_2 + \epsilon_{num})}, \quad \epsilon_{num} = 10^{-9}. \quad (75)$$

Empty centroids are skipped. With no eligible cluster, a singleton is created. If the highest similarity meets the direct-merge threshold, the case joins the corresponding cluster. Otherwise the LLM receives the recalled previews and returns a cluster index or a new-cluster decision. A valid existing index is accepted. An out-of-range index creates a singleton, while a parse error propagates for retry. No temporal window is applied. Merging a singleton into C gives

$$\mu_C^+ = \frac{n_C \mu_C + \mathbf{z}_c}{n_C + 1}, \quad n_C^+ = n_C + 1. \quad (76)$$

The update appends the member, retains the latest timestamp, and limits previews to five.

Within $own(a)$, clustering serializes its read–assign–write sequence. The resulting cluster identifier, target case content, and embedding are carried by an update event. The skill extractor can therefore reconstruct its target without waiting for the case’s search-index row. Skill writes use a separate lock for the same owner partition, since updates to different clusters can still target the same skill name.

Selecting and Formatting the Update Context. With the cluster assigned, let $Catalog_C$ be its existing Markdown skills. Select an ordered list $Skills_C = (s_0, \dots, s_{m-1})$, with $m \leq k_{skill}$, where k_{skill} is the existing-skill cap. If $|Catalog_C| \leq k_{skill}$, use the Markdown listing directly. Otherwise

the index ranks relevant skills by the case vector. Accept only hits with matching Markdown records and backfill missing slots in Markdown order. The files determine existence and content, while the index supplies an ordering.

For a skill s , let $\Lambda(s)$ be its ordered supporting-case identifier list. The candidate support identifiers and collected case pool are

$$\text{SupIDs}_C = \left(\bigcup_{s \in \text{Skills}_C} \Lambda(s) \right) \setminus \{\text{id}_c\}, \quad (77)$$

$$\text{Cases}_C = \text{Take}_{k_{\text{case}}}(\text{Sort}_{(\text{qual}_c, t_{c'})} \downarrow \text{Fetch}_{\text{own}(a)}(\text{SupIDs}_C)). \quad (78)$$

Here k_{case} is the supporting-case pool cap. The union treats lineage lists as sets of identifiers. $\text{Fetch}_{\text{own}(a)}$ returns available indexed cases in the same owner partition, once per identifier. Records absent from the index are excluded from the current support pool. Sorting prioritizes quality, then recency. The target is always excluded from its own support pool.

Support formatting applies a separate limit to each skill. For each $s \in \text{Skills}_C$, the formatter takes the last matching identifiers in $\Lambda(s)$ whose records occur in Cases_C , up to the per-skill support cap and preserving lineage order. It supplies their summaries alongside that skill’s name, confidence, description, and body. Description, body, and supporting-approach prefixes have separate token budgets, with omission markers appended on truncation. The new target is supplied separately. These token budgets differ from the online gate’s character limits.

Generating Explicit Update Operations. The target quality determines the prompt mode:

$$\text{mode}(c) = \begin{cases} \text{skip}, & \text{qual}_c < \theta_{\text{skip}}, \\ \text{failure}, & \theta_{\text{skip}} \leq \text{qual}_c < 0.5, \\ \text{success}, & 0.5 \leq \text{qual}_c \leq 1. \end{cases} \quad (79)$$

For a non-skipped case, one structured LLM call uses this context to produce an ordered operation list

$$\text{Ops} = \text{SkillExtract}_{\text{mode}(c)}(\text{Format}(c, \text{Skills}_C, \text{Cases}_C)) = (\text{op}_1, \dots, \text{op}_{|\text{Ops}|}). \quad (80)$$

A well-formed operation $\text{op}_l = (\text{act}_l, i_l, \Delta_l^{\text{fields}})$ has action $\text{act}_l \in \{\text{add}, \text{update}, \text{none}\}$, optional target index i_l , and proposed field values Δ_l^{fields} . Update indices are zero-based positions in the fixed input list Skills_C , not database identifiers or indices into newly added records. Malformed list items or unknown actions are skipped. An empty operation list means no change. Malformed top-level responses are errors handled by the strategy retry path.

Validation, Field Changes, and Lineage. An addition requires a body with at least five nonempty lines and fifty characters after trimming, and at least one of name or description. For an update, the target must satisfy $0 \leq i_l < m$ and must not have been addressed by an earlier update in the same batch. The index is reserved before validating replacement content. A supplied nonempty replacement body must pass the same length checks, while omitted or empty text fields preserve their previous values. An update is accepted only if it changes a text field or supplies a parseable confidence value. Supplying the existing confidence value can therefore still append new evidence.

Let $\chi(s) \in [0, 1]$ denote confidence. A valid proposed value is clipped to $[0, 1]$. A malformed value retains prior confidence on update and defaults to 0.5 on addition. An accepted normal update produces a revised record with lineage

$$\Lambda(s^+) = \text{Unique}(\Lambda(s) ++ [\text{id}_c]), \quad (81)$$

where $++$ concatenates lists and Unique preserves first occurrence. An addition starts with $[\text{id}_c]$. These checks produce an emitted record list $\text{Emitted} = \text{ApplyOps}(\text{Ops}, \text{Skills}_C, c)$. Only emitted records are written, leaving the owner’s other skills unchanged.

A confidence below 0.1 does not remove a skill. EverOS writes that record as an ordinary skill, and later retrieval can return it. The pipeline does not invoke `retire_confidence`. Optional maturity scoring is skipped by default: new skills receive its default value 1.0, and retained maturity metadata is not an independent assessment of task utility.

Persistence and Subsequent Reuse. Emitted records are written as owner-scoped Markdown skills in the EverOS store, with cluster membership and case lineage. These files are separate from the workspace catalog. Updates preserve identity within the extraction batch to reconcile renames: write the new sanitized name, then remove an obsolete directory unless another emitted record claimed that old name. Non-emitted skills remain unchanged. The owner lock serializes competing strategy executions. It does not make multiple file writes an atomic transaction. Cascade indexing later exposes the revisions to the memory source in Algorithm 5.

Algorithm 6: Incremental Case-to-Skill Evolution

Input: new case c , owner $\text{own}(a)$, owned cluster state, Markdown skill store, case index, configured model services.

Output: emitted skill records persisted with case lineage, possibly an empty update. Failures follow the strategy retry policy.

- 1 **if** embedding is unavailable or $\text{qual}_c < \theta_{\text{skip}}$, **return** no skill update.
- 2 Under the clustering lock for $\text{own}(a)$, embed the intent, recall centroids, and assign or create a cluster as described above.
- 3 Persist the cluster, emit its identifier together with c and $\mathbf{z}_c$, and release the clustering lock.
- 4 Under the skill-update lock for $\text{own}(a)$, verify that the cluster exists. A missing cluster is a retryable error.
- 5 Enumerate cluster skills from Markdown and select Skills_C with the k_{skill} budget and index-ranking/backfill rule.
- 6 Collect Cases_C by Eq. (78) and format the bounded per-skill support and text fields.
- 7 Request Ops using Eqs. (79) and (80). Malformed top-level output raises an error before skill writes.
- 8 Initialize reserved targets $\text{UsedTargets} \leftarrow \emptyset$ and emitted list $\text{Emitted} \leftarrow []$.
- 9 **for each** $\text{op}_i \in \text{Ops}$, in order:
- 10 **if** the action is update: parse i_i . Skip if invalid, outside $[0, m)$, or already in UsedTargets . Otherwise, set $\text{UsedTargets} \leftarrow \text{UsedTargets} \cup \{i_i\}$.
- 11 Validate an add/update operation’s body and fields. Skip rejected operations, none, and unknown actions.
- 12 Construct the accepted record, apply Eq. (81), and append it to Emitted . A confidence below 0.1 is still emitted and remains retrievable.
- 13 Write emitted records in order under sanitized owner-scoped names. Retain the written-name map for rename reconciliation.
- 14 Reconcile obsolete names after writes and release the update lock on completion or error. Index changes asynchronously.

Across these stages, case quality, catalog quality, source rank, skill confidence, and measured task outcome are different quantities. Neither a structurally accepted update nor a high-confidence skill establishes an improvement. Section 7.7 reports the effect of the fixed catalog on task performance with the model and harness held fixed.

7 Evaluation

We evaluate Raven’s effectiveness in organizing specialist work, improving execution with a fixed model, and reusing procedural knowledge. Planning experiments test specialist selection and dependency prediction, while domain evaluations measure task outcomes and resource use. Harness-evolution and skill-reuse experiments then examine how adaptation and reusable procedures improve agent performance. Together, these evaluations connect Raven’s architectural choices to measurable gains in planning, execution, and reuse.

Comparison Protocol. The planning comparison fixes the backbone across systems to assess their orchestration capabilities. Harness-evolution experiments keep the task model frozen and separate adaptation tasks from held-out testing. Skill experiments hold the backbone and harness fixed within each comparison, isolating the effect of adding retrieved procedures. Specialist results report task quality alongside runtime, token use, and monetary cost where available. Benchmark-specific settings describe the task split, tool access, attempt count, scoring procedure, and resource budget. Adaptation data and development compute are distinguished from final evaluation.

Evaluation Structure. We first evaluate multi-agent orchestration and harness self-evolution, then assess the four specialists on research, software engineering, visual design, and sustained operation. The final experiments measure the benefit of retrieving and executing skills from a fixed catalog. Appendix C gives scoring conventions and protocol details for these comparisons.

7.1 Multi-Agent Orchestration

Multi-agent planning determines which specialists receive a request, what information they receive, and which subtasks may execute concurrently. End-to-end task accuracy reflects both planning decisions and downstream execution quality, making planning errors difficult to isolate. We therefore evaluate the host’s proposed graph before dispatching workers. This protocol measures specialist selection and dependency prediction independently of specialist execution.

7.1.1 The Multi-Agent Orchestration Benchmark

We introduce the *Multi-Agent Orchestration Benchmark* (MAOB), a suite of 140 requests modeled on occupational tasks, each paired with a reference directed acyclic graph (DAG) over a fixed roster of four specialist sub-agents. The benchmark measures agreement between the host’s proposed graph and a reviewed reference graph for each request.

Specialist Roster. MAOB defines four specialist domains:

- **research:** searching the web and producing a source-supported analysis.
- **coding:** writing, executing, and debugging code on the host.
- **content:** converting source material into a deliverable, such as a presentation, report, or briefing.
- **oncall:** executing and monitoring a long-running or remote job through completion.

The fixed roster permits exhaustive coverage of domain combinations. Four domains yield exactly 11 subsets containing at least two domains, and MAOB includes all 11 (Table 3). This coverage is exhaustive for domain combinations within the declared roster, but does not extend to all occupational scenarios or tool configurations.

Reference-Guided Construction. MAOB specifies the reference graph before generating the request. This provides an explicit construction target and reduces reliance on a model’s post hoc interpretation of a request. However, graph-first generation still requires review: the resulting request must justify the selected specialists and dependencies. The construction procedure has six stages:

1. **Pattern extraction.** We survey public occupational task collections, GDPval and JobBench [50, 36], and extract the task structure as the triple (occupation, deliverable, reference-material type). No task text is reused, reducing direct overlap with the source collections while retaining their occupational framing.
2. **Template library.** Each extracted pattern becomes a template specifying an occupation, scenario, deliverable, and compatible graphs. A template may support several graphs, allowing the same occupational scenario to represent different dependency structures.
3. **Reference graph first.** For each template instance we fix the reference DAG, including its node set and partial order, before generating the request. The authored graph provides a construction target whose consistency with the resulting request is checked during review. Reference graphs are authored with Claude Opus 5, chosen from a different model family than the systems and backbones under test to reduce shared-family labeling effects.
4. **Backward generation.** The request text is then generated from the graph with GLM-5.2, which also produces task metadata and corpus support files. The generator is instructed to make the dependencies inferable from the request without stating the graph explicitly.
5. **Leakage filtering.** Every generated request passes an automatic filter before it enters the pool. A request that enumerates steps (*first...then...finally*), or that names a domain outright (*research, write code, make a deck*), is rejected and regenerated. The filter shares its vocabulary with the generation prompt, providing consistent exclusion criteria. Lexical filtering alone cannot rule out paraphrased planning cues, which remain a review concern.

Table 3: Coverage of domain combinations. Every subset containing at least two domains from the four-domain roster is represented.

Domains per task	Subsets covered	Tasks
2	6 / 6	66
3	4 / 4	47
4	1 / 1	27
Total	11 / 11	140

6. **Review and admission.** Surviving samples are reviewed against their reference graph. The review records node-set agreement, ordering ambiguity, and the attribution of request elements to graph nodes.

Step 5 limits explicit planning cues that would allow a system to reproduce a listed sequence without inferring dependencies. A reference chain such as `research` $\rightarrow$ `coding` $\rightarrow$ `content` is admissible only when the request motivates each specialist and the information passed between them. Omitting an explicit sequence from the request is therefore necessary for this construction protocol but does not by itself establish the validity or uniqueness of the reference graph.

Quality Control. Every generated sample must pass automatic admission checks before evaluation. A failed check stops the construction pipeline. In addition to the leakage filter in step 5, the checks reject three classes of defect:

- **Inconsistent specification.** Contradictory quantities or requirements make a request unsatisfiable as written and can confound the assessment of planning ability.
- **Role-boundary violation.** A reference graph that assigns work outside a specialist’s remit, such as asking the content agent to retrieve or to compute, contradicts the roster the system under test is given.
- **Field inconsistency.** Task metadata must agree with the reference graph. A mismatch can exclude a sample from its intended metric group.

A further pass removes reference nodes that the request does not justify. This correction is applied during dataset construction. Masking a reference node only at scoring time could otherwise penalize a prediction made under the original task specification. After removal, edges are reconstructed from the transitive closure to preserve dependencies among the remaining nodes.

Samples that pass these checks undergo expert review of the reference node set, partial order, and attribution of request elements to nodes. Structured review records preserve the basis for each admission decision.

Dataset Statistics. The released benchmark comprises 140 tasks drawn from 137 distinct occupations across 140 templates, with one template per task. Reference graphs contain 2.72 nodes and 1.84 edges on average, with 257 reference edges in total. Every request is self-contained text without attachments, so the evaluation does not require file-handling capability.

Structurally, 98 tasks are purely serial and 42 admit parallel execution. The latter include independent successors of a shared predecessor and independent entry nodes that contribute to a common deliverable. Including both structures requires the planner to distinguish sequential dependencies from opportunities for concurrency.

Domain frequency is close to even across the three deliverable-producing domains: `content` appears in 106 tasks, `research` in 103, and `coding` in 102. The `oncall` domain appears in 70 tasks that require sustained execution or monitoring. By subset size, 66 tasks span two domains, 47 span three, and 27 span all four (Figure 12).

7.1.2 Metrics

Let $\hat{G}^{\text{raw}}$ and $G^{\star, \text{raw}}$ denote the submitted and reference graphs. A domain map assigns each invocation to its specialist label. The projection Fold identifies nodes with the same label, removes

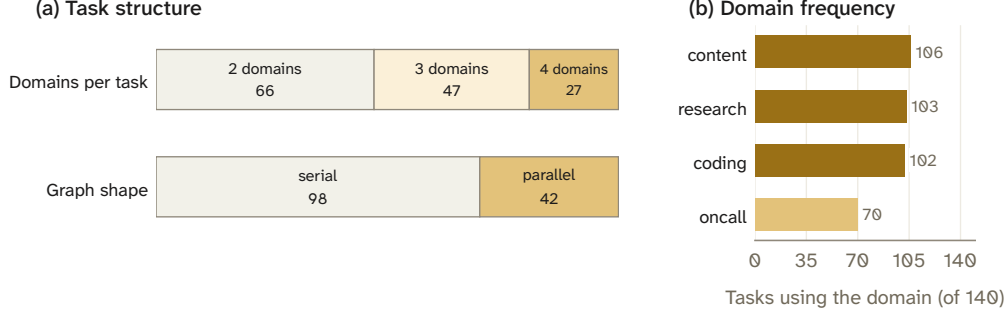


Figure 12: MAOB composition. (a) Number of domains per task and the share of tasks whose reference graph contains parallel branches. (b) Number of tasks that use each specialist domain.

edges within a domain, and deduplicates the remaining directed edges. Write

$$\hat{G} = (\hat{V}, \hat{E}) = \text{Fold}(\hat{G}^{\text{raw}}), \quad G^* = (V^*, E^*) = \text{Fold}(G^{*,\text{raw}}).$$

All metrics use these domain-level graphs to assess specialist selection and inter-domain dependencies. Invocation names and subdivisions within a domain do not affect the scores. The formulas first describe order-comparable, acyclic predictions. Projection conflicts, cycles, admissible reference variants, and aggregation are specified in Appendix C.1.

For finite sets X_1, X_2 , define the total set-agreement function

$$\text{F1}(X_1, X_2) = \begin{cases} 1, & X_1 = X_2 = \emptyset, \\ 2|X_1 \cap X_2| / (|X_1| + |X_2|), & \text{otherwise.} \end{cases}$$

Node F1. Specialist-selection agreement is

$$\text{NodeF1} = \text{F1}(\hat{V}, V^*).$$

This penalizes omitted reference domains and additional predicted domains. Every MAOB reference has at least two domains, so its ordinary denominator is positive even for an empty prediction.

Edge F1. Let $\text{TR}(G)$ denote a DAG’s transitive reduction, which removes edges implied by longer directed paths while preserving reachability. Let $\hat{E}_{\text{red}}$ be the edge set of $\text{TR}(\hat{G})$, and let $\mathfrak{E}_{\text{red}}^*$ contain the reduced edge sets allowed by the selected reference’s order annotations. Without alternative annotations this family contains only the edge set of $\text{TR}(G^*)$. The score is

$$\text{EdgeF1} = \max_{E' \in \mathfrak{E}_{\text{red}}^*} \text{F1}(\hat{E}_{\text{red}}, E').$$

Thus a redundant transitive edge is not penalized. If both compared edge sets are empty, their F1 is one. The metric compares the normalized dependency structure.

Partial Order Accuracy (POA). Let $V_{\cap} = \hat{V} \cap V^*$ and define

$$\mathcal{P}_{\text{pair}} = \{(u, v) : u, v \in V_{\cap}, u \prec_{\text{dom}} v\},$$

where $\prec_{\text{dom}}$ is the fixed roster order research, coding, content, oncall, so each unordered pair is counted once. The relation $\text{ord}_G(u, v)$ is *before*, *after*, or *unordered*, determined by reachability in G . Let $\mathcal{O}_{uv}^*$ be the reference’s accepted relations for that oriented pair. It is the singleton $\{\text{ord}_{G^*}(u, v)\}$ unless an annotation explicitly allows alternatives. Then

$$\text{POA} = \begin{cases} \frac{1}{|\mathcal{P}_{\text{pair}}|} \sum_{(u,v) \in \mathcal{P}_{\text{pair}}} \mathbb{I}[\text{ord}_{\hat{G}}(u, v) \in \mathcal{O}_{uv}^*], & |V_{\cap}| \geq 2, \\ 1, & |V_{\cap}| = 1, \\ 0, & |V_{\cap}| = 0. \end{cases}$$

A single shared node provides no ordering pair to compare and receives a score of one. A prediction with no shared node receives zero. POA must therefore be interpreted together with Node F1. Different node or edge sets can still preserve the relative order of their shared nodes.

Exact Match Rate. For an acyclic, order-comparable prediction, the per-record Exact Match indicator is one precisely when $\hat{V} = V^*$ and every shared pair’s predicted relation belongs to $\mathcal{O}_{uv}^*$. The reported rate averages this indicator over records with a defined value. Without alternative annotations this means equal node sets and equal reachability, or equivalently equal transitive reductions. It does not require identical raw edge lists. An alternative valid plan can differ from the reference.

The four metrics are reported separately. They distinguish specialist selection, reduced dependencies, accepted precedence, and whole-plan agreement at the domain level. They are not combined into a single score.

7.1.3 Experimental Setup

Systems Compared. We evaluate three orchestrating agent systems on identical inputs: Raven and two baseline harnesses, Claude Code and Hermes Agent. Each system plans through its native agent loop.

Backbones. To control for backbone choice, each system is evaluated with two backbones: Qwen3.8-27B and DeepSeek-V4-Flash-0731. A result is reported only when all three systems have been run on the same backbone, over the same 140 tasks, in the same protocol.

Protocol. Planning is measured in isolation. Each system is given the request text and submits its graph through a tool call to its native orchestration interface. The submitted graph is captured and scored without dispatching workers. The measured behavior and associated evaluation cost therefore cover planning alone.

Elicitation Parity. All three systems receive identical request text and the same delegation instruction, taken verbatim from a shared constant. Domain descriptions are provided only through each system’s orchestration-tool schema, without an additional system-level preamble. This controls the task and domain information provided to the models while retaining each system’s native interface.

7.1.4 Results

Figure 13 reports the four metrics for all three systems under both backbones.

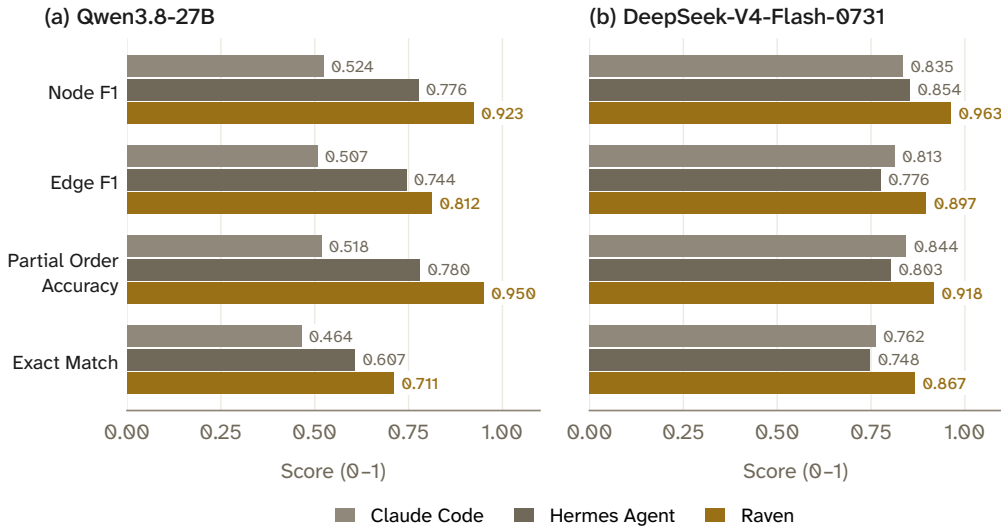


Figure 13: Multi-Agent Orchestration Benchmark results with backbone (a) Qwen3.8-27B and (b) DeepSeek-V4-Flash-0731.

Consistent Gains Across Metrics and Backbones. Raven outperforms both baseline systems on all four metrics with each backbone, demonstrating effective specialist selection and dependency

planning across the two model families. With Qwen3.8-27B, Raven reaches 0.923 Node F1 against 0.776 for the strongest baseline, a margin of 14.7 points on a percentage scale, and obtains 0.950 POA. With DeepSeek-V4-Flash-0731, Raven reaches 0.963 Node F1 and 0.897 Edge F1, compared with 0.854 and 0.813 for the strongest baseline on each metric. The gains cover both the choice of specialists and the relationships that organize their work.

Exact Graph Agreement. The improvements also extend to complete plans, for which both the specialist set and all precedence relations must match an accepted reference. On Qwen3.8-27B, Raven achieves an Exact Match rate of 0.711, compared with 0.607 for the strongest baseline, a 10.4-percentage-point improvement. On DeepSeek-V4-Flash-0731, the corresponding rates are 0.867 and 0.762, a 10.5-percentage-point improvement. These gains show that Raven more often produces a complete dependency plan consistent with the reference, complementing its advantages on individual nodes and edges.

Comparison Under Matched Backbones. Raven retains its leading position even as the relative ranking of the baselines changes. Hermes Agent leads Claude Code on all four metrics with Qwen3.8-27B but trails it on POA with DeepSeek-V4-Flash-0731. All systems receive the same request text, delegation instruction, and domain descriptions (§7.1.3). Raven’s consistent advantage with matched backbones supports the effectiveness of its orchestration design across both tested models.

Improved Dependency Prediction. Raven improves Edge F1 over the strongest baseline by +0.068 with Qwen3.8-27B and +0.084 with DeepSeek-V4-Flash-0731. Even with these gains, Edge F1 remains lower than Node F1 for every system under both backbones, indicating that dependency prediction is a more difficult part of this benchmark. Raven’s gains on this metric show that its advantage extends to identifying which specialists must exchange information and which subtasks depend on earlier results.

7.2 Harness Self-Evolution

We next examine harness self-evolution, which improves an agent by adapting the execution policy around a frozen backbone. The published HarnessBank experiments [39] evaluate the diagnosis, search, and screening procedure on tasks withheld from evolution.

Setup. The seven benchmarks are Terminal-Bench-2 [41], five domains of EvoAgentBench [18] (LiveCodeBench, Omni-MATH, BrowseComp+, GDPval, and SWE-bench Verified), and AppWorld [60]. Figure 14 presents held-out Pass@1 results with a frozen Qwen3.6-27B backbone, averaged over three attempts per task. The evolved harness is selected using the training split and then assessed on the test split.

Cross-Domain Gains with a Frozen Backbone. The evolved harness improves held-out Pass@1 on all seven benchmarks. The largest gains are 15.4 percentage points on AppWorld, 13.9 on BrowseComp+, and 13.7 on LiveCodeBench. Improvements also extend to terminal operation, mathematical reasoning, professional tasks, and repository-level issue resolution. Because the task model remains frozen and the test tasks are withheld from evolution, these results demonstrate that harness adaptation can improve execution across several domains and transfer beyond the tasks used to guide the search. Six comparisons pass the source’s paired-gain criterion. SWE-bench Verified also improves, although its 26-task test split does not pass that criterion.

Mechanism Analysis. To examine the mechanisms behind these gains, we consider the reported interventions, which address diagnosed execution failures, including reasoning-budget overruns and premature finalization. The source links these edits to observable events in the execution traces, providing a behavioral explanation for how changes around a fixed model improve task completion. Screening combines evidence that an intervention executed with a paired comparison of task outcomes. This connects candidate selection to both operational behavior and measured utility, while held-out evaluation tests the benefit of the selected harness on new tasks.

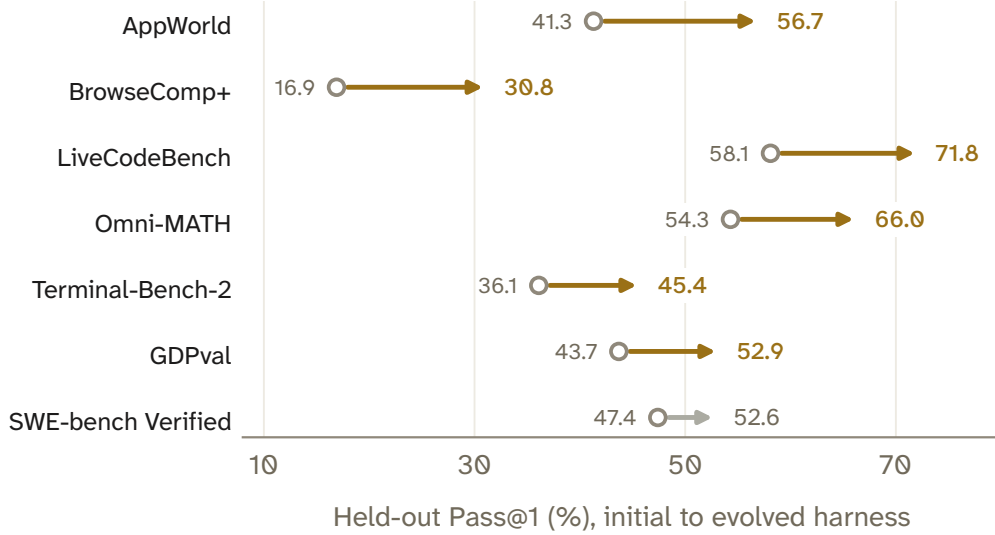


Figure 14: Held-out Pass@1 reported in HarnessBank [39] for the initial and evolved harnesses with a frozen Qwen3.6-27B backbone. Gray denotes SWE-bench Verified. Table 11 lists the values.

7.3 Raven-Research: Evidence-Grounded Deep Research

Raven-Research answers questions using evidence from the live web. Its harness addresses recurring failure modes in deep research: repeated searches without reading retrieved pages, late restarts of an investigation, turns that end without an answer, and unsupported claims. To address these failures, its single plugin, `research-flow`, replaces the built-in search, fetch, and clarification tools. The search tool caches repeated queries and stops once consecutive queries surface nothing new. The fetch tool extracts the passages that answer an explicit information request and falls back to alternative addresses for pages with little content. Checks at fixed phases of the agent loop can block or redirect execution. They require page reads after extended search sequences, prevent late restarts, and recover turns that end without an answer. Before delivery, an independent reviewer, a separate model call without the agent’s context, checks the decisive claims of the draft against the gathered evidence. The three session modes use different stopping rules under the same resource budget. Each report gives a full URL for every number, date, and quotation, and a research trail computed from the tool-call record flags any citation whose page the agent never opened.

Setup. We evaluate this workflow on DeepResearch Mixed, which combines questions from BrowseComp [66], FRAMES [31], the text-only exact-match setting of Humanity’s Last Exam [7], and xBench-DeepSearch [8, 71]. An LLM judge grades each short final answer for semantic equivalence with the reference, and accuracy is pooled over all questions. We compare Raven-Research with MiroFlow [58] and DeepSeek-Harness [13] on three shared backbones, Qwen3.6-35B-A3B, Qwen3.5-397B-A17B, and DeepSeek-V4-Flash, with the same search and page-retrieval tools, and we include three commercial deep research services as references [42, 51, 19]. The runs took place between August 10 and August 18, with each system answering each question once. Appendix C.3 gives the benchmark composition, grading, cost accounting, and budgets.

Consistent Accuracy Gains Across Backbones. Raven-Research achieves the highest pooled accuracy with all three shared backbones, exceeding the strongest baseline by 6.6, 3.3, and 7.6 percentage points, respectively (Table 4). The consistent gains across the Qwen and DeepSeek models demonstrate the effectiveness of the research harness across different backbones. With DeepSeek-V4-Flash, Raven-Research reaches 69.3% on BrowseComp and 60.0% on Humanity’s Last Exam, compared with at most 62.4% and 43.3% for the other two harnesses (Figure 15a). The larger margins on these two benchmarks account for an important part of the overall advantage. All three harnesses exceed 86% on FRAMES, while DeepSeek-Harness leads Raven-Research by 3.4 percentage points on xBench-DeepSearch. Raven-Research nevertheless obtains the strongest aggregate performance across the combined task set.

Table 4: DeepResearch Mixed results. Accuracy (%) is pooled over all questions. Input tokens (including cache reads), output tokens, and cost are means per question. A dash marks a quantity that was not measured. Bold marks the best accuracy within each shared-backbone group.

System	Accuracy	Input (M)	Output (k)	Cost (USD)
<i>Qwen3.6-35B-A3B</i>				
MiroFlow	47.7	1.493	11.3	—
DeepSeek-Harness	49.7	4.587	16.2	—
Raven-Research	56.3	1.706	30.0	—
<i>Qwen3.5-397B-A17B</i>				
MiroFlow	48.0	1.799	15.3	—
DeepSeek-Harness	56.0	0.679	6.6	—
Raven-Research	59.3	1.837	7.6	—
<i>DeepSeek-V4-Flash</i>				
MiroFlow	67.2	3.737	39.3	0.0374
DeepSeek-Harness	68.9	2.975	31.8	0.0211
Raven-Research	76.5	2.380	41.6	0.0242
<i>Commercial API Reference Systems</i>				
MiroThinker-1.7	50.3	≈ 1.247	≈ 18.5	5.62
Perplexity Sonar Deep Research	52.0	—	11.7	0.55
Gemini Deep Research (2604)	63.2	1.178	97.0	≈ 1.67

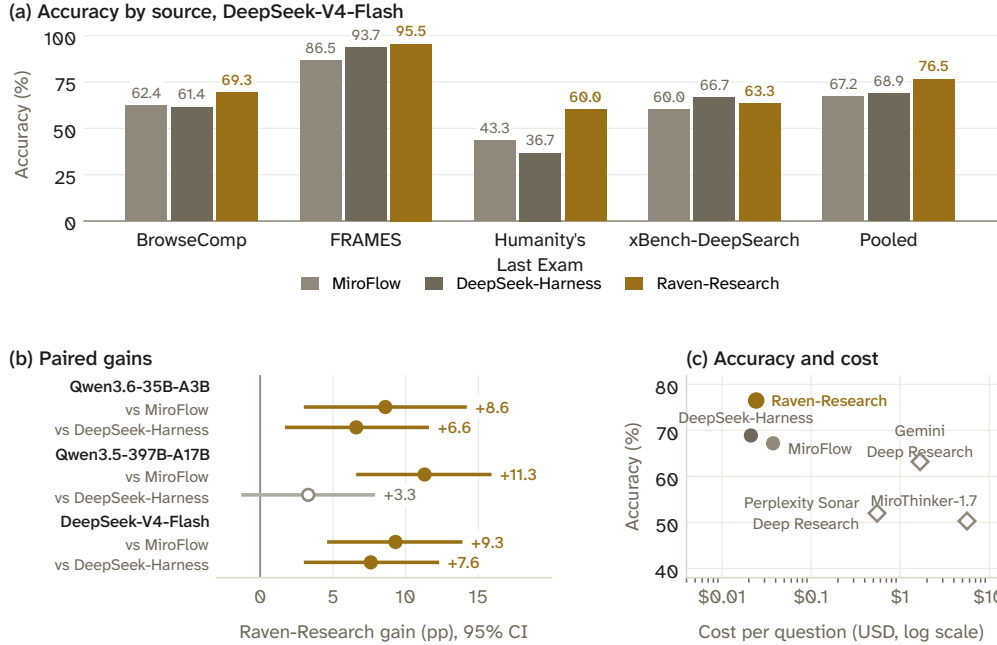


Figure 15: Raven-Research results on DeepResearch Mixed. (a) Accuracy per source under DeepSeek-V4-Flash. (b) Paired accuracy gain of Raven-Research over each comparison on the same backbone, with stratified bootstrap 95% intervals over all questions. The hollow marker denotes the comparison with DeepSeek-Harness on Qwen3.5-397B-A17B, with exact McNemar $p = 0.21$. (c) Accuracy against mean cost per question, with filled markers for the DeepSeek-V4-Flash harnesses and hollow markers for the commercial services, whose costs follow each provider's own accounting.

Paired Statistical Comparisons. Paired comparisons reinforce the overall accuracy advantage. Exact McNemar tests over all questions give $p < 0.02$ for five of the six same-backbone comparisons (Figure 15b). The remaining comparison, against DeepSeek-Harness with Qwen3.5-397B-A17B, shows a positive 3.3-percentage-point difference with $p = 0.21$. Thus, the numerical gains are consistent across all six comparisons, with statistical support in five.

Higher Accuracy at Comparable Inference Cost. With DeepSeek-V4-Flash, Raven-Research combines the highest accuracy with a mean cost of 0.0242 USD per question, between the costs of the two locally evaluated baselines. It uses fewer input tokens and more output tokens than both, achieving better answers without incurring the highest inference cost. Raven-Research also outperforms the three commercial reference services, whose accuracies range from 50.3% to 63.2% (Figure 15c). However, those services use their own models and search stacks, with provider-specific cost accounting (Appendix C.3). Accordingly, the shared-backbone comparison provides the clearest evidence that Raven’s research workflow offers an effective balance between answer quality and inference cost.

7.4 Raven-Code: Repository-Aware Software Engineering

Raven-Code writes, runs, and debugs code in the working directory assigned by the Host Agent. Its harness addresses errors arising from incomplete repository context, including edits to stale file contents, changes that violate project conventions, and completion claims unsupported by tests. To address these errors, its plugin, `code-flow`, adds the repository’s own instruction files, such as `AGENTS.md` and `CLAUDE.md`, to every model call and warns the agent when other sessions share the directory. The tool interface follows the conventions of common coding assistants and adds safeguards. An edit is refused unless the session has read the current version of the file, every Python write returns a syntax check, and the shell returns partial output when a command times out. A set of coding instructions, selected by model family, asks the agent to reproduce a failure before editing, make minimal root-cause fixes, and prefer the project’s own tests as evidence. Each task ends with a harness manifest derived from repository state, including the base commit, uncommitted files, and the diff. This provides an execution record independent of the model’s account of its work.

Setup. We evaluate issue resolution on SWE-bench Pro [14] (731 tasks) and SWE-bench Verified [46, 26] (500 tasks), repository-level development on the 80-task code subset of WorkBuddy Bench [4], and whole-repository stack migration on SWE-Refactor [21] (20 tasks). For systems evaluated locally, we use AgentEval, an internal framework that runs each attempt in a disposable container and applies the benchmark’s official grading code. Patches are generated without network access on SWE-bench Pro and with network access on SWE-bench Verified. Hollow markers in Figure 16 are public leaderboard values [61, 59, 44]. Appendix C.4 gives the versions, budgets, and accounting.

Strong Performance on Complex Repository Tasks. Raven-Code achieves the highest score in six of the eight settings in Figure 16, with its largest margins on whole-repository migration. On SWE-Refactor, it exceeds the compared leaderboard entries by 9.5 points with DeepSeek-V4-Flash and 3.0 points with GPT-5.6 Luna. Each task requires migrating an entire repository to a new stack over several hours. The gains demonstrate Raven-Code’s effectiveness on sustained development tasks that require coordinated changes across files. This result is consistent with the harness’s emphasis on repository instructions, current file contents, and execution-based verification.

Improved Issue Resolution. Raven-Code also improves resolution of existing repository issues. On SWE-bench Pro, it resolves 15 more of the 731 tasks than Claude Code with the same Qwen3.8-27B backbone. On SWE-bench Verified, its advantages of 0.6 points with DeepSeek-V4-Flash and 1.2 points with Claude Opus 4.8 correspond to 3 and 6 additional solved tasks out of 500. These results extend the gains from repository migration to targeted bug fixing. The Verified runs permit network access during generation (Appendix C.4).

Competitive Repository Development. On WorkBuddy-Code, Raven-Code outperforms all three locally evaluated baselines with DeepSeek-V4-Flash, by margins of 1.4 to 4.0 points. It remains close to the strongest alternatives with the other backbones: the Claude Code leaderboard entry leads by 1.2 points with Claude Opus 4.8, and OpenCode leads by 0.4 points with Qwen3.6-35B-A3B,

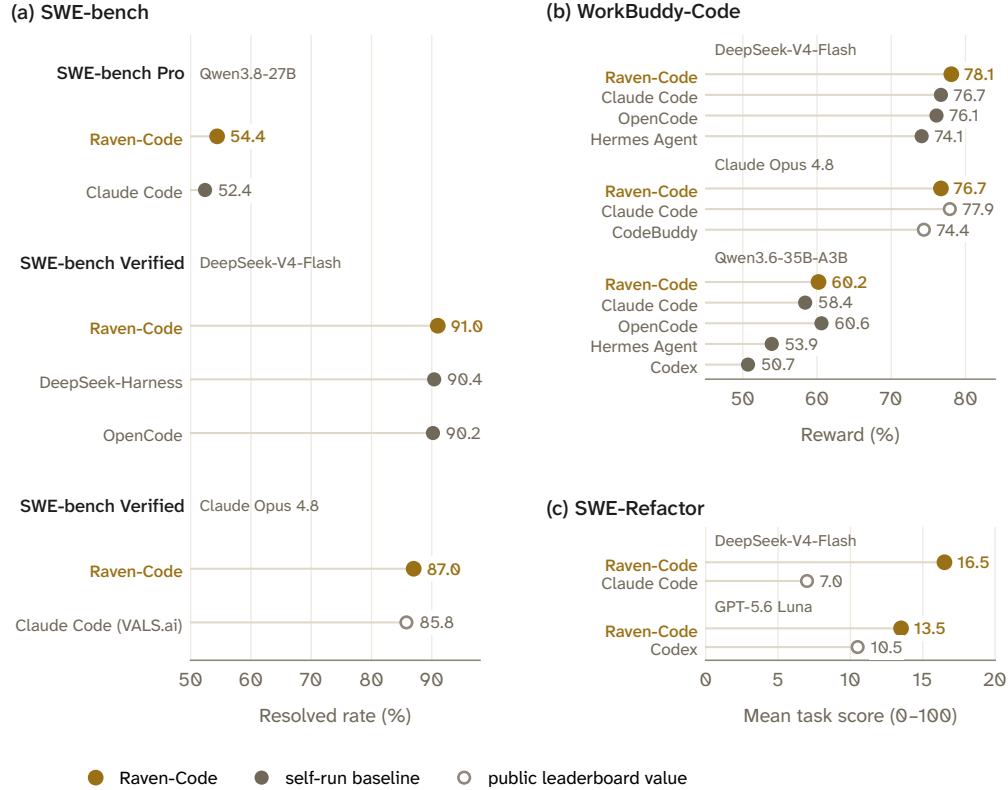


Figure 16: Coding results. Each row is one system, grouped by benchmark and backbone. SWE-bench reports the resolved rate, WorkBuddy-Code the reward, and SWE-Refactor the mean task score. Hollow markers are public leaderboard values.

where the three strongest systems are within 2.2 points. Raven-Code therefore remains competitive across the tested development settings, alongside its larger advantages on migration.

High Accuracy on Database Analysis. Beyond software modification, we assess database analysis using DataAgentBench [40]. This benchmark asks an agent to answer 54 natural-language questions over 12 datasets stored in PostgreSQL, MongoDB, SQLite, and DuckDB. Following the leaderboard protocol with five trials per query, Raven-Code with Claude Opus 5 as its main model reaches a Pass@1 of 0.8762 and a Pass@5 of 0.9097. On August 24, 2026, this was the highest Pass@1 among the entries in Figure 17, with the other entries taken from the public leaderboard on that date. This extends the evidence for Raven-Code beyond software modification to answering analytical questions over heterogeneous database systems.

7.5 Raven-Design: Render-and-Inspect Visual Design

Visual defects such as overflowing text, obscured chart labels, and poor layout may become apparent only after rendering. Raven-Design therefore incorporates rendering and inspection into the production of slide decks, charts, diagrams, and web interfaces. A single entry in the Host Agent’s registry provides access to two implementations. The design engine selects the relevant packaged domain skills for each request, maintains a persistent Task State containing goals and requirements, and renders HTML, SVG, and Office outputs into pages and previews that the model inspects before delivery. Requests based on a user-provided template are routed internally to Raven-PPT. This deck engine specifies each slide’s main claim in an outline and builds the deck using python-pptx. Each build is checked for 68 types of issue, nine of which block publication. Any slide not yet inspected by the agent is sent to a second reviewer with an empty context.

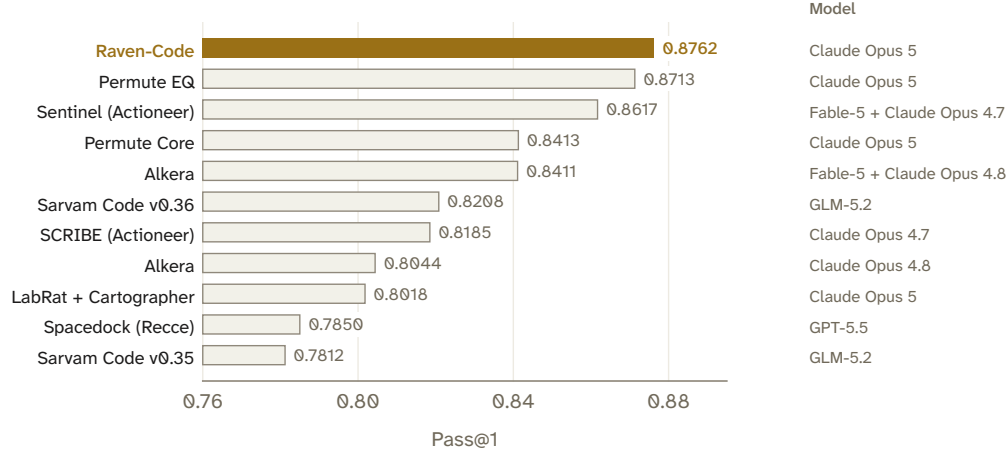


Figure 17: DataAgentBench Pass@1 on August 24, 2026, with the horizontal axis starting at 0.76. The comparison entries are the public leaderboard values on that date, and the right column gives their models.

Slide Generation Setup. PresentBench [10] contains 238 slide-generation tasks from five domains, each with an instruction and background materials. The agent produces an editable PPTX file, and the official evaluator scores it from 0 to 100 over five equally weighted dimensions: presentation fundamentals, visual design and layout, content completeness, content correctness, and content fidelity. Reported means cover successfully graded decks, and task coverage varies across configurations. Appendix C.5 gives the protocol and coverage of each configuration.

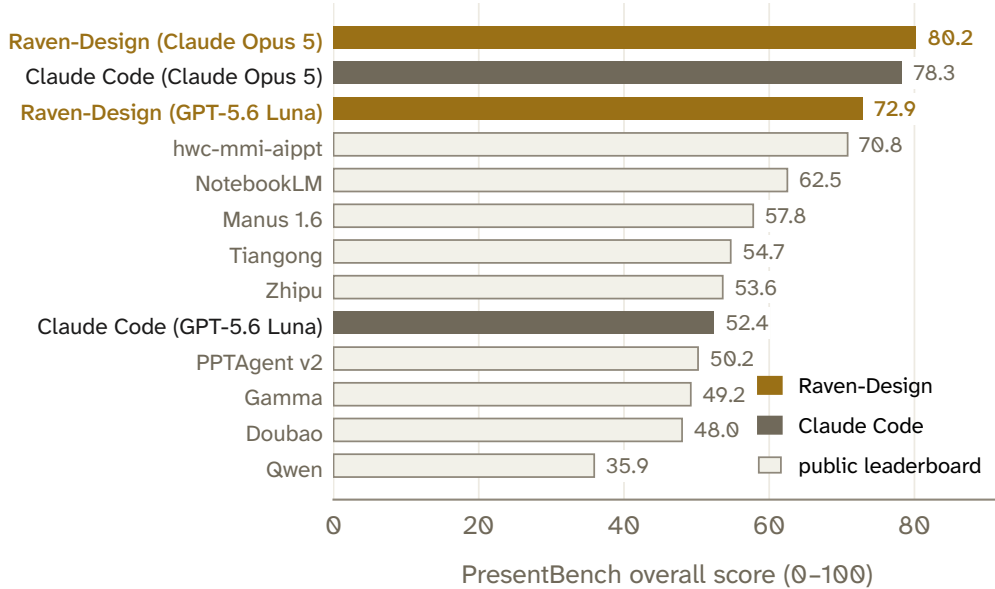


Figure 18: PresentBench overall scores (0-100). Gold bars are Raven-Design and dark bars Claude Code, with the backbone in parentheses. Light bars are public leaderboard entries evaluated by the benchmark authors, whose model configurations are not given.

Improved Slide Generation. Raven-Design produces higher-scoring decks than Claude Code with both tested backbones. It scores 80.2 with Claude Opus 5 and 72.9 with GPT-5.6 Luna, compared with 78.3 and 52.4 for Claude Code (Figure 18). The gains of 1.9 and 20.5 points show the benefit of a specialized slide-generation harness, with a particularly large improvement for GPT-5.6 Luna. Both Raven-Design scores also exceed every public leaderboard entry shown in the figure. Those

entries provide product-level reference scores, with unspecified models and evaluation on all 238 tasks. The results support the effectiveness of structuring slide generation around explicit layouts and artifact-level evaluation.

Visual Artifact Setup. Beyond slide decks, the evaluation includes SVG generation and data-visualization dashboard tasks from ArtifactsBench [76], while GDPval [50] contributes professional deliverables such as spreadsheets, reports, and slide decks. An internal grader renders each deliverable, replays interactions where the task requires them, and scores the benchmark’s checklist or rubric from 0 to 100.

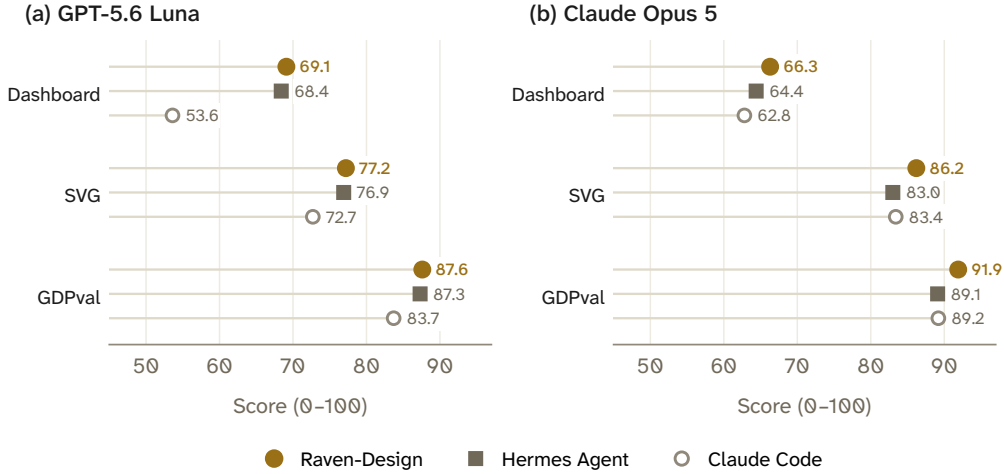


Figure 19: Visual-design scores (0–100) under (a) GPT-5.6 Luna and (b) Claude Opus 5. Scores come from an internal grader with GPT-5.6 Luna as the judge and are not comparable with the official leaderboards.

Consistent Gains Across Visual Tasks. Raven-Design achieves the highest score in all six benchmark and backbone settings (Figure 19), extending its advantages to SVG generation, interactive dashboards, and professional deliverables. It leads the strongest alternative, Hermes Agent in four settings and Claude Code in two, by 0.3 to 2.8 points. The largest gap against an individual baseline is 15.5 points over Claude Code on Dashboard tasks with GPT-5.6 Luna. Although the margins over the strongest alternative are modest, their consistency across task types and both backbones supports the broader utility of Raven-Design’s visual workflow.

Case Study Setup. To complement the benchmark scores with examples of generated deliverables, we examine seven runs from the showcase in the Raven repository (docs/showcase.md): four slide decks, two posters, and one interactive website (Figure 20). In each run, Raven planned a task graph from the user’s request, ending with a Raven-Design node that consumed outputs from other specialists (Table 5). These selected cases provide qualitative examples of completed collaborative workflows, without human preference ratings.

A Distinct Visual Language for Each Deck. Although the four decks share the same task graph, they adopt visual designs suited to their subjects. In each run, three Raven-Research nodes study separate aspects of the topic in parallel, a fourth synthesizes their reports, and Raven-Design builds the deck. The Chinese deck on Song-dynasty domestic aesthetics sets a vertical title on ink-wash photography beside a red seal-style block (Figure 20a). The deck on Greek polychromy splits its cover portrait into a white-marble half and a painted half, visually introducing the presentation’s central claim (Figure 20b). The deck on pop music uses pop-art illustrations, and the deck on abstract art is built from flat geometric tiles (Figures 20c and 20d). Within each deck, the interior slides retain a consistent palette and title position while using timelines, card rows, numbered lists, and image-led layouts. The Raven-Design node took between 19 and 65 minutes per deck, and the showcase lists a cost of about US\$0.80 for each deck. The shared workflow accommodates different subjects and visual styles while preserving consistency within each presentation.

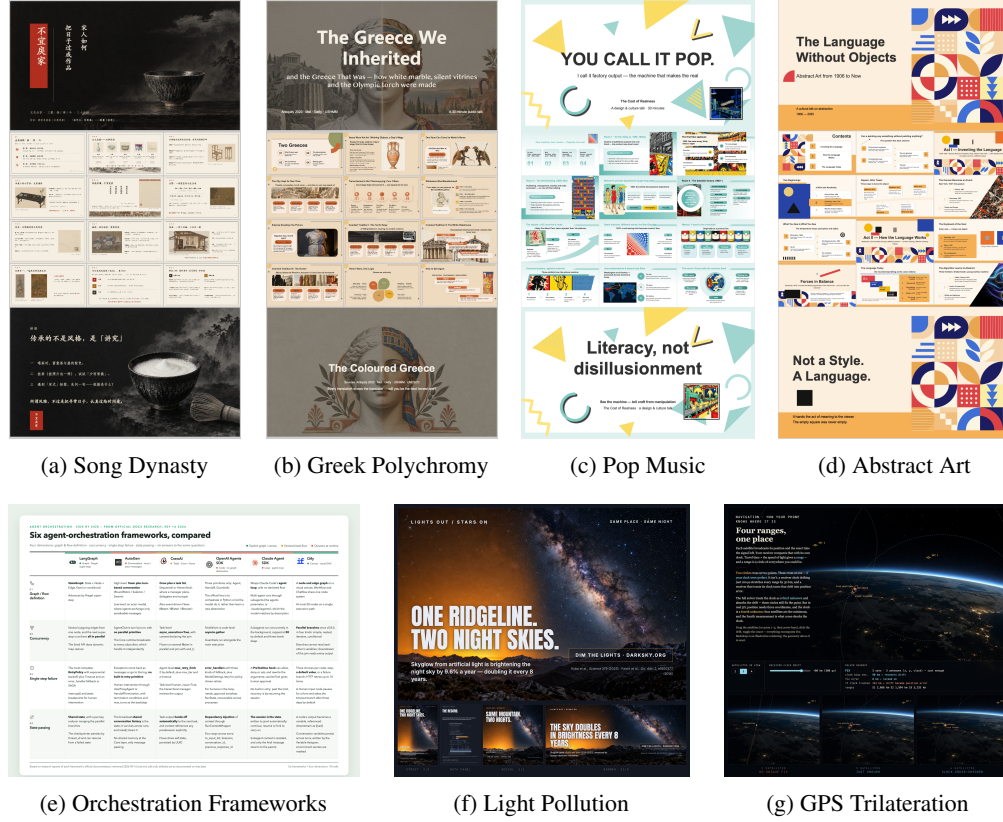


Figure 20: Deliverables of the Raven-Design case studies. Panels (a)–(d) show the cover, twelve interior slides, and the closing slide of each deck. Panel (e) shows the complete board, which compares six frameworks on four dimensions. Panel (f) shows the key visual above its four derived formats, and panel (g) shows the live page above the position fix with two, three, and four satellites. Table 5 gives the task graph of each run.

Table 5: Task graphs of the case studies in Figure 20. R, C, O, and D denote Raven-Research, Raven-Code, Raven-Oncall, and Raven-Design nodes, a numeral counts parallel nodes of one kind, $\rightarrow$ marks a dependency, and $\parallel$ marks branches that run concurrently. Design time is the wall-clock time of each Raven-Design node in the task graph, with the image-plate node first where a graph has two.

Case	Deliverable	Task Graph	Design Time (min:s)
(a) Song dynasty	Deck (Chinese)	$3R \rightarrow R \rightarrow D$	18:45
(b) Greek polychromy	Deck (English)	$3R \rightarrow R \rightarrow D$	55:34
(c) Pop music	Deck (English)	$3R \rightarrow R \rightarrow D$	47:03
(d) Abstract art	Deck (English)	$3R \rightarrow R \rightarrow D$	64:50
(e) Orchestration frameworks	Comparison board	$2R \rightarrow D$	6:57
(f) Light pollution	Poster series	$[(3R \rightarrow C) \parallel D] \rightarrow D$	2:58, 13:57
(g) GPS trilateration	Interactive website	$[(2C \rightarrow O) \parallel D] \rightarrow D$	3:03, 17:03

Composing the Work of Other Specialists. The posters and website illustrate how Raven-Design incorporates evidence and computations from other specialists. The comparison board of six orchestration frameworks is built from two Raven-Research reports on their official documentation, and it arranges 24 cells, one for each framework and dimension, under a three-color key for how each framework defines its flow (Figure 20e). For the light-pollution campaign, a Raven-Code node computes the figures for the data panel from three research reports while a separate Raven-Design node generates the key visual. A final Raven-Design node then composes the key visual and four derived formats, a 2:3 street poster, the data panel, a 1:1 social square, and a 21:9 banner (Figure 20f).

Both posters print their sources. For the website, two Raven-Code nodes write the GPS trilateration computation and an independent counterpart, Raven-Oncall cross-checks them, and Raven-Design builds a page on which the reader drags satellites, changes the receiver clock drift, and sees the position fix recomputed (Figure 20g). The page labels its Earth backdrop as illustrative and its geometry as exact. In the last two graphs the Host Agent assigns Raven-Design both a narrow subtask, the image plate, and the final composition. These cases demonstrate how Raven composes research, computation, verification, and design into complete visual deliverables. The task graph also allows a specialist to contribute at different stages, according to the artifact dependencies of the request.

7.6 Raven-Oncall: Unattended Run-and-Watch Automation

Raven-Oncall manages long-running jobs on registered machines, including training runs, solver cases, and parameter sweeps. Such jobs require monitoring for divergence, efficient waiting between state changes, and final reports grounded in recorded measurements. To meet these requirements, each job is represented as a campaign, a persistent record containing its target, measurements, permitted actions, and budget. The agent submits one round at a time and registers a wake-up with Raven’s proactivity scheduler under the campaign identifier. It then waits without issuing model calls. A monitor that operates without model calls wakes the agent early if the round finishes or diverges. The tools enforce the campaign’s operating constraints. Every decision to wait, terminate, or resubmit must cite a recent measurement, compute requests are clamped to the remaining budget, and machine capacity is allocated across campaigns. A campaign closes only through a report gate that rejects reports without a baseline and claims contradicted by the recorded state.

AI4AI Setup. The AI4AI task is autoresearch [29], a public single-GPU derivative of nanochat [28] in which an agent improves the pretraining recipe of a GPT model with about 50M parameters. Each training run receives 300 seconds on one GPU, each campaign has a training budget of 5 GPU-hours on two A800 80GB GPUs, and the metric is validation bits per byte (BPB), averaged over several seeds of the final configuration. Runtime is the wall-clock time of the whole campaign, including work outside the training runs. The unmodified starting script reaches 1.108 BPB. Each system ran one campaign. Claude Code received a separately written task statement with the same budget and constraints. Appendix C.6 gives the rules and accounting.

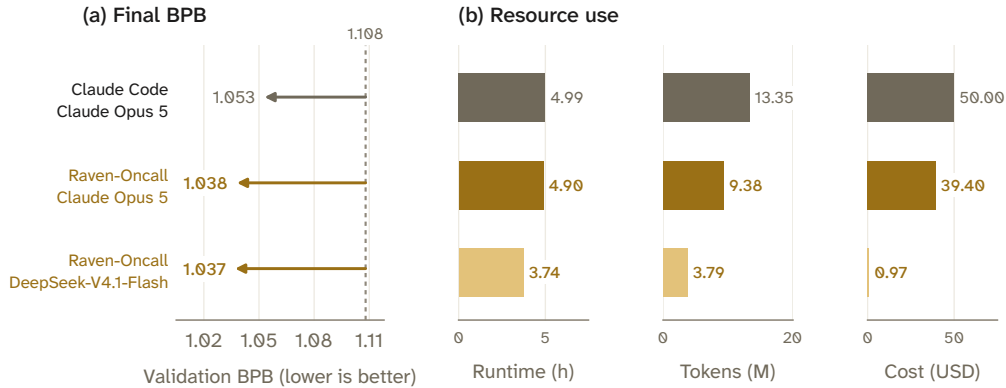


Figure 21: AI4AI nanochat pretraining. (a) Final validation BPB of each campaign, starting from 1.108 (dashed line). (b) Runtime, reported tokens, and USD cost. Lower is better throughout. The lighter bars use the DeepSeek backbone.

Lower BPB at Lower Cost Under Claude Opus 5. Raven-Oncall achieves better pretraining quality with lower reported token use and cost under the shared Claude Opus 5 backbone. It reaches 1.038 BPB, compared with 1.053 for Claude Code, in a similar runtime of 4.90 versus 4.99 hours (Figure 21). From the starting value of 1.108, these results correspond to reductions of 0.070 and 0.055. With the same training budget, Raven-Oncall therefore finds a better final configuration while spending fewer model tokens on managing the campaign. With DeepSeek-V4.1-Flash, it reaches 1.037 BPB in 3.74 hours at a reported cost of 0.97 USD, demonstrating that effective campaign execution is also possible with a low-cost backbone.

AI4S Setup. To evaluate a broader range of sustained tasks, we use the AI4S benchmark, an internal set of 17 automatic scientific research tasks. The tasks execute solvers and training jobs on registered machines, covering structural mechanics, computational fluid dynamics, molecular simulation, numerical computation, LLM batch inference, embedding-model training, retrieval tuning, and platform monitoring. Success rate is the number of successful tasks divided by 17, and runtime, tokens, and cost are averages over the tasks. Appendix C.6 describes the tasks and the success judgment.

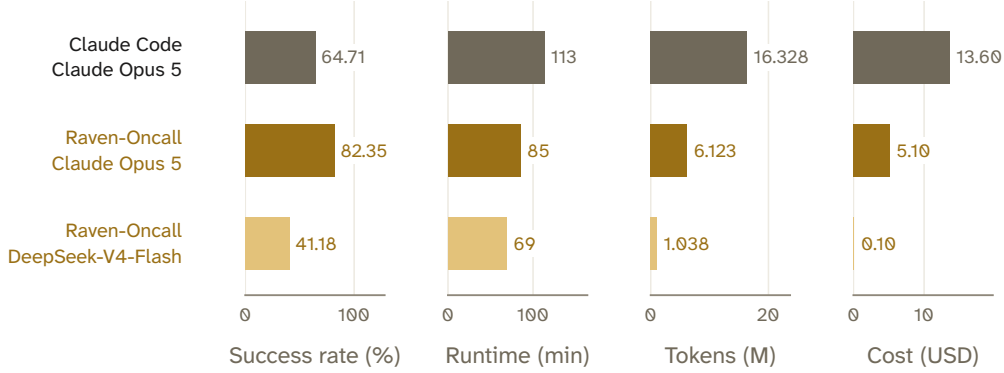


Figure 22: AI4S internal benchmark. Success rate on the 17 tasks, and runtime, tokens, and USD cost averaged per task. The lighter bars use the DeepSeek backbone.

Higher Task Success with Lower Resource Use. Raven-Oncall solves 14 of the 17 scientific tasks with Claude Opus 5, compared with 11 for Claude Code, while reducing average runtime, token use, and cost (Figure 22). Completing more tasks with lower resource use demonstrates the practical value of its campaign-based approach to sustained execution. With DeepSeek-V4-Flash, it solves 7 tasks at an average cost of 0.10 USD, offering a lower-cost operating point. Success is judged manually against task-specific reference values and supporting evidence withheld from the agent (Appendix C.6). Together, the training and scientific-task results support Raven-Oncall’s effectiveness in managing real jobs through completion under explicit resource budgets.

7.7 Skill Retrieval and Reuse

We evaluate how retrieved procedures improve task performance with the backbone and harness held fixed. The published SkillCorpus experiments [64] attach the curated catalog of Section 6.1 and its reference retrieval-and-selection stack to two agent harnesses, one of which is Raven. The skill library stays fixed throughout these experiments, allowing us to assess both the benefit of reusable procedural knowledge and the harness’s ability to translate that knowledge into successful execution.

Setup. The evaluation covers 407 tasks from three third-party benchmarks. SkillsBench [35] contributes 87 tasks across eight domains with deterministic verifiers. GDPval [50] contributes 220 professional tasks graded by an LLM judge, and QwenClawBench [53] contributes 100 tasks drawn from a real-user distribution and graded by automated checks together with an LLM judge. Raven and OpenClaw [57], two independently developed harnesses that load SKILL.md skills, are each paired with Qwen3.5-27B and Qwen3.5-397B-A17B. Each of the four harness–backbone cells attempts every task under a no-skill baseline and with SkillCorpus. In the latter condition, the reference stack recalls and reranks catalog candidates, and an LLM selector injects at most two full skill bodies into the agent’s context. Both harnesses receive identical precomputed selections, allowing their performance gains to be compared under the same procedural input. SkillsBench reports Pass@1, and GDPval and QwenClawBench report mean rewards on [0, 1] multiplied by 100. Every configuration is averaged over three runs, and gains are reported in points on these scales. Appendix C.7 gives the grading and accounting details.

Consistent Gains Across Benchmarks. Skill retrieval improves task performance in all twelve cell–benchmark comparisons in Figure 23. This consistent pattern spans both harnesses, both backbones, and all three benchmarks. SkillCorpus pools each benchmark by averaging each task’s paired gain

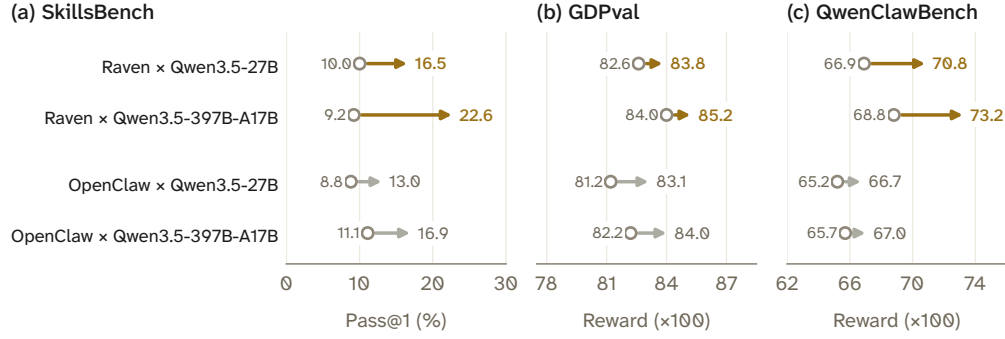


Figure 23: Effect of the curated skill library reported by SkillCorpus [64]. Each arrow runs from the no-skill baseline (hollow marker) to the condition with SkillCorpus skills for one harness–backbone cell. Values are three-run means. Horizontal axes start at different values. Table 12 lists the values.

over the four cells and computing a task-clustered standard error. The pooled gains are $+7.5 \pm 2.3$ points on SkillsBench, $+1.51 \pm 0.49$ on GDPval, and $+2.79 \pm 0.70$ on QwenClawBench, with z -scores of 3.2, 3.1, and 4.0. The gains span different baseline performance levels: SkillsBench starts near 10% Pass@1, whereas GDPval and QwenClawBench start between 65 and 85 points. A single-run check with Claude Opus 4.7 in OpenClaw raises SkillsBench Pass@1 from 39.1% to 47.1%, showing that the library also provides useful procedural knowledge to a stronger backbone.

Larger Gains with the Raven Harness. Raven obtains larger gains than OpenClaw from identical skill selections on SkillsBench and QwenClawBench with both backbones. On SkillsBench, Raven gains 6.5 and 13.4 points with Qwen3.5-27B and Qwen3.5-397B-A17B, compared with 4.2 and 5.8 points for OpenClaw. On QwenClawBench, the corresponding gains are 3.9 and 4.4 points for Raven and 1.5 and 1.3 points for OpenClaw. With Qwen3.5-397B-A17B, Raven reaches the highest SkillsBench Pass@1 in the grid, 22.6%. In one run, adding skills resolves 19 tasks that fail without skills, with regressions on 2 tasks (exact McNemar test, $p < 0.001$). These gains demonstrate Raven’s ability to turn the same retrieved procedures into more successful task executions.

Trace inspection in SkillCorpus provides a qualitative explanation for this difference. On tasks that Raven solves and OpenClaw fails, OpenClaw often stops after reasoning, leaves generated scripts unexecuted, or finishes without running the verifier. Raven completes an execute–verify–fix loop. The traces connect its stronger skill utilization to carrying procedures through execution and checking their outcomes.

On GDPval, however, all four cells improve by 1.2 to 1.9 points, with per-task reward standard deviations of 20 to 27 points. These smaller individual-cell gains are not statistically distinguishable from zero.

Contributions of Curation and Retrieval. To examine which components contribute to these gains, SkillCorpus reports single-run ablations of catalog curation and trained retrieval. The analysis uses the strongest configuration, Raven with Qwen3.5-397B-A17B on SkillsBench (Table 13). Replacing the fine-tuned retriever and reranker with off-the-shelf Qwen3 embedding and reranking models lowers Pass@1 from 22.6% to 13.8%. Replacing the curated catalog with the raw crawl of about 821,000 skill files lowers it to 14.9%. Both ablated configurations remain above the no-skill baseline of 9.2%, while the full pipeline performs best. The results support combining a curated procedural library with retrieval models that can identify applicable skills. As a standalone retriever on the 75 core SkillsBench tasks, the fine-tuned stack matches SkillRouter [81] on its Easy tier with a Hit@1 of 0.760 and exceeds it on the Hard tier, 0.760 against 0.720. This improvement on the harder retrieval tier complements the downstream gains from the trained retrieval pipeline.

Larger Gains with Better Skill Coverage. Gains also vary with skill coverage: tasks with higher skill-coverage scores show larger performance gains. SkillCorpus uses the reranker score of the highest-ranked skill as a proxy for task coverage. Pooled over the four cells, the mean SkillsBench gain rises from 2.2 points for the 26 tasks scoring below 0.45 to 6.2 points for the 40 tasks between 0.45 and 0.75 and 25.1 points for the 17 tasks at or above 0.75. Per task, the score correlates with the

gain at Pearson $r = 0.31$ to 0.40 across the four cells. The score is essentially uncorrelated with the no-skill baseline, and controlling for the baseline leaves partial correlations of 0.34 to 0.40 . However, Mathematics with 4 tasks and Finance & Economics with 9 tasks show gains near zero on Raven with Qwen3.5-397B-A17B. The overall relationship between coverage and improvement supports task-aware skill retrieval and identifies catalog coverage as a concrete direction for extending its benefits across domains.

8 Related Work

8.1 Multi-Agent Collaboration and Orchestration

Multi-agent collaboration can be organized around conversation, a fixed workflow, a dynamically revised task state, or an executable dependency graph. The choice determines what the system can inspect before execution, how it transfers intermediate results, and where coordination failures are observed. Raven requires an orchestration interface that accommodates heterogeneous model–harness pairs and their different execution capabilities.

Conversation-oriented frameworks coordinate agents through configurable message exchanges. AutoGen lets developers compose agents through messages and configurable conversation patterns [69]. MetaGPT assigns software roles and encodes a development procedure in prompts and artifacts [22]. These systems support role specialization through communication patterns and roles configured by the developer. However, such a configuration does not by itself provide a typed graph that can be validated before execution. The same issue appears in systems that use a prescribed sequence of agent calls: the sequence is inspectable, but adapting it to a new task often requires editing the workflow itself.

Other systems make the execution structure more explicit or more adaptive. MacNet represents interaction with a directed acyclic graph (DAG), although its agents are drawn from a comparatively homogeneous collaboration pool [52]. Magentic-One maintains a task ledger and assigns agents as the ledger changes during execution [17]. MasRouter learns to route requests among large language models in a multi-agent system, while Dang et al. evolve orchestration strategies over tasks [75, 12]. These approaches address executor selection or workflow adaptation, but they generally assume a stable roster and a training or evaluation distribution from which routing behavior can be learned. However, executor selection alone does not specify the artifact contract required for a subsequent handoff.

Studies of scaling show that agent count alone is insufficient to predict collaboration performance. Kim et al. report that coordination gains depend on the interaction between task structure and system architecture [30]. More agents can add coverage when their capabilities are complementary, while redundant messages and extra handoffs can consume the budget without improving the result. AgentBench exposes related limits in long-horizon agent tasks, where tool use, state tracking, and recovery remain coupled [38]. MAST provides an empirical taxonomy of multi-agent failures that includes coordination, message, and execution effects [6]. Together these results motivate evaluating the structure of a collaboration plan separately from the final quality of an artifact.

Formal methods offer useful vocabulary for making that structure testable. Hoare logic connects a program’s preconditions, commands, and postconditions [20]. Separation logic explains how local reasoning can remain sound when concurrent components own disjoint resources [45]. Assume–guarantee reasoning and its probabilistic variants express how one component’s assumptions constrain another component’s guarantees [32]. Recent discussions of contracts for language-model agents argue that deployment needs several layers of probabilistic guarantees rather than an informal message protocol [3]. These approaches motivate explicit executor identities, input scopes, output conditions, and resource limits, although verifying open-ended language-model execution remains unresolved.

Raven uses this contract perspective at the orchestration boundary. A Host Agent can delegate directly or submit a DAG whose nodes identify an executor, a prompt template, dependencies, input bindings, and optional stateful handles. The runtime checks identifiers, acyclicity, path scope, backend capabilities, registration status, recursion, and the shared dispatch budget before it starts a worker. Every completed node produces a recorded artifact that later nodes reference by content or path. The host therefore selects a model–harness pair at invocation time from a heterogeneous registry, while the graph fixes the predecessor relation and the handoff representation. This separates planning

from execution and allows the same interface to represent a one-agent delegation or a multi-stage workflow.

8.2 Agent Harnesses and Self-Evolution

An agent harness is the execution policy around a language model. It can define the initial context, reasoning and action loop, tool interfaces, memory access, iteration limits, verification rules, and recovery behavior. This view distinguishes the model that proposes tokens from the system that turns those tokens into a bounded task execution. ReAct couples reasoning traces with actions in a prompt-level loop [74], while SWE-agent shows that an agent-computer interface can be a decisive part of software-engineering performance [73]. Harness engineering therefore includes the interaction surface and control policy alongside the backbone model [67].

Several methods improve only part of this execution policy. GEPA evolves prompts through reflective feedback and keeps the underlying agent program fixed [1]. ADAS searches over agent designs, and AFlow generates workflow programs before deployment [24, 78]. AgentSquare recombines four predefined modules, planning, reasoning, tool use, and memory, within a modular design space [56]. These methods show that textual instructions, workflow structure, and modular control logic can all be optimization variables. Their search spaces and evaluation loops differ, however, so an improvement in one prompt or workflow does not imply a reusable representation for changing another harness or another agent family.

Open-ended evolution extends the search to executable agent code. The Darwin Gödel Machine edits its own implementation and evaluates descendants in an open-ended loop [77]. Growing Harness takes a narrower position on a frozen model. The model and tools remain fixed while an offline optimizer revises the harness using execution traces and diagnostic artifacts, including evaluator feedback and errors unavailable to the deployed harness [34]. Compared with AgentSquare’s module recombination, the optimization target is the harness as a whole, and development and service run in separate loops. The resulting specialist harnesses are then reused across tasks. However, separating the loops still leaves a credit-assignment problem. A candidate may change a recovery branch, a memory instruction, or a tool policy while the measured task score reflects all of them together. Without a stable edit vocabulary and a fixed screening protocol, the evolver can select a patch whose causal effect is unclear or whose gain does not transfer.

Beyond generating candidates, evolutionary search can retain multiple solutions. MAP-Elites maintains high-performing elites across cells that partition the search space by behavior [43]. HarnessBank applies this principle to agent-harness self-evolution with a semantic gene bank, diagnosed failure categories, recombination, and gated verification [39]. Its screening procedure checks that a proposed intervention actually activates, that the score comparison is paired on the same tasks, and that the observed gain passes an operational threshold. These checks combine evidence of activation with paired performance measurements during candidate selection.

In Raven, the harness is factored into Memory, Planning, Capability, and Action strategy interfaces. This boundary lets the evolver propose a targeted change and the evaluator attribute it to an edit category within the diagnosis, gene-bank, and gated-screening procedure. The development loop searches and screens candidates, then freezes the selected harness before service use. The Host Agent remains responsible for choosing among the resulting model-harness pairs and composing them with other agents. This separation distinguishes Raven from prompt-only optimization, workflow synthesis, and unrestricted self-editing while retaining their useful search primitives.

8.3 Long-Term Memory for Agents

Agent memory research varies along several independent dimensions: the type of record, the time scale, the retrieval index, the treatment of provenance, and whether maintenance can proceed while the agent serves a request. A recent survey separates factual, experiential, and working memory and emphasizes that a memory system is more than a vector store [25]. This taxonomy is useful for Raven because user facts, agent cases, task artifacts, and reusable procedures have different owners and different consequences when they are retrieved.

Early systems emphasize context management and reflection. MemGPT treats the model as an operating system that moves information between memory tiers with different access costs [48].

Generative Agents record observations, retrieve relevant episodes, and periodically reflect to form higher-level memories [49]. A-MEM constructs and updates a network of linked note cards, while Zep represents agent memory as a temporal knowledge graph with source and validity information [72, 54]. These systems illustrate different trade-offs between a compact working context, associative retrieval, and explicit temporal structure. These distinctions motivate explicit scope and temporal information for records used in later decisions.

Beyond context management, production-oriented memory systems address scale and operational updates, using different principles to organize memory. Mem0 runs a two-phase extraction and update pipeline and leaves hierarchical memory architectures to future work [11]. MemoryOS divides storage by conversational time scale into short-term, mid-term, and long-term personal memory [27]. MemOS instead distinguishes memory by substrate, separating plaintext, activation, and parameter memory [37]. MIRIX keeps procedural memory alongside other memory types in one multi-agent system [65]. Letta’s sleep-time updates demonstrate that memory consolidation can run asynchronously so that a foreground request does not wait for every update [33]. These systems improve availability and reuse, yet a memory write and a task dependency remain different kinds of relation. A record may describe a previous attempt without being a required predecessor for the current computation.

EverOS organizes memory through extraction, consolidation, and retrieval processes for long-horizon reasoning, with separate user and agent tracks [23, 15]. Its user track can retain profiles and episodes, while its agent track records cases and experience-derived skills. The distinction gives an agent a durable source of experience without treating another user’s private context as a shared resource. It also supports asynchronous writes and retrieval adapters that can be selected by the host application.

Raven uses these memory facilities to supply context within the orchestration structure. On a user-facing invocation, the host may add a lexical profile slice and selected EverOS user memories. For a specialist, the adapter can retrieve agent cases and skills within that agent’s scope. The collaboration layer decides what is visible at each invocation, and the prompt marks retrieved text as untrusted context. A worker’s session record can be stored for later memory extraction, while the graph’s predecessor artifacts remain explicit in the artifact ledger. This separation keeps retrieved experience distinct from the declared artifact dependencies and authorized inputs of each invocation.

8.4 Skill Acquisition, Retrieval, and Evolution

Skills provide a middle ground between one-off tool calls and changes to a general harness. They package a task class, instructions, and optional resources so that an agent can reuse a procedure across requests. Voyager learns executable code skills in an embodied environment [62]. The Agent Skills specification standardizes a lightweight SKILL.md format with a name, description, and on-demand loading [2]. ExpeL stores natural-language insights from successful experience, and SkillWeaver discovers and refines procedures for web interaction [79, 80]. LATM instead treats reusable tool construction and request-conditioned tool selection as the central problem [5]. These methods differ in whether the learned object is code, instructions, or a tool wrapper, yet all make procedural knowledge available outside the immediate context.

The open-skill ecosystem introduces a separate corpus problem. A public catalog can contain duplicate names, near-duplicate bodies, malformed metadata, unsafe instructions, and procedures whose descriptions do not match their implementation. SkillCorpus addresses these issues with source collection, structural filtering, semantic deduplication, quality and safety judgments, release admission, and retrieval artifacts [64]. Its reference stack rewrites a request when needed, retrieves and reranks candidates, and lets an LLM select a bounded number of full skill bodies. This separation between corpus curation and online selection matters because a high retrieval score alone does not show that a procedure is executable or appropriate for the receiving agent.

Skill maintenance is increasingly treated as an explicit operation. Memp separates skill construction, retrieval, and update, which makes it possible to evaluate procedural memory independently from initial discovery [16]. ExpeL and SkillWeaver update procedures from experience, but their learned artifacts are tied to the task environments and feedback signals used during acquisition. A persistent skill also needs versioning, supporting cases, and a policy for rejecting regressions. These requirements become more demanding when one catalog serves agents with different tools and context limits.

Raven builds a single task-aware selection path over three sources: local skills, memory-derived skills, and SkillHub. It fuses source hits by name, keeps provenance, applies bounded retrieval and body budgets, and uses a compatibility gate before injecting selected procedures. The update path is separate. An execution case enters EverOS, where clustering and skill operators can propose a new or revised procedure for later indexing. Retrieval can therefore continue while revisions are pending. Persisted revisions become available for subsequent selection after indexing.

Taken together, the four research lines are complementary at different levels of the Raven architecture. Orchestration determines which executor runs and which artifacts it receives. Harness evolution changes how that executor reasons, uses tools, and recovers. Memory preserves user context and agent experience, while Skill Forge exposes selected procedures to future runs. Raven preserves explicit boundaries between these mechanisms. Memory records remain distinct from graph edges, skill selection remains distinct from tool permission, and harness updates remain distinct from the host’s routing policy. The evaluation therefore reports planning, harness adaptation, specialist execution, and skill reuse as separate capability levels.

9 Conclusion and Future Work

We introduced Raven, *The Harness of Harnesses*, an open-source multi-agent ecosystem that addresses the growing complexity of harness design and the difficulty of composing specialized capabilities across domains. Raven treats each executable model–harness pair as a unit of intelligence that can be constructed, improved through experience, and composed with other agents. The Host Agent organizes native and third-party agents through explicit task dependencies, while modular harness evolution, persistent memory, and Skill Forge support adaptation and reuse across tasks. Our theory establishes sufficient conditions under which composition expands reliable task coverage beyond that of the available individual agents under a shared resource budget, accounting for planning and coordination costs.

To assess the system’s components, our evaluation examines planning quality, specialist performance, harness adaptation, and skill reuse. On MAOB, Raven ranks first among the compared systems on all four planning metrics under both tested backbones, improving Exact Match over the strongest baseline by 10.4 and 10.5 percentage points, respectively. Evaluations of the four native specialists characterize task performance and resource use in research, software engineering, visual design, and sustained operation. The published HarnessBank experiments demonstrate the benefits of harness adaptation with a frozen backbone, while the SkillCorpus experiments show that a curated skill library improves Raven on three benchmarks, with larger gains than OpenClaw on two of them.

Building on this ecosystem, our future work centers on an interconnected multi-agent system spanning everyday devices and environments. We envision a Raven instance for each device or environment, managing and orchestrating all agents within its scope. We will develop communication and coordination mechanisms between these instances, using protocols for agent-to-agent (A2A) communication to exchange capability information, delegate tasks, and return results. Each agent will remain managed by its local Raven instance, while the instances coordinate collaboration across devices. Connecting all of a user’s agents in this way would form an *All-Domain Collaboration Network*, making the network’s capabilities accessible through requests from any connected device or interaction interface.

We aim for this network to function as a persistent AI assistant that maintains continuity across devices and adapts through experience. To provide this continuity, we will connect the memory systems of Raven instances, transfer task context, and synchronize execution progress so that users can move between interfaces while ongoing work continues. User preferences and accumulated experience will inform coordination across the network, allowing an interaction begun on one device to continue on another without requiring the user to reconstruct its history. Harness evolution and reusable skills will support continued adaptation as the network encounters new tasks and environments. End-to-end evaluations will guide this development by measuring task success, continuity of user experience, and coordination cost.

References

- [1] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning*

- Representations*, volume 2026, pages 8479–8565. 2026.
- [2] Anthropic. Agent Skills, 2025. URL <https://agentskills.io/>. Open format specification; accessed September 26, 2026.
 - [3] Saddek Bensalem, Yi Dong, Martin Fränzle, Xiaowei Huang, et al. Position: A three-layer probabilistic assume-guarantee architecture is structurally required for safe LLM agent deployment. *arXiv preprint arXiv:2605.18672*, 2026. doi:10.48550/arXiv.2605.18672. URL <https://arxiv.org/abs/2605.18672v1>.
 - [4] Siqi Cai, Shaopeng Chen, Xiang Fei, Yong Mao, et al. Tencent WorkBuddy Bench: A multi-domain coding-agent benchmark with contamination-resistant task construction. *arXiv preprint arXiv:2607.20911*, 2026. URL <https://arxiv.org/abs/2607.20911>.
 - [5] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, et al. Large language models as tool makers. In *International Conference on Learning Representations*, volume 2024, pages 54067–54089. 2024.
 - [6] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, et al. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems*, volume 38. 2026.
 - [7] Center for AI Safety, Long Phan, Alice Gatti, Nathaniel Li, et al. A benchmark of expert-level academic questions to assess AI capabilities. *Nature*, 649(8099):1139–1146, 2026. doi:10.1038/s41586-025-09962-4. Humanity’s Last Exam. Preprint: arXiv:2501.14249.
 - [8] Kaiyuan Chen, Yixin Ren, Yang Liu, Xiaobo Hu, et al. xbench: Tracking agents productivity scaling with profession-aligned real-world evaluations. *arXiv preprint arXiv:2506.13651*, 2025. URL <https://arxiv.org/abs/2506.13651>.
 - [9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
 - [10] Xin-Sheng Chen, Jiayu Zhu, Pei-lin Li, Hanzheng Wang, et al. PresentBench: A fine-grained rubric-based benchmark for slide generation. *arXiv preprint arXiv:2603.07244*, 2026. URL <https://arxiv.org/abs/2603.07244>.
 - [11] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, et al. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025. doi:10.48550/arXiv.2504.19413. URL <https://arxiv.org/abs/2504.19413>.
 - [12] Yufan Dang, Chen Qian, Xueheng Luo, Jingru Fan, et al. Multi-agent collaboration via evolving orchestration. In *Advances in Neural Information Processing Systems*, volume 38, pages 165025–165059. 2026.
 - [13] DeepSeek AI. DeepSeek Harness (dsh), 2026. URL <https://github.com/deepseek-ai/deepseek-harness>. Software repository; version 0.1.0-rc.5 evaluated.
 - [14] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025. URL <https://arxiv.org/abs/2509.16941>.
 - [15] EverMind AI. EverOS: A portable memory layer for AI agents, 2026. URL <https://github.com/EverMind-AI/EverOS>. Software repository, accessed September 27, 2026.
 - [16] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, et al. Memp: Exploring agent procedural memory. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 17490–17502. 2026.
 - [17] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, et al. Magentic-One: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468*, 2024. doi:10.48550/arXiv.2411.04468. URL <https://arxiv.org/abs/2411.04468v1>.
 - [18] Xingze Gao, Chuanrui Hu, Hongda Chen, Pengfei Yao, et al. EvoAgentBench: Benchmarking agent self-evolution via ability transfer. *arXiv preprint arXiv:2607.05202*, 2026. URL <https://arxiv.org/abs/2607.05202>.
 - [19] Google. Gemini Deep Research agent, 2026. URL <https://ai.google.dev/gemini-api/docs/deep-research>. Model deep-research-preview-04-2026; accessed September 25, 2026.

- [20] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969. doi:[10.1145/363235.363259](https://doi.org/10.1145/363235.363259). URL <https://doi.org/10.1145/363235.363259>.
- [21] Deyao Hong, Yizhe Chi, Wenyi Li, Xiaoqiu Wang, et al. SWE Refactor Bench: Can coding agents complete a long-horizon, whole-repository stack migration? *arXiv preprint arXiv:2608.23564*, 2026. URL <https://arxiv.org/abs/2608.23564>.
- [22] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xianwu Zheng, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, volume 2024, pages 23247–23275. 2024.
- [23] Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, et al. EverMemOS: A self-organizing memory operating system for structured long-horizon reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 45836–45853. 2026.
- [24] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, volume 2025, pages 21344–21377. 2025.
- [25] Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, et al. Memory in the age of AI agents. *arXiv preprint arXiv:2512.13564*, 2026. doi:[10.48550/arXiv.2512.13564](https://arxiv.org/abs/2512.13564). URL <https://arxiv.org/abs/2512.13564v2>. Version 2, revised January 2026; first posted December 2025.
- [26] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, et al. SWE-bench: Can language models resolve real-world Github issues? In *The Twelfth International Conference on Learning Representations*. 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- [27] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 25972–25981. 2025.
- [28] Andrej Karpathy. nanochat: The best ChatGPT that \$100 can buy, 2025. URL <https://github.com/karpathy/nanochat>. Software repository; accessed September 25, 2026.
- [29] Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically, 2026. URL <https://github.com/karpathy/autoresearch>. Software repository; accessed September 25, 2026.
- [30] Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, et al. Towards a science of scaling agent systems. *arXiv preprint arXiv:2512.08296*, 2026. doi:[10.48550/arXiv.2512.08296](https://arxiv.org/abs/2512.08296). URL <https://arxiv.org/abs/2512.08296v3>. Version 3, revised April 2026; first posted December 2025.
- [31] Satyapriya Krishna, Kalpesh Krishna, Anhad Mohananey, Steven Schwarcz, et al. Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 4745–4759. 2025. URL <https://aclanthology.org/2025.naacl-long.243/>.
- [32] Marta Kwiatkowska, Gethin Norman, David Parker, and Hongyang Qu. Assume-guarantee verification for probabilistic systems. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 6015 of *Lecture Notes in Computer Science*, pages 23–37. Springer, 2010. doi:[10.1007/978-3-642-12002-2_3](https://doi.org/10.1007/978-3-642-12002-2_3). URL <https://qav.cs.ox.ac.uk/papers/tacas10.pdf>.
- [33] Letta. Agent memory: How to build agents that learn and remember, 2025. URL <https://www.letta.com/blog/agent-memory/>. Blog post, July 7, 2025; accessed September 26, 2026.
- [34] Laizhen Li, Jiarui Li, Juanjuan Zhao, Kejiang Ye, et al. Grow the harness, not the context: From strategy-free scaffolds to reusable specialist agents. *arXiv preprint arXiv:2609.26760*, 2026. doi:[10.48550/arXiv.2609.26760](https://arxiv.org/abs/2609.26760). URL <https://arxiv.org/abs/2609.26760v2>. Version 2.
- [35] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, et al. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026. URL <https://arxiv.org/abs/2602.12670>.

- [36] Yuetai Li, Yichen Feng, Zhangchen Xu, Zixian Ma, et al. JobBench: Aligning agent work with human will. *arXiv preprint arXiv:2605.26329*, 2026. doi:10.48550/arXiv.2605.26329. URL <https://arxiv.org/abs/2605.26329>.
- [37] Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, et al. MemOS: A memory OS for AI system. *arXiv preprint arXiv:2507.03724*, 2025. doi:10.48550/arXiv.2507.03724. URL <https://arxiv.org/abs/2507.03724v4>. Version 4, revised December 2025.
- [38] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, et al. AgentBench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations*. 2024. URL <https://arxiv.org/abs/2308.03688>.
- [39] Xiaotian Luo, Dizhan Xue, Fengxingyu Wang, Chuanrui Hu, et al. HarnessBank: Semantic gene-bank search with gated verification for agent-harness self-evolution. *arXiv preprint arXiv:2607.13683*, 2026. URL <https://arxiv.org/abs/2607.13683v2>. Version 2.
- [40] Ruiying Ma, Shreya Shankar, Ruiqi Chen, Yiming Lin, et al. Can AI agents answer your data questions? A benchmark for data agents. *arXiv preprint arXiv:2603.20576*, 2026. URL <https://arxiv.org/abs/2603.20576>.
- [41] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *International Conference on Learning Representations*, volume 2026, pages 40903–40986. 2026.
- [42] MiroMind Team. MiroThinker-1.7 and H1: Towards heavy-duty research agents via verification. *arXiv preprint arXiv:2603.15726*, 2026. URL <https://arxiv.org/abs/2603.15726>.
- [43] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015. URL <https://arxiv.org/abs/1504.04909>.
- [44] Navers Lab, Einsia AI. SWE Refactor Bench leaderboard, 2026. URL <https://lab.einsia.ai/swe-refactor-bench/leaderboard/>. Accessed September 25, 2026.
- [45] Peter W. O’Hearn. Resources, concurrency, and local reasoning. *Theoretical Computer Science*, 375(1–3):271–307, 2007. doi:10.1016/j.tcs.2006.12.035. URL <https://flint.cs.yale.edu/cs428/doc/concurrency.pdf>.
- [46] OpenAI. Introducing SWE-bench Verified, 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>. Blog post, August 13, 2024.
- [47] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. 2022.
- [48] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, et al. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL <https://arxiv.org/abs/2310.08560>.
- [49] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, et al. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22. 2023.
- [50] Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, et al. GDPval: Evaluating AI model performance on real-world economically valuable tasks. In *International Conference on Learning Representations*, volume 2026, pages 24005–24040. 2026.
- [51] Perplexity. Sonar Deep Research, 2025. URL <https://docs.perplexity.ai/docs/sonar/models/sonar-deep-research>. API documentation; accessed September 25, 2026.
- [52] Chen Qian, Zihao Xie, YiFei Wang, Wei Liu, et al. Scaling large language model-based multi-agent collaboration. In *International Conference on Learning Representations*, volume 2025, pages 41488–41505. 2025.
- [53] Qwen Team, Alibaba Group. QwenClawBench: Real-user-distribution benchmark for OpenClaw agents, 2026. URL <https://github.com/SKYLENAGE-AI/QwenClawBench>. Released April 2026.

- [54] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, et al. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025. doi:10.48550/arXiv.2501.13956. URL <https://arxiv.org/abs/2501.13956>.
- [55] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*. 2023.
- [56] Yu Shang, Yu Li, Keyu Zhao, Likai Ma, et al. AgentSquare: Automatic LLM agent search in modular design space. In *International Conference on Learning Representations*, volume 2025, pages 3841–3865. 2025.
- [57] Peter Steinberger and OpenClaw Contributors. OpenClaw, 2025. URL <https://github.com/openclaw/openclaw>. Open-source agent harness.
- [58] Shiqian Su, Sen Xing, Xuan Dong, Muyan Zhong, et al. MiroFlow: Towards high-performance and robust open-source agent framework for general deep research tasks. *arXiv preprint arXiv:2602.22808*, 2026. URL <https://arxiv.org/abs/2602.22808>.
- [59] Tencent WorkBuddy Bench Team. WorkBuddy Bench code leaderboard, 2026. URL <https://workbuddybench.com/code.html>. Code subset, three runs averaged; accessed September 25, 2026.
- [60] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, et al. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024.
- [61] Vals AI. SWE-bench Verified benchmark, 2026. URL <https://www.vals.ai/benchmarks/swebench>. Leaderboard page updated September 1, 2026; accessed September 25, 2026.
- [62] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, et al. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. URL <https://arxiv.org/abs/2305.16291>.
- [63] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- [64] Yanze Wang, Pengfei Yao, Tianyi Sun, Chuanrui Hu, et al. SkillCorpus: Consolidating and evaluating the open skill ecosystem for real-world LLM agents. *arXiv preprint arXiv:2607.15557*, 2026. URL <https://arxiv.org/abs/2607.15557v6>. Version 6.
- [65] Yu Wang and Xi Chen. MIRIX: Multi-agent memory system for LLM-based agents. *arXiv preprint arXiv:2507.07957*, 2025. doi:10.48550/arXiv.2507.07957. URL <https://arxiv.org/abs/2507.07957>.
- [66] Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, et al. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025. URL <https://arxiv.org/abs/2504.12516>.
- [67] Lilian Weng. Harness engineering for self-improvement, 2026. URL <https://lilianweng.github.io/posts/2026-07-04-harness/>. Lil’Log, July 4, 2026; accessed September 26, 2026.
- [68] David H. Wolpert and William G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. doi:10.1109/4235.585893. URL <https://doi.org/10.1109/4235.585893>.
- [69] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023. URL <https://arxiv.org/abs/2308.08155>.
- [70] Yuhao Wu, Jingyuan Zhang, Jiajun Shi, Xinping Lei, et al. HarnessDev: Can LLMs create and evolve their own agent harness? *arXiv preprint arXiv:2609.01437*, 2026. doi:10.48550/arXiv.2609.01437. URL <https://arxiv.org/abs/2609.01437v1>.
- [71] xbench Team. xbench-DeepSearch-2510, 2025. URL <https://huggingface.co/datasets/xbench/DeepSearch-2510>. Dataset of 100 Chinese deep-search questions; accessed September 25, 2026.

- [72] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, et al. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, pages 17577–17604. 2026.
- [73] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, et al. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, pages 50528–50652. 2024.
- [74] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, et al. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*. 2023. URL <https://arxiv.org/abs/2210.03629>.
- [75] Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, et al. MasRouter: Learning to route LLMs for multi-agent systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15549–15572. Association for Computational Linguistics, 2025. doi:10.18653/v1/2025.acl-long.757. URL <https://aclanthology.org/2025.acl-long.757/>.
- [76] Chencheng Zhang, Yuhang Li, Can Xu, Jiaheng Liu, et al. ArtifactsBench: Bridging the visual-interactive gap in LLM code generation evaluation. *arXiv preprint arXiv:2507.04952*, 2025. URL <https://arxiv.org/abs/2507.04952>.
- [77] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, et al. Darwin Gödel Machine: Open-ended evolution of self-improving agents. In *International Conference on Learning Representations*, volume 2026, pages 104223–104294. 2026.
- [78] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, et al. AFlow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2025. doi:10.48550/arXiv.2410.10762. URL <https://arxiv.org/abs/2410.10762v4>. Version 4, revised April 2025.
- [79] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, et al. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642. 2024.
- [80] Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, et al. SkillWeaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025. doi:10.48550/arXiv.2504.07079. URL <https://arxiv.org/abs/2504.07079>.
- [81] YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, et al. SkillRouter: Skill routing for LLM agents at scale. *arXiv preprint arXiv:2603.22455*, 2026. URL <https://arxiv.org/abs/2603.22455>.

A Additional Theory Details

A.1 Notation and Scope

Table 6: Principal notation. States and correctness predicates are analytical objects. An executor receives only its permitted observations.

Symbol	Meaning
a_i, M_i, h_i	Execution policy, underlying model, and harness.
$\mathcal{H}_i, \mathcal{U}_i, \Delta$	Permitted histories, actions, and probability distributions.
$H, a_0, \mathcal{A}$	Host controller, its nondelegating executor, and the pool including a_0 .
S, S_H	A generic system, either the composed system or a standalone agent, and the composed system $H[\mathcal{A}]$.
t, r_t, μ_t, E_t, V_t	Task, request, initial law, response kernel, and objective correctness.
$\omega, \text{ref}, s, \ell, Y$	Reference conditions, their read-only state projection, full state, immutable-record ledger, and final record.
B, C_S, δ	Total resource budget, actual accrued cost, and tolerated failure probability.
$p_S, \mathcal{C}_S$	Budgeted success probability and reliable task set.
π, G, σ, λ	Annotated plan, DAG, executor assignment, and expanded operation order.
$K_{\text{op}}, \lambda_j, \mathcal{X}_v, \mathcal{Y}_v$	Number of operations, the j th operation, and node input and output spaces.
$\Phi_v, d, \mathcal{D}_{\text{pkt}}, \mathcal{M}_e$	Input assembly, authorized packet, packet space, and edge message space.
P_v, Q_v, J_e	Node precondition, node transition guarantee, and handoff guarantee.
A_j, D_j, I_j	Operation entry/transition predicates and the invariant after its prefix.
$\Gamma, \mathbf{b}$	Contract/invariant annotations and planning/node/edge allocations.
κ, K_κ, ν	Operation provider, conditional outcome kernel, and entry law.
$\mathcal{Z}_\kappa, \mathcal{O}_\kappa, \text{Pr}_{\nu, \kappa}$	Provider configuration and outcome spaces, and the local experiment.
$\mathfrak{L}, \Xi_j, \mathcal{O}_j$	Admissible entry laws and actual invocation configuration/outcome.
Σ_j, L_j, C_j	State, ledger, and charged cost at operation j .
Z, Π, G_t, S_t, q_t	Planning transcript, selected plan, valid-planning event, success event, and selected-plan success.
$R_{j-1}, \text{Ok}_j, F_j$	Observable prefix, operation-success event, and success through the current operation.
$\epsilon_j, \epsilon(\zeta, \pi)$	Local conditional failure bound and its sum for a selected plan.
$\eta_H, \bar{\epsilon}, \eta_\star, \epsilon_\star$	Planning-failure bound, uniform valid-plan execution envelope, and their family-level bounds.
$\mathcal{R}_v, \mathcal{C}^{\text{new}}, \mathcal{T}_\star$	Eligible realizers, new-task coverage, and a specified task family.
$\text{rec}_\pi, \mathcal{I}, \mathcal{L}$	Final-record projection, admitted initial pairs, and ledger space.
$\Omega_{\text{tr}}, \mathcal{W}, \mathcal{Y}_{\text{rec}}$	Trace, reference-condition, and execution-record spaces.

All task, state, contract, and probability objects refer to fixed system versions and a stated input law. In conditional arguments, the planning values (ζ, π) are fixed while execution randomness remains. Conditional probabilities are used only for conditioning events with positive probability. The countable encoding convention makes this conditioning unambiguous without requiring probability densities.

A.2 Operation Records and Execution Semantics

The ledger is a finite map whose existing records are immutable. Write $\ell[k]$ for the record stored under identifier k and $\ell[k \mapsto r]$ for extension by a fresh identifier k . The extension is defined only when $k \notin \text{dom}(\ell)$. A node record r_v contains its actual input $\text{in}(r_v)$, output $\text{out}(r_v)$, and relevant invocation trace. These fields record execution behavior independently of the worker’s account of it. An edge record contains the delivered message. The root record contains the authorized packet and task-reference bookkeeping. Only permitted fields are exposed to an executor.

For $v \in V$, let $x_v = \Phi_v(d, (\ell[e])_{e \in \text{In}(v)})$. The entry and transition predicates in the main text are defined by

$$A_v(s, \ell) \iff v \notin \text{dom}(\ell) \wedge \text{In}(v) \subseteq \text{dom}(\ell) \wedge P_v(x_v, s), \quad (82)$$

$$D_v(s, \ell, s', \ell') \iff \exists r_v : \ell' = \ell[v \mapsto r_v] \\ \wedge \text{in}(r_v) = x_v \wedge Q_v(x_v, s, \text{out}(r_v), s'). \quad (83)$$

All referenced records must also have their declared types. Otherwise, these predicates are false. Every D_v and D_e additionally requires $\text{ref}(s') = \text{ref}(s)$, preserving the read-only task reference. For

$e = (u, v)$, put $y_u = \text{out}(\ell[u])$. The transfer predicates are

$$\begin{aligned} A_e(s, \ell) &\iff e \notin \text{dom}(\ell) \wedge u \in \text{dom}(\ell), \\ D_e(s, \ell, s', \ell') &\iff \exists m : \ell' = \ell[e \mapsto m] \wedge J_e(y_u, s, m, s'). \end{aligned} \quad (84)$$

Costs and acceptance are outcome fields, so successful completion also requires acceptance and compliance with the allocation. Transitions D_v, D_e express semantic correctness independently of those fields. The correspondence with the schedule is $(A_j, D_j) = (A_{\lambda_j}, D_{\lambda_j})$ and $\epsilon_v = \epsilon_{\lambda^{-1}(v)}$, $\epsilon_e = \epsilon_{\lambda^{-1}(e)}$.

With these record definitions, serial execution starts from the post-planning state (Σ_0, L_0) . Operation λ_j is attempted only after its predecessors in λ have completed and been accepted. Its outcome supplies (Σ_j, L_j) , cost C_j , and its acceptance status. The protocol need not observe the truth of D_j . A rejection, crash, timeout, or nontermination can prevent plan completion. If every operation succeeds, the protocol completes the finite sequence, delivers the final execution record $\text{rec}_\pi(L_{K_{\text{op}}}, \Sigma_{K_{\text{op}}})$ in the final operation, and terminates. Final delivery and its cost belong to that operation. The success event therefore includes any delay or failure associated with final delivery.

Extending the analysis to a concurrent protocol requires each of its runs to have a measurable serial representation respecting λ with the same invocation inputs and outputs, relevant state transitions, acceptance outcomes, terminal record, and total charged cost. Probability bounds then use the law induced by the actual concurrent runs on these representations. Standalone serial error rates cannot be substituted. This trace-preservation premise permits the pathwise deterministic lemma and the probability argument to apply. Footprint separation is useful for proving such premises in suitable program models [45], but does not alone guarantee preservation of timing-sensitive service responses, acceptance, or cost. The grammar compilation result below is proved for its serial protocol. Applying it to concurrent execution requires the same additional premise.

Ledger Extension. A compiled fragment reads only its own record identifiers and its declared input interface. If a ledger ℓ_{ext} has a disjoint domain and none of its keys is read by the fragment, adding ℓ_{ext} leaves its input resolvers and entry predicates unchanged. Each transition above lifts by retaining those extra records in both ℓ and ℓ' . This follows by substitution in Eqs. (82) to (84): freshness and all referenced values are preserved, and the state predicates do not inspect unrelated records. The fragment's invariants lift by restricting the larger ledger to the fragment's identifiers and interface records. No claim is made for arbitrary predicates that inspect the size or unrelated contents of the global ledger.

A.3 A Finite-Budget Information Limit

The following search problem illustrates the effect of a finite information budget in a black-box setting consistent with no-free-lunch results [68]. Let $\mathcal{X}$ contain N_{pos} positions, where $N_{\text{pos}} \geq 2$ is an integer, and let the unknown target be $z \in \mathcal{X}$. The oracle returns $f_z(x) = 0$ at $x = z$ and 1 otherwise. An algorithm A has no side information about z , its internal seed is independent of z , and it returns an evaluated position $\hat{x}$ after at most n_{qry} distinct queries, where n_{qry} is an integer with $1 \leq n_{\text{qry}} < N_{\text{pos}}$.

Proposition 2 (Finite-query coverage limit). *Every possibly randomized and adaptive algorithm in this setting satisfies*

$$\frac{1}{N_{\text{pos}}} \sum_{z \in \mathcal{X}} \mathbb{P}_A(\hat{x} = z \mid f_z) \leq \frac{n_{\text{qry}}}{N_{\text{pos}}}. \quad (85)$$

Thus some target has success probability at most $n_{\text{qry}}/N_{\text{pos}}$.

Proof. Fix the internal seed and follow the branch on which every response is 1. It queries at most n_{qry} positions. If z lies outside this set, the actual transcript follows exactly that branch, so the returned evaluated position cannot be the target. Hence at most n_{qry} targets permit success for this seed. Averaging over a uniform target and over the independent seed proves the inequality. At least one target must then have a success probability no greater than the bound on the mean. $\square$

If $n_{\text{qry}}/N_{\text{pos}} < 1 - \delta$, reliable coverage of every target is impossible. A host and workers sharing n_{qry} total queries induce another adaptive querying algorithm and obey the same bound because

parallel queries can be serialized without adding observations. If an unevaluated final guess is allowed, the all-failure branch adds at most one successful target, giving $\min\{1, (n_{\text{qry}} + 1)/N_{\text{pos}}\}$ instead. The result does not exclude advantages from structured priors or exhaustive search with enough queries.

A.4 From Local Experiments to Workflow Reliability

Fix the actual law $\mathbb{P} = \mathbb{P}_{t, S_H, B}$ and a valid planning pair (ζ, π) . For a host-visible prefix value ϱ , let

$$H_{j, \varrho} = F_{j-1} \cap \{R_{j-1} = \varrho, Z = \zeta, \Pi = \pi\}, \quad \nu_{j, \varrho}(c_{\text{cfg}}) = \mathbb{P}(\Xi_j = c_{\text{cfg}} \mid H_{j, \varrho}),$$

whenever $\mathbb{P}(H_{j, \varrho}) > 0$. The configuration Ξ_j contains the operation's entry state and ledger as defined by the protocol. Write $A_j(c_{\text{cfg}})$ for evaluation of A_j on those components.

Lemma 2 (Local-to-workflow consistency). *Suppose provider κ_j realizes $(A_j, D_j; b_{\lambda_j}, \epsilon_j)$ over $\mathfrak{L}_j$. For each positive-probability $H_{j, \varrho}$, assume $\nu_{j, \varrho} \in \mathfrak{L}_j$ and*

$$\mathbb{P}(O_j = o \mid \Xi_j = c_{\text{cfg}}, H_{j, \varrho}) = K_{\kappa_j}(o \mid c_{\text{cfg}}) \quad (86)$$

for all c_{cfg} of positive conditional mass and all outcomes o . Then

$$\mathbb{P}(\text{Ok}_j^c \mid R_{j-1} = \varrho, F_{j-1}, Z = \zeta, \Pi = \pi) \leq \epsilon_j, \quad (87)$$

and Eq. (15) follows.

Proof. The successful-prefix part of Lemma 1 gives A_j at entry. Using the actual definition of Ok_j and Eq. (86),

$$\begin{aligned} \mathbb{P}(\text{Ok}_j \mid H_{j, \varrho}) &= \sum_{c_{\text{cfg}}, o} \nu_{j, \varrho}(c_{\text{cfg}}) K_{\kappa_j}(o \mid c_{\text{cfg}}) \text{ok}_{D_j, b_{\lambda_j}}(c_{\text{cfg}}, o) \\ &= \Pr_{\nu_{j, \varrho}, \kappa_j}(\text{ok}_{D_j, b_{\lambda_j}}) \geq 1 - \epsilon_j. \end{aligned}$$

The final inequality is contract realization for the specified entry law. Let $H_j = F_{j-1} \cap \{Z = \zeta, \Pi = \pi\}$ and $w_j(\varrho) = \mathbb{P}(R_{j-1} = \varrho \mid H_j)$. For $\mathbb{P}(H_j) > 0$,

$$\mathbb{P}(\text{Ok}_j^c \mid H_j) = \sum_{\varrho: w_j(\varrho) > 0} w_j(\varrho) \mathbb{P}(\text{Ok}_j^c \mid H_{j, \varrho}) \leq \epsilon_j. \quad (88)$$

This is the required successful-prefix bound. $\square$

Kernel consistency links the local experiment to the actual invocation. Configurations can retain full relevant history or latent state. If planning or earlier outcomes change the conditional law of an omitted variable, a kernel calibrated only on visible inputs may not satisfy Eq. (86). The configuration must then include the missing information, or the outcome model must be refined. These analytical representations leave the worker's permitted observations unchanged.

Conditioning on prior correctness is necessary for the stated local bound. For example, let U be a fair bit, R_1 constant, $\text{Ok}_1 = \{U = 0\}$, and $\text{Ok}_2 = \{U = 1\}$. Then $\mathbb{P}(\text{Ok}_2^c \mid R_1) = 1/2$ but $\mathbb{P}(\text{Ok}_2^c \mid R_1, F_1) = 1$, while joint success is zero. Thus the conditional bound cannot generally omit F_1 , and the joint success probability cannot generally be obtained by multiplying the two marginal success probabilities. It does not refute Boole's bound using actual marginal failure rates. Those rates also yield a union bound, but isolated-provider error rates are not automatically the actual marginals or successful-prefix rates of a selected workflow.

Beyond the additive bound, the same successful-prefix bounds imply

$$\mathbb{P}(F_{K_{\text{op}}} \mid Z = \zeta, \Pi = \pi) \geq \prod_{j=1}^{K_{\text{op}}} (1 - \epsilon_j) \geq \left[1 - \sum_{j=1}^{K_{\text{op}}} \epsilon_j \right]_+. \quad (89)$$

If all factors are positive, induction ensures positive-probability prefixes and permits multiplication of their conditional bounds. If a factor is zero the product lower bound is trivial. The final inequality follows by expanding two factors, $(1 - a)(1 - b) \geq 1 - a - b$ for $a, b \in [0, 1]$, and induction. This argument does not require independence.

For plans with at most n_0 nodes and r_0 handoffs, suppose node and handoff cost allocations are at most $b_V^{\max}$ and $b_E^{\max}$ and their conditional errors are at most $\epsilon_V^{\max}$ and $\epsilon_E^{\max}$, respectively. Here n_0, r_0 are nonnegative integers, $b_V^{\max}, b_E^{\max} \geq 0$, and $\epsilon_V^{\max}, \epsilon_E^{\max} \in [0, 1]$. A sufficient design-time resource envelope is

$$b_H + n_0 b_V^{\max} + r_0 b_E^{\max} \leq B. \quad (90)$$

If valid planning has probability at least $1 - \eta_*$, with $\eta_* \in [0, 1]$, Theorem 1 and Proposition 1 give

$$p_{S_H}(t; B) \geq (1 - \eta_*)[1 - n_0 \epsilon_V^{\max} - r_0 \epsilon_E^{\max}]_+. \quad (91)$$

The envelope gives a sufficient design-time condition for resource feasibility. A large error sum weakens the lower bound without establishing a corresponding decrease in actual performance.

These local reliability bounds connect to executor selection through the eligible sets, as formalized next.

Proposition 3 (Eligible assignments). *Fix a decomposition with nonempty eligible sets $\mathcal{R}_v$, and choose $\sigma(v) \in \mathcal{R}_v$ for every node. Suppose that this assignment and its instance bindings give a compatible, feasible plan, that the transfers realize their contracts, and that the actual contexts satisfy Eq. (86) and entry-law membership for all operations. At every valid planning pair the plan then satisfies Eq. (15), hence the bound in Theorem 1. If $\bigcap_v \mathcal{R}_v = \emptyset$, no constant assignment realizes all the node contracts under those same allocations and law families.*

Proof. Membership in $\mathcal{R}_v$ gives each selected node’s realization. Together with transfer realization and the context conditions, Lemma 2 gives every successful-prefix error bound. The reliability theorem applies. A constant realizing assignment would select an executor belonging to every $\mathcal{R}_v$, contradicting the empty intersection. $\square$

Actual invocation laws can depend on the whole assignment. Therefore, nonempty eligible sets alone do not certify their context-membership premise. The proposition concerns this decomposition and does not exclude other ways an individual agent might solve the complete task.

A.5 A Typed Task Grammar and Its Compilation

To establish compatible-plan existence constructively, we define a restricted task grammar. Its primitive specifications, input distributions, and admissible composition rules are fixed before observing the host’s success.

Primitive Specifications and Syntax. A primitive $p \in \mathcal{P}$ has input/output types $(\mathcal{X}_p, \mathcal{Y}_p)$, precondition $P_p(x, s)$, postcondition $Q_p(x, s, y, s')$, an assigned executor in $\mathcal{A}$, and a declared state footprint. Its relational specification is the set of tuples (x, s, y, s') with $P_p(x, s) = 1$ and $Q_p(x, s, y, s') = 1$. A postcondition specifies the required result. A separate realization bound quantifies the executor’s ability to achieve it. Expressions have the syntax

$$\psi ::= p \mid \text{Seq}_\phi(\psi_1, \psi_2) \mid \text{Par}_{p_{\text{fork}}, p_{\text{join}}}(\psi_1, \psi_2). \quad (92)$$

The primitives $p_{\text{fork}}, p_{\text{join}} \in \mathcal{P}$ are explicit fork and join operations. A type judgment $\psi : \mathcal{X} \rightsquigarrow \mathcal{Y}$ means that the expression accepts an input of type $\mathcal{X}$ and returns an output of type $\mathcal{Y}$, with precondition P_ψ and postcondition Q_ψ . Write $\mathcal{X}_\psi$ and $\mathcal{Y}_\psi$ for these declared input and output types. Admitted expressions form the least class containing the primitives and closed under the two guarded rules below.

Sequential Rule. For $\psi_i : \mathcal{X}_i \rightsquigarrow \mathcal{Y}_i$, let $\phi : \mathcal{Y}_1 \rightarrow \mathcal{X}_2$ be a typed projection or serialization from the specified wiring library. The sequential expression has type $\mathcal{X}_1 \rightsquigarrow \mathcal{Y}_2$ and is admitted only if

$$P_{\psi_1}(x, s) \wedge Q_{\psi_1}(x, s, y_1, s_1) \implies P_{\psi_2}(\phi(y_1), s_1) \quad (93)$$

for all well-typed values. Define $P_\psi = P_{\psi_1}$ and

$$Q_\psi(x, s, y, s') \iff \exists y_1, s_1 : Q_{\psi_1}(x, s, y_1, s_1) \wedge Q_{\psi_2}(\phi(y_1), s_1, y, s'). \quad (94)$$

Thus the output relation composes the two specifications through the declared transfer. The connecting handoff contract requires exactly $\phi(y_1)$ and preserves the mutable state. Its cost and error are explicit. Any content transformation outside the wiring library, including deterministic computation, must be represented as another primitive instead of being included implicitly in ϕ .

Parallel Rule and State Separation. For $\psi_i : \mathcal{X}_i \rightsquigarrow \mathcal{Y}_i$, require $p_{\text{fork}} : \mathcal{X} \rightsquigarrow (\mathcal{X}_1 \times \mathcal{X}_2)$ and $p_{\text{join}} : (\mathcal{Y}_1 \times \mathcal{Y}_2) \rightsquigarrow \mathcal{Y}$. The constructor is admitted only if its branch implementations and contracts have a common state decomposition

$$\mathcal{S} = \mathcal{S}_{\text{sh}} \times \mathcal{S}_1 \times \mathcal{S}_2 \times \mathcal{S}_{\text{rest}}.$$

The shared component is read-only during the branches. Subject to its executor's permitted observations, a branch's state accesses are confined to the shared and its own components. It may change only its own component. For $k \in \{1, 2\}$, there must exist local predicates $\tilde{P}_k, \tilde{Q}_k$ such that

$$P_{\psi_k}(x, s) = \tilde{P}_k(x, s^{[\text{sh}]}, s^{[k]}), \quad (95)$$

$$\begin{aligned} Q_{\psi_k}(x, s, y, s') &\iff \tilde{Q}_k(x, s^{[\text{sh}]}, s^{[k]}, y, (s')^{[k]}) \\ &\quad \wedge (s')^{[\text{sh}]} = s^{[\text{sh}]} \wedge (s')^{[\neg k]} = s^{[\neg k]}. \end{aligned} \quad (96)$$

Here $s^{[\neg k]}$ collects the other mutable components. These footprint conditions also constrain the implementations. Under these conditions, each branch preserves the other's precondition in the serial composition proof. Nested partitions refine the parent branch's mutable component and may inherit only its read-only shared components. They cannot expose a sibling's mutable state. The join runs after both branches. Concurrent use additionally requires the trace-preservation premise in Appendix A.2.

The fork must establish both branch preconditions:

$$P_{p_{\text{fork}}}(x, s) \wedge Q_{p_{\text{fork}}}(x, s, (x_1, x_2), s_{\text{fork}}) \implies P_{\psi_1}(x_1, s_{\text{fork}}) \wedge P_{\psi_2}(x_2, s_{\text{fork}}). \quad (97)$$

Use branch order ψ_1 then ψ_2 for the logical serialization. The join condition, universally quantified over the fork and branch results, is

$$\begin{aligned} &P_{p_{\text{fork}}}(x, s) \wedge Q_{p_{\text{fork}}}(x, s, (x_1, x_2), s_{\text{fork}}) \\ &\quad \wedge Q_{\psi_1}(x_1, s_{\text{fork}}, y_1, s_1) \wedge Q_{\psi_2}(x_2, s_1, y_2, s_2) \\ &\implies P_{p_{\text{join}}}((y_1, y_2), s_2). \end{aligned} \quad (98)$$

This joint condition requires compatibility between the branch outputs. Set $P_\psi = P_{p_{\text{fork}}}$ and define

$$\begin{aligned} Q_\psi(x, s, y, s') &\iff \exists x_1, x_2, y_1, y_2, s_{\text{fork}}, s_1, s_2 : Q_{p_{\text{fork}}}(x, s, (x_1, x_2), s_{\text{fork}}) \\ &\quad \wedge Q_{\psi_1}(x_1, s_{\text{fork}}, y_1, s_1) \\ &\quad \wedge Q_{\psi_2}(x_2, s_1, y_2, s_2) \\ &\quad \wedge Q_{p_{\text{join}}}((y_1, y_2), s_2, y, s'). \end{aligned} \quad (99)$$

This relation specifies the combined task before any success measurement. These conditions admit the serial fork-join constructor. Shared mutable interactions require additional compatibility rules, and parallel execution retains the separate trace-preservation obligation.

Task Interpretation and Root Binding. For the family construction, a root expression has a declared task-state projection $\theta_\psi : \mathcal{S} \rightarrow \mathcal{S}_\psi$ through which its root contract factors:

$$P_\psi(x, s) = \bar{P}_\psi(x, \theta_\psi(s)), \quad Q_\psi(x, s, y, s') = \bar{Q}_\psi(x, \theta_\psi(s), y, \theta_\psi(s')).$$

The projection includes every component relevant to the root specification. Controller-private bookkeeping can be excluded only when the root contract does not depend on it. For each admitted root, fix an input/reference law μ_ψ on $(x_{\text{ref}}, u_{\text{ref}})$ supported on $\bar{P}_\psi(x_{\text{ref}}, u_{\text{ref}})$. The task $t_\psi = (r_\psi, \mu_\psi, E_\psi, V_\psi)$ requests relation Q_ψ on the supplied input, uses the specified primitive-tool environment E_ψ , and evaluates the delivered output y and final state s' recorded in Y by

$$V_\psi((x_{\text{ref}}, u_{\text{ref}}), Y) = \bar{Q}_\psi(x_{\text{ref}}, u_{\text{ref}}, y, \theta_\psi(s')).$$

Aborted and nonterminating records have value zero. The reference field of the analytical state carries $(x_{\text{ref}}, u_{\text{ref}})$.

Planning must preserve this reference meaning. A correctly bound entry has $\theta_\psi(\Sigma_0) = u_{\text{ref}}$, and its authorized packet's input field is x_{ref} . Thus planning may change private bookkeeping but may not change the task state on which the requested relation depends. Necessary task-state mutations belong in explicit primitive operations.

Compilation with Explicit Interfaces. Write $\text{Compile}(\psi, \iota)$ for compilation with a resolver ι that maps the authorized packet and declared interface messages to $\mathcal{X}_\psi$. It returns a graph with one entry and one exit. At the top level, $\iota(d) = x_{\text{ref}}$ is the input field of the authorized packet d . The notation $\text{Compile}(\psi)$ uses this root binding. For a primitive, its single node has $\Phi_p = \iota$. All embedded subgraphs receive fresh identifiers, and every identifier reference is renamed consistently.

For $\text{Seq}_\phi(\psi_1, \psi_2)$, compile ψ_1 with ι , add a handoff from its exit carrying $m = \phi(y_1)$, and compile ψ_2 with the resolver that reads that message. The child entry consumes m , not the parent's original packet input. For $\text{Par}_{p_{\text{fork}}, p_{\text{join}}}(\psi_1, \psi_2)$, the fork node receives the input resolved by ι and returns (x_1, x_2) . Two handoffs carry the respective projections, and each child's resolver reads its corresponding message. Two exit handoffs carry y_1, y_2 , and the join's resolver constructs the ordered pair (y_1, y_2) . The join is the new exit. Edge transformations and node assembly are distinct charged stages, as in Eq. (5). All added transfers preserve mutable task state on a successful transition.

Fragment contracts take their input from this explicit interface. Child invariants use the corresponding input value and entry state. They are not reused with the parent's original input. Providers access only declared packet fields and messages. The ledger-extension property in Appendix A.2 permits a fragment to execute with unrelated parent or sibling records present. Instance bindings must separately satisfy its state footprint because fresh names alone do not isolate memory.

The serial schedule is recursive, with a primitive call as its base case. A sequence executes the first child, the connecting handoff, then the second child. A parallel expression executes the fork, outgoing handoffs, first child, second child, incoming handoffs, then the join. Fresh subgraphs and these connections preserve acyclicity. Only the whole plan's exit performs final delivery, while internal exits return to their parent. Root-delivery work belongs to the exit's allocation and provider model.

Let $\ell_{\text{root}}(x, u)$ contain the correctly bound packet and the immutable reference record, with all compiled identifiers initially fresh. For a reference (x, u) define

$$\mathcal{I}_\psi(x, u) = \{(s, \ell_{\text{root}}(x, u)) : \text{ref}(s) = (x, u), \theta_\psi(s) = u, \bar{P}_\psi(x, u) = 1\}.$$

Allowed unrelated planning records may be added by ledger extension. The terminal projection extracts the actual delivered result and state. It cannot replace the original reference by a post-planning one.

Proposition 4 (Sound compilation of the task grammar). *For an admitted root expression ψ , its serial compilation is a finite plan compatible with t_ψ and every $\mathcal{I}_\psi(x, u)$. For every compiled fragment, each nonterminal successful operation prefix enables its next operation. Complete successful execution establishes the fragment's declared input/output relation.*

Proof. Induct on the syntax tree, retaining the fragment input, its entry state, and its interface binding. The induction assertion includes internal enabling conditions, postcondition correctness, and preservation under unrelated ledger extension.

For the base case, a primitive's resolver supplies its declared input. Its precondition enables the node and its transition establishes the primitive postcondition. Freshness and the record definitions give ledger extension.

For a sequence, apply the first child's induction assertion. Every internal successful prefix enables its next step. On completion it gives $Q_{\psi_1}(x, s, y_1, s_1)$. The transfer is enabled by its source record, and its success preserves s_1 and delivers $\phi(y_1)$. Equation (93) enables the second child's explicitly rebound entry. Apply its full induction assertion, including all its internal prefixes. The actual intermediate (y_1, s_1) witnesses Eq. (94).

For a fork-join expression, the fork establishes both branch preconditions by Eq. (97). The outgoing transfers deliver the respective inputs without changing the task state. Apply the first child's assertion. Its frame preserves the second precondition, so the second child can be invoked with its own bound input. Its assertion gives all internal enabling steps and Q_{ψ_2} . The exit records enable the two incoming transfers, and their ordered messages satisfy the join precondition by Eq. (98). The successful join gives $Q_{p_{\text{join}}}$. The actual intermediate states and outputs witness Eq. (99). Disjoint ledger identifiers and the extension property justify each child invocation.

For each resulting graph, take I_k to express reachability from its declared initial set through the first k transitions satisfying D_j . The prefix assertions just proved establish Eq. (8). Appending a specified

transition establishes Eq. (9). The final relation gives $Q_\psi(x, s_0, y, s') = \bar{Q}_\psi(x, u, y, \theta_\psi(s'))$ because $\theta_\psi(s_0) = u$. This is $\forall_\psi = 1$, establishing Eq. (10). Finite syntax yields a finite graph. $\square$

This proposition proves semantic compatibility for serial execution. Extension to a parallel implementation requires the trace-preservation condition in Appendix A.2, beyond the graph structure and state footprints.

Resource and Error Recurrences. To account for resource use and error in these compiled plans, let $n_{\text{node}}(\psi)$ and $n_{\text{edge}}(\psi)$ count expanded nodes and handoffs. Let $\text{cost}(\psi)$ be the sum of their allocated costs, excluding host planning, and let $\text{err}(\psi)$ sum their conditional error allocations. For a primitive,

$$n_{\text{node}}(p) = 1, \quad n_{\text{edge}}(p) = 0, \quad \text{cost}(p) = b_p, \quad \text{err}(p) = \epsilon_p.$$

Here p denotes an instantiated primitive occurrence, and b_p and ϵ_p are that call's cost and conditional-error allocations. Repeated occurrences may carry different allocations. An occurrence at the whole plan's exit includes final delivery in its allocation and provider model, while an internal occurrence returns to its parent. For a sequence with connecting handoff allocation b_ϕ^{hd} and error ϵ_ϕ ,

$$n_{\text{node}}(\psi) = n_{\text{node}}(\psi_1) + n_{\text{node}}(\psi_2), \quad n_{\text{edge}}(\psi) = n_{\text{edge}}(\psi_1) + n_{\text{edge}}(\psi_2) + 1, \quad (100)$$

$$\text{cost}(\psi) = \text{cost}(\psi_1) + \text{cost}(\psi_2) + b_\phi^{\text{hd}}, \quad \text{err}(\psi) = \text{err}(\psi_1) + \text{err}(\psi_2) + \epsilon_\phi. \quad (101)$$

For a parallel expression, denote its four connecting handoff allocations by $(b_k^{\text{hd}}, \epsilon_k^{\text{hd}})$, $k = 1, \dots, 4$. Then

$$\begin{aligned} n_{\text{node}}(\psi) &= n_{\text{node}}(\psi_1) + n_{\text{node}}(\psi_2) + 2, \\ n_{\text{edge}}(\psi) &= n_{\text{edge}}(\psi_1) + n_{\text{edge}}(\psi_2) + 4. \end{aligned} \quad (102)$$

$$\begin{aligned} \text{cost}(\psi) &= \text{cost}(\psi_1) + \text{cost}(\psi_2) + b_{p_{\text{fork}}} + b_{p_{\text{join}}} \\ &\quad + \sum_{k=1}^4 b_k^{\text{hd}}, \end{aligned} \quad (103)$$

$$\begin{aligned} \text{err}(\psi) &= \text{err}(\psi_1) + \text{err}(\psi_2) + \epsilon_{p_{\text{fork}}} + \epsilon_{p_{\text{join}}} \\ &\quad + \sum_{k=1}^4 \epsilon_k^{\text{hd}}. \end{aligned} \quad (104)$$

The fresh-copy construction gives these equalities by counting each fork, join, and input/output transfer as an explicit operation.

However, these sums yield reliability bounds only if the declared contract realizations apply to every invocation law induced by the compiled graphs, conditional on planning and successful prefixes. Typing and structural induction do not establish this distributional premise. For example, a join input assembled from earlier outputs may have a different law from its standalone evaluation inputs. The stated entry-law membership condition must cover it.

An Independent Bounded Task Family. The primitive specifications, constructor rules, root projections, and laws defining t_ψ are fixed independently of observed agent success. For integers $n_0 \geq 1$ and $r_0 \geq 0$, let

$$\mathcal{T}_{n_0, r_0} = \{t_\psi : \psi \text{ is an admitted root, } n_{\text{node}}(\psi) \leq n_0, n_{\text{edge}}(\psi) \leq r_0\}. \quad (105)$$

The family has bounded graph size but may contain infinitely many tasks.

For a task t_ψ , define B_ψ as the (Z, Π) -measurable event that planning finishes within b_H , returns $\text{Compile}(\psi)$ up to consistent identifier renaming, and uses the declared serial protocol (or its certified parallel implementation). Correct root binding must hold throughout the conditional support of (Σ_0, L_0) : the packet supplies x_{ref} , $\theta_\psi(\Sigma_0) = u_{\text{ref}}$, references and fresh identifiers are preserved, and fragment interfaces are instantiated as specified. These are planning-pair properties. Future success is not included in B_ψ .

Corollary 3 (Coverage through correct compilation). *For every $t_\psi \in \mathcal{T}_{n_0, r_0}$, suppose $\mathbb{P}(B_\psi) \geq 1 - \eta_\star$, where $\eta_\star \in [0, 1]$. On every positive-probability planning pair in B_ψ , assume the provider realizations*

and kernel-consistency hypotheses of Lemma 2. Let node and edge allocations be bounded uniformly by $b_V^{\max}$ and $b_E^{\max}$ and their conditional errors by $\epsilon_V^{\max}$ and $\epsilon_E^{\max}$. If

$$b_H + n_0 b_V^{\max} + r_0 b_E^{\max} \leq B, \quad \eta_\star + (1 - \eta_\star)(n_0 \epsilon_V^{\max} + r_0 \epsilon_E^{\max}) \leq \delta < 1, \quad (106)$$

then $\mathcal{T}_{n_0, r_0} \subseteq \mathcal{C}_{S_H}(B, \delta)$.

Proof. Correct binding and Proposition 4 establish semantic compatibility on B_ψ . The resource envelope establishes feasibility, so $B_\psi \subseteq G_{t_\psi}$. Local realization and consistency supply the successful-prefix bounds. Let $\epsilon_\star = n_0 \epsilon_V^{\max} + r_0 \epsilon_E^{\max}$. The risk inequality implies $\eta_\star < 1$ and $\epsilon_\star < 1$. Thus Theorem 1 gives $q_{t_\psi}(\zeta, \pi) \geq 1 - \epsilon_\star$ on B_ψ . Averaging on that event,

$$p_{S_H}(t_\psi; B) \geq \mathbb{E}[\mathbb{I}[B_\psi] q_{t_\psi}(Z, \Pi)] \geq (1 - \eta_\star)(1 - \epsilon_\star) \geq 1 - \delta.$$

Apply the capability definition to each task. $\square$

The construction gives one sufficient way to meet the coverage conditions. Other planning outcomes may yield successful executions that increase the overall success probability beyond this lower bound. The general result Corollary 2 continues to apply to any host meeting its stated valid-plan and reliability premises.

B Method Implementation Details

This appendix records the interfaces, implementation defaults, and protocol provenance for Sections 3 to 6. Experimental configurations are described in Appendix C.

B.1 Notation Across Method Modules

The theory’s notation is collected in Table 6. Complementing this notation, Tables 7 and 8 define the main method-specific objects. Ordinary loop indices and finite-set dummy variables are local to their stated subsection. A font, superscript, or descriptive subscript that distinguishes an object type is part of its notation.

Table 7: Runtime and Memory Notation.

Symbol	Meaning
$G_{\text{rt}}, G_{\text{ann}}$	Submitted graph and its analytical expansion.
$\text{Tpl}_v, \text{Dep}(v), \text{Bind}_v$	Prompt template, dependency identifiers, and slot bindings.
$\text{Cfg}_v, \text{inst}_v$	Invocation capability overrides and stateful-instance handle.
$j_{\text{rt}}, z_{j_{\text{rt}}}(v)$	Scheduler-event index and node status at that event.
$x_v^{\text{task}}, x_v^{\text{base}}, x_v$	Rendered task, task plus memory paths, and final dispatched input.
$\text{Paths}_v, \text{MemPath}(u)$	Advertised ancestor-memory paths and a node’s reserved memory-file location.
L_a, g_v	Owner-partitioned library and a worker’s group-memory context.
$\text{Verd}_{r,a}^{(p)}, \text{id}_H$	Agent assessment at round/phase and its host author identifier.
$\text{Hist}, \text{seg}_i, \text{ep}_i$	Interaction stream, source segment, and episode narrative.
$\text{scene}_j, \mu_j, n_j, t_{j,\text{last}}$	Scene, centroid, member count, and latest timestamp.
$\text{Cell}_i, \mathbf{z}_i, \mathbf{z}_c$	Memory cell, episode embedding, and case embedding.
$\text{Rel}_i(q), \text{Rel}_f(q)$	Calibrated episode and fact relevance scores.
$k_{\text{group}}, k_{\text{verdict}}$	Per-library group recall and recent-verdict caps.
$k_{\text{mem}}, k_{\text{recall}}$	Backend memory-candidate and final adapter-result caps.
$\text{Prof}_u, \text{Segments}$	User profile and source-segment collection operator.
$\Delta_{\text{time}}, \epsilon_{\text{num}}$	Clustering time window and numerical stabilizer.
$c_{\text{RRF}}^{\text{mem}}, \alpha_{\text{mix}}$	Memory-retrieval rank damping and fact-score mixing weight.
$\text{MemContext}(q)$	Host archive and user recall rendered for query q .
$++, \text{Fence}_{B_{\text{char}}}$	Ordered concatenation and untrusted-context wrapper under a character budget.
$L_{\ell}^{\text{hit}}(q), \text{RRF}_i(q), d_{\text{emb}}$	Channel hit list, episode expansion priority, and embedding dimension.

B.2 Graph and Prompt Contracts

Table 9 specifies the fields exchanged at the graph and model interfaces. Graph admission follows Table 1, and skill response handling follows Eq. (73). The case-to-skill operation format is defined in Section 6.3.

Table 8: Harness Evolution and Skill Notation.

Symbol	Meaning
$\text{id}_h, \text{Exec}(h)$	Candidate identity and executable components.
$K_{\text{att}}, N_{\text{keep}}, R_{\text{round}}$	Attempts per task, survivor cap, and round cap.
$\text{cell}, \text{desc}(h)$	Archive cell and the candidate’s assigned descriptor.
$\Pi_{\text{eval}}, \lambda_{\text{screen}}$	Fixed evaluation protocol and paired-gain screening threshold.
$\mathcal{X}_{\text{edit}}, \mathcal{K}_{\text{fixed}}$	Editable and protected harness locations.
Δ_{patch}	Executable harness patch.
Util, Log	Task-averaged utility and indexed evaluation record.
$\text{Log}_{h'}^{\text{sc}}, \text{Log}_h$	Fresh screening record and cached full-training record.
$\text{active}_{i,k}, \text{stale}$	Activation indicator and consecutive non-updating round count.
$\gamma_i, \tilde{\gamma}_r, t_{\text{pair}}$	Paired task gain, mean paired gain, and paired statistic.
$B_r, \perp$	Gene bank at round r and the value of an empty cell.
s, h	Skill record and source retrieval hit.
$\text{util}_s, \text{rob}_s, \text{safe}_s$	Normalized utility, robustness, and safety facets.
$\mathcal{U}_{\text{tool}}, \mathcal{A}_{\text{spec}}$	Receiving agent’s available tools and specialist roster.
qid, name	Qualified hit identifier and display-name extraction.
$n, L_j^{\text{hit}}, \text{Hits}_n$	Display-name value, source hit list, and hits sharing that name.
$\text{Pool}, \text{IDs}, S_{\text{sel}}$	Filtered candidate list, gate-returned identifiers, and selected entries.
$k_{\text{pool}}, k_{\text{sel}}, k_{\text{fb}}$	Fused skill-pool, normal selection, and fallback caps.
$c_{\text{RRF}}^{\text{skill}}, \mathcal{I}_{\text{src}}$	Skill rank damping and retrieval-source set.
$\text{RRF}(n), \text{Ops}, \text{op}_i$	Fused name score, skill-update operation list, and one operation.
$\text{own}(a), \text{Skills}_C, \text{Cases}_C$	Agent-memory partition, existing skill list, and supporting-case pool.
$\text{Catalog}_C, \text{UsedTargets}, \text{Emitted}$	Cluster skill collection, reserved update targets, and emitted records.
$\text{qual}_c, \text{qual}_{\text{cur}}(s), \chi(s)$	Case quality, catalog quality, and skill confidence.

Table 9: Graph and Model Interface Fields.

Interface	Fields and Constraints
Graph call	Required: nonblank <code>task_summary</code> and a <code>nodes</code> array. Optional booleans: <code>background</code> and <code>confirm</code> .
Node	Required nonblank strings: <code>id</code> , <code>subagent</code> , <code>node_summary</code> , <code>prompt_template</code> . Optional: <code>depends_on</code> (identifier array), <code>inputs</code> (map), and <code>instance</code> (nonblank string).
Input value	Exactly one literal string, <code>{"file": "path"}</code> , or <code>{"node": "id"}</code> . Reference objects have one key and a nonempty value. Template references and declared input keys must agree.
Node verdict	Required outcome: <code>accomplished</code> or <code>not_accomplished</code> . Optional negative-verdict fields: <code>category</code> , <code>what_is_missing</code> , <code>evidence</code> .
Verdict category	<code>missing_user_input</code> , <code>missing_credential</code> , <code>tool_failure</code> , <code>dependency_output_unusable</code> , <code>output_limit</code> , or <code>other</code> .
Node resolution	Requires <code>run_id</code> , <code>node_id</code> , and <code>decision</code> . Both <code>continue</code> and <code>replan</code> require a nonblank message. Replanning also requires a nonempty successor <code>nodes</code> array. <code>abandon</code> requires neither payload.
Query rewriting	JSON fields: <code>need_retrieval</code> (boolean) and <code>rewritten_query</code> (string or null).
Skill selection	JSON fields: <code>plan</code> (rationale string) and <code>skills</code> (array of source-qualified identifiers, possibly empty). The parser requires the array and permits an omitted rationale.

B.3 Implementation Defaults

Table 10 collects numerical settings used by the method implementations. EverOS defines the memory and case-to-skill defaults. Raven defines the defaults for graph execution, recall adaptation, and skill routing. Experiment-specific settings are described in Appendix C.

Table 10: Implementation Defaults. Harness-evolution run settings are separated from these defaults in Appendix B.4.

Mechanism	Setting	Value
DAG execution	Shared concurrency for graph nodes and spawns	8
DAG execution	Private fallback concurrency without a shared gate	5
DAG admission	Dispatch budget for graph runs and spawns	30 per hour
DAG verdict	Judge timeout / transcript-tail budget	180 s / 8,000 characters
DAG recovery	Additional continuations per node	2
DAG recovery	Decision deadline for a suspended node	600 s
DAG output	Terminal-output cap in the run report	128,000 characters
Clarification	Host autofill timeout	20 s
EverOS clustering	Cosine threshold / time window	0.65 / 7 days
EverOS user retrieval	RRF damping / facts per expansion / unchanged limit	60 / 3 / 10
Raven user recall	Result clamp / profile-text cap	1–100 / 1,200 characters
Raven user recall	Timeout / default result count	5 s / 5
Raven store queue	Session / global / in flight	64 / 256 / 4
Raven store retry	Attempts / waits of 2, 5, 15, 30 s	5
Skill rewriting	Request prefix	2,000 characters
Skill routing	Over-fetch factor / gate pool / ungated pool	2 / 10 / 2
Skill routing	Local / memory / SkillHub weights	1.0 / 0.9 / 0.85
Skill routing	RRF damping	10
Skill selection	Normal cap / failure fallback	2 / 5
Skill selection	Description / body-excerpt limits	200 / 300 characters
SkillHub detail policy	Minimum parseable safety score	0.7
Case admission	Minimum quality θ_{skip} for clustering	0.2
Case clustering	Recall cap / direct-merge similarity	30 / 0.85
Skill extraction	Existing-skill cap k_{skill} / supporting-case cap k_{case}	10 / 9
Skill extraction	Supporting summaries per existing skill	3
Skill extraction	Description / body prefix budgets	400 / 5,000 tokens
Skill extraction	Supporting-approach prefix budget	200 tokens

SkillHub Detail Policy. When discovery returns only metadata and a detail client is configured, the minimum safety threshold applies if the retrieved detail score is present and parseable. Missing or malformed scores do not trigger that rejection. Detail checks also reject blocked skills and incompatible external-home-path references. The corpus release predicate in Eq. (67) is applied separately. Token-prefix budgets in the table exclude omission markers.

B.4 Harness Evolution Protocol and Evidence

The seven benchmark results in Section 7.2 and Table 11 are reproduced from HarnessBank v2, Section 4 and Table 1 [39]. The reported experiments use a frozen Qwen3.6-27B task model, Claude Opus 4.8 as the evolver, and three attempts per task, with training-based selection followed by held-out testing. The method specification in Algorithms 3 and 4 states per-round sampling, cached parent records, invalid-run retries, descriptor assignment, ties, and zero variance explicitly. The published results do not establish the exact values of every run-specific setting.

The source paper does not provide a complete per-run manifest for the seven results. In particular, its result table does not identify each run’s screening subset and proposal order or all values of $(n, J, N_{\text{keep}}, R_{\text{round}}, P)$ and retry limits. Reconstructing those runs therefore requires their run records.

B.5 Source Prior for Skill Curation

For corpus source a , let n_a denote its skill count, $\overline{\text{qual}}_a$ its mean normalized *content* quality before the source-prior mixture, and $\text{lic}_a \in [0, 1]$ its license-coverage rate under the corpus policy. These source statistics belong to the curation snapshot. $\overline{\text{qual}}_a$ is not the final composite qual_{cur} . This

distinction avoids circular score computation. SkillCorpus uses

$$\text{track}(a) = \frac{n_a \overline{\text{qual}}_a + K_0 \mu_0}{n_a + K_0}, \quad (107)$$

$$p_{\text{src}}(a) = \text{clip}_{[0.30,1]}(0.95 \text{track}(a) + 0.05 \text{lic}_a + 0.10 \mathbb{I}[a \in \text{TrustedSrc}]), \quad (108)$$

with shrinkage pseudo-count $K_0 = 10$, prior mean content quality $\mu_0 = 0.685$, and $\text{TrustedSrc} = \{\text{anthropics}, \text{antigravity}, \text{karanb192}\}$. For an empty source, define $n_a \overline{\text{qual}}_a = 0$, so its track score reduces to μ_0 . The shrinkage estimate of empirical quality contributes most to the prior. The source reports that these settings were fixed before downstream evaluation and were not optimized using benchmark outcomes.

C Evaluation Details

This appendix supplements Section 7 with scoring conventions, evaluation configurations, grading procedures, and cost accounting. It records departures from official benchmark protocols. The main-text figures report orchestration and specialist scores. The tables below reproduce the harness-evolution results, skill-library results, and skill ablation from their cited sources.

C.1 Multi-Agent Orchestration

Scoring Conventions. The metric definitions in Section 7.1.2 follow the DAGbench scorer used for these results. Domain folding drops unrecognized agent entries, flags dangling dependencies, removes within-domain edges, and unions duplicate inter-domain edges. When distinct domains interleave in an otherwise valid invocation graph, folding may introduce a cycle. The adapter marks this as a projection conflict. Node F1 remains defined, while Edge F1, POA, and Exact Match are unavailable for that record.

By contrast, a cyclic prediction without that projection-conflict flag receives zero Node F1 and Edge F1 and a false Exact Match indicator. Its POA is zero when a shared pair exists and unavailable otherwise. For ordinary acyclic predictions, no shared node gives POA zero and a single shared node gives POA one. Two empty edge sets have F1 one. These values follow explicit scoring conventions for degenerate cases.

Reference annotations can admit alternative edge sets, node folds, or pair orders. The scorer expands explicit edge alternatives and allowed node folds, evaluates each resulting reference, and selects the candidate with the largest lexicographic tuple (Node F1, Edge F1, POA). Unavailable order scores count as zero only for this selection, and ties retain the first candidate. An empty prediction is scored against the original reference without benefiting from node folds. Within a selected reference, Edge F1 uses the best accepted reduced edge set, while POA and Exact Match use the annotated accepted pair relations. With no such annotations the reference family is a singleton.

Each reported metric is the arithmetic mean of its defined values over scored multi-domain records. Unavailable values are omitted from that metric’s denominator, and an empty collection has no reported mean. Defined zero scores, including ordinary empty-delegation predictions, remain in the mean. Repeated attempts are separate records. Consequently, different metrics can have different denominators, and the nominal 140-task count alone does not determine them.

C.2 Harness Self-Evolution

Table 11 lists the values plotted in Figure 14.

Table 11: Held-out Pass@1 (%) before and after harness self-evolution with frozen Qwen3.6-27B. Scores and percentage-point gains are reproduced as reported by HarnessBank.

Benchmark	Initial	Evolved	Gain
Terminal-Bench-2	36.1	45.4	+9.3
LiveCodeBench	58.1	71.8	+13.7
Omni-MATH	54.3	66.0	+11.7
BrowseComp+	16.9	30.8	+13.9
GDPval	43.7	52.9	+9.2
AppWorld	41.3	56.7	+15.4
SWE-bench Verified	47.4	52.6	+5.1

C.3 Raven-Research

Benchmark Composition. DeepResearch Mixed combines questions from BrowseComp (English), FRAMES, Humanity’s Last Exam, and xBench-DeepSearch release 2510, which is in Chinese. Humanity’s Last Exam is evaluated in its text-only exact-match setting.

Grading, Tokens, and Cost. An LLM judge (GPT-5.6 Luna Pro at temperature 0) grades the short final answer of each system for semantic equivalence with the reference answer. Accuracy is the fraction of questions answered correctly, and questions without an answer, including timeouts, count as incorrect. Regrading the same answers changed accuracy by approximately 1.3 percentage points. Token counts are per-question means of provider-reported usage, with cache reads included in the input. For the DeepSeek-V4-Flash group, cost applies one DeepSeek list-price schedule to the recorded usage of all three systems. This gives the three systems a common pricing basis, although the resulting estimates differ from the amounts billed. By contrast, costs of the commercial services are not on a common basis: the Perplexity cost is measured, the Gemini cost converts the billed rate, and the MiroThinker cost applies its list price.

Systems and Budgets. Qwen3.6-35B-A3B and a 4-bit GPTQ build of Qwen3.5-397B-A17B are self-hosted on internal infrastructure with thinking enabled, and DeepSeek-V4-Flash is deepseek-v4-flash, accessed through the official DeepSeek API. MiroFlow [58] runs its main agent with a 150-turn limit and extracts its final answer with Qwen3.6-35B-A3B in all three groups, including the DeepSeek-V4-Flash group. DeepSeek-Harness [13] runs its default loop. The three local harnesses use Serper for search and Jina Reader for page retrieval, and each keeps its own loop, context policy, and sampling settings. Raven-Research was evaluated with the two web tools, at most 150 tool iterations, 16,384 output tokens per call, high reasoning effort, and a per-question limit of 90 minutes on DeepSeek-V4-Flash (120 and 200 minutes on the two self-hosted models). The three commercial services [42, 51, 19] ran end to end on the same questions with their own models and search. Each system answered each question once, between August 10 and August 18, 2026.

Comparability. Because the harnesses retain different turn limits, time limits, and context policies, these results provide an observational comparison without isolating the effect of individual harness components. The same questions were also used while the research flow was developed. None of the evaluated systems blocked access to public copies of the benchmarks. The absolute BrowseComp and Humanity’s Last Exam scores may therefore overestimate performance in an uncontaminated evaluation.

C.4 Raven-Code

Harness and Baselines. The self-run coding results were produced with AgentEval, an internal evaluation framework that runs each attempt in a disposable Docker container, records every model request through a usage proxy, and grades the result in a fresh container with each benchmark’s own grading code. AgentEval invokes Raven-Code as a single-turn command-line agent. Baselines were installed from their public packages: Claude Code through claude-agent-sdk, OpenCode, Hermes Agent, and DeepSeek-Harness through its SDK. Unless stated otherwise, each task was attempted once, attempts that failed for infrastructure reasons count as unsolved, and no task was excluded.

SWE-bench Pro. The evaluation uses the full public set of 731 tasks from 11 repositories with the official images and grading scripts. Generation is offline: the agent container has no network interface, model traffic passes through a Unix socket, and git objects that could reveal the reference fix are removed. Grading is online because some test suites install packages. Both systems use Qwen3.8-27B served by vLLM with a budget of 3,600 seconds per task.

SWE-bench Verified. The 500 tasks are graded with the upstream grading functions, with 3,600 seconds per task. In contrast to SWE-bench Pro, the agent container has network access during generation. The DeepSeek-V4-Flash systems run through the same AgentEval setup, and the Claude Code entry for Claude Opus 4.8 is the VALS.ai value for the Claude Code harness [61].

WorkBuddy-Code. Following the official protocol, each of the 80 tasks receives three attempts of up to 3,600 seconds, and its reward is the mean hidden-test pass rate over the attempts. The Claude Opus 4.8 baselines are official leaderboard values [59], and all other entries are self-run. The Raven-Code runs with DeepSeek-V4-Flash and Claude Opus 4.8 also blocked 35 code-hosting and search domains, a stricter setting than that of the other self-run systems.

SWE-Refactor. Each of the 20 stack migrations is scored from 0 to 100 by a three-stage procedure: a model-based migration audit that acts as a gate, an all-or-nothing behavioral test stage, and six

adversarial verification rounds. Each Raven-Code attempt had network access limited to the model gateway, and the reported score combines task results from several runs. For one task whose audit gate the benchmark maintainers acknowledged as unreliable, the audit was repeated for Raven-Code. The baselines are official leaderboard values at maximum reasoning effort [44]. Raven-Code also uses maximum reasoning effort with GPT-5.6 Luna. Reasoning effort is reported separately from the model name.

DataAgentBench. Following the leaderboard protocol, Raven-Code ran five trials per query with benchmark hints enabled. The official validator checked each answer. Pass@1 averages the per-query pass rates within each dataset and then averages the dataset scores. Pass@5 assigns one to a query if at least one of its five trials succeeds and zero otherwise, then uses the same dataset-level averaging. All 270 planned trials in this run were graded, so these quantities require no missing-trial convention for the reported run. The run took place on August 24, 2026, with a limit of 5,400 seconds per trial and 24 concurrent trials. It used Claude Opus 5 through OpenRouter as the main model, together with a data-agent tool set and DAB-specific checks. One semantic-mapping step also used Claude Haiku 4.5 and Claude Sonnet 5, so under the leaderboard’s categories the entry counts as a tuned-prompt system. The entry has not been submitted to the leaderboard maintainers.

C.5 Raven-Design

Harness and Baselines. The design results were produced with AgentEval, with one attempt per task, in containers with 2 CPUs and 8 GB of memory, in August 2026. The evaluated deck pipeline fixes slide coordinates and typography, and the agent fills the resulting layout slots. The visual-artifact results use the rendering and preview tools, Task State, and always-loaded design skills. Claude Code ran through `claude-agent-sdk` and Hermes Agent through the Hermes Agent Agent package, with built-in web search disabled for Claude Code.

PresentBench. Each task provides an instruction and background materials mounted read-only, and the agent must produce an editable PPTX file. The official evaluator converts the deck to PDF and its judge, Gemini 3 Flash, answers about 54 binary checklist items per task. In our runs the judge receives the rendered deck through a different file transport from the official evaluator. The public leaderboard entries were evaluated by the benchmark authors on all 238 tasks. However, not every Raven-Design and Claude Code configuration was evaluated on the full task set, and failed generations are excluded from the reported mean. For Raven-Design with GPT-5.6 Luna, 229 tasks produced a valid deck and 228 were scored, and that run did not use image generation or image search.

ArtifactsBench and GDPval. ArtifactsBench contributes its SVG generation tasks and its data-visualization dashboard tasks after the removal of tasks with known source defects. GDPval contributes its public gold subset, restricted to tasks without audio or video that do not require live web search. Agents run in multiple turns and declare their final files. An internal grader renders each deliverable and asks GPT-5.6 Luna to assess it. This differs from both benchmarks’ official protocols. For configurations generated with GPT-5.6 Luna, the generator and judge use the same model. For ArtifactsBench the grader captures timed screenshots, replays frozen interaction intents with no-action controls, and scores the official ten-item checklist from 0 to 100. For GDPval it scores each rubric item of the public release as met or not met and computes a weighted score from 0 to 100.

C.6 Raven-Oncall

AI4AI. The agent may modify only the training script of autoresearch. The data pipeline, tokenizer, held-out validation shard, and evaluation function remain fixed. Package installation is prohibited, and the parameter count may not exceed the starting model’s 50,332,176 parameters. Every training run has a budget of 300 seconds of single-GPU training time, and each campaign has 5 GPU-hours on a machine with two A800 80GB GPUs. BPB is measured over 20,971,520 tokens of the held-out shard, and each reported BPB is the mean over several seeds of the final configuration (5 seeds for Claude Code, 6 for Raven-Oncall with Claude Opus 5, and 10 with DeepSeek-V4.1-Flash). The two Raven-Oncall campaigns share one task statement, whereas Claude Code received a separately written statement with the same budget and constraints. Runtime is the wall-clock time of the whole campaign, including work outside the training runs, and cost is the campaign total. The Claude Opus 5 cost

of Raven-Oncall is the amount billed through OpenRouter, the DeepSeek-V4.1-Flash cost applies DeepSeek’s peak-hour prices, and the Claude Code cost applies the Claude Opus 5 API prices to its recorded usage.

AI4S. The 17 tasks cover engineering simulation, compute tuning, model training, and other scientific and operational workloads. Seven tasks concern engineering simulation. Three use CalculiX: an elastic-plastic cantilever limit-load problem, a contact problem requiring evidence that the specified penalty stiffness was retained, and a 25-minute limit-load task that tests whether the agent requests additional budget. Four use OpenFOAM: two dam-break runs with a divergent fixed step and a loose Courant limit, a transient dam-break case with a seeded viscosity error, and a steady turbulent bump case with approximately 880,000 cells.

Two compute-tuning tasks use Qwen3-8B for batch inference. One minimizes batch wall time, and the other prohibits truncation. Three model-training tasks use Qwen3-Embedding-0.6B on NFCorpus: fine-tuning, resuming an interrupted job, and continuing a partially completed campaign from a previous shift. The remaining tasks concern a two-dimensional heat equation with an analytic solution, BM25 parameter tuning on NFCorpus, a GitLab merge-request review bot and its pipeline, and two molecular simulations. The molecular tasks estimate the density and self-diffusion of liquid ethanol and the hydration free energy of phenol by thermodynamic integration.

Each task has a specified budget in wall-clock, core, or GPU minutes. Success is judged manually against the task statement and a task-specific rubric containing reference values and supporting evidence withheld from the agent. All systems received the same budgets and machines. For simulation and batch inference, Claude Code’s task statements also included machine addresses and job commands, which Raven-Oncall selects itself.

C.7 Skill Retrieval and Reuse

Table 12 lists the values plotted in Figure 23, and Table 13 lists the ablation. All values are reproduced as reported by SkillCorpus [64].

Grading and Accounting. SkillsBench reports the binary pass rate over all 87 tasks. The 4 tasks that do not build in the evaluation environment count as failures under both conditions. A GPT-4o judge, from a different model family than the evaluated backbones, grades GDPval. QwenClawBench uses its official configuration of automated checks and an LLM judge. Qwen3.5-397B-A17B runs as a 4-bit GPTQ build. Each backbone runs with its default inference settings, with greedy decoding for the Qwen models and the default thinking effort for Claude Opus 4.7. Both conditions of a cell share these settings. Absolute baseline scores are therefore not directly comparable with leaderboard values obtained under other configurations. Comparisons within each cell, however, retain the same inference settings. A pooled gain averages each task’s paired difference over the four cells, and its standard error divides the standard deviation of these averages by the square root of the number of tasks. This task-clustered error is more conservative than treating cell–task pairs as independent observations.

Frontier Check, Ablation, and Retrieval. The Claude Opus 4.7 check runs OpenClaw on SkillsBench once per condition. The ablation replaces exactly one component of the full pipeline in a single run. The standalone retrieval evaluation follows the SkillRouter protocol [81] on the 75 core SkillsBench tasks, which exclude 12 generic-only cases. Hit@1 is the fraction of these tasks for which the top-ranked skill belongs to the task’s designated ground-truth skill set under that protocol.

Table 12: Effect of SkillCorpus skills on each harness–backbone cell. Each benchmark gives the three-run mean without skills (None), with skills (Skills), and their difference in points. SkillsBench reports Pass@1 (%), and GDPval and QwenClawBench report mean rewards multiplied by 100. The last two rows give the task-clustered pooled gain with one standard error and its z -score.

Cell	SkillsBench			GDPval			QwenClawBench		
	None	Skills	Gain	None	Skills	Gain	None	Skills	Gain
Raven $\times$ Qwen3.5-27B	10.0	16.5	+6.5	82.6	83.8	+1.2	66.9	70.8	+3.9
Raven $\times$ Qwen3.5-397B-A17B	9.2	22.6	+13.4	84.0	85.2	+1.2	68.8	73.2	+4.4
OpenClaw $\times$ Qwen3.5-27B	8.8	13.0	+4.2	81.2	83.1	+1.9	65.2	66.7	+1.5
OpenClaw $\times$ Qwen3.5-397B-A17B	11.1	16.9	+5.8	82.2	84.0	+1.8	65.7	67.0	+1.3
Pooled gain		$+7.5 \pm 2.3$			$+1.51 \pm 0.49$			$+2.79 \pm 0.70$	
z -score		3.2			3.1			4.0	

Table 13: Single-run ablation on Raven $\times$ Qwen3.5-397B-A17B, SkillsBench Pass@1 (%) over 87 tasks. Each ablation replaces one component of the full pipeline, and the gain is measured against the no-skill baseline.

Catalog	Retrieval Stack	Pass@1	Gain
No skills		9.2	–
Curated	Fine-tuned	22.6	+13.4
Curated	Off-the-shelf Qwen3	13.8	+4.6
Raw crawl	Fine-tuned	14.9	+5.7

D Author List

Project Leader: Chuanrui Hu

Core Contributors: Dizhan Xue, Chuanrui Hu

Contributors: Zuyi Zhou, Hongda Chen, Xingze Gao, Zhao Wang, Pengfei Yao, Zhengwei Wu, Tong Li, Ethan Wang, Xiaotian Luo, Juwei Yue, Jie Huang, Hui Zhang, Weixiang Chen, Chang Zhang, Yuqi Yang, Yifan Chen, Yunyun Han

Corresponding Author: Yafeng Deng